orcAI Desktop

Техническое задание, сценарии использования, оценка текущей версии и макеты нового интерфейса

Настольное приложение для macOS поверх оригинальной orcAI 3.0.2: распознавание акустических сигналов косаток на спектрограммах, подготовка обучающих данных, обучение и дообучение моделей.

Основание: полный функционал оригинальной orcAI (ethz-tb/orcAI, DOI 10.1111/mms.70083). Любое расхождение с оригиналом считается дефектом документа.

документ 2.0 · 22.08.2026 · базовая версия приложения 0.1.8 · целевая 0.2 — 0.6

40 PNG (17 основных + 21 доп. + 2 схемы) · 141 требований (121 FR + 20 NFR) · 24 сценария использования

Содержание

Документ читается сверху вниз: требования → сценарии → оценка текущей версии → проектные решения и макеты → план работ.

№	Раздел	Что внутри	стр.
0	**Как читать документ**	Условные обозначения, границы ответственности, метод оценки	3
1	**Контекст, архитектура и границы**	Пользователи, слои приложения, языки интерфейса, перенос на Linux и Windows	4
2	**Источник правды: orcAI 3.0.2**	Модель данных, артефакты, 12 команд, параметры, инварианты	6
3	**Техническое задание**	121 функциональное и 20 нефункциональных требований по разделам	11
4	**Сценарии использования**	24 use case: от первого запуска до дообучения модели	21
5	**Оценка версии 0.1.8**	Матрица покрытия, сильные решения, дефекты и пробелы	26
6	**Принципы интерфейса**	Минимализм и скевоморфизм: материалы, компоненты, состояния	29
7	**Разделы: как есть → как будет**	Разбор и макет каждого раздела приложения	32
8	**Мастер обучения моделей**	Логика, шаги, валидации, ветка дообучения, монитор	53
9	**Дорожная карта**	Пять этапов с составом работ и критериями приёмки	69
10	**Риски и приложения**	Реестр рисков, соответствие ТЗ ↔ CLI, глоссарий, чек-лист приёмки	71

Кому адресован документ

Разработчику приложения — как ТЗ и план; научному пользователю — как описание того, что появится и почему; рецензенту — как способ проверить, что desktop-обёртка не искажает поведение оригинальной orcAI.

0. Как читать документ

Три вещи, без которых остальной текст читается неверно.

0.1 Условные обозначения покрытия

Каждое требование и каждый сценарий помечены состоянием в версии 0.1.8. Оценка сделана по живым экранам приложения и по журналу выполнения задач, а не по намерениям разработчика.

Знак	Состояние	Что это значит на практике
●	**Реализовано**	Функция доступна из интерфейса, работает на реальном проекте и соответствует поведению оригинальной команды orcAI.
◐	**Частично**	Функция есть, но недоступна часть параметров оригинала, либо результат не показывается пользователю, либо состояние отображается недостоверно.
○	**Нет**	В интерфейсе нет способа выполнить действие; при необходимости пользователю пришлось бы идти в терминал.

0.2 Границы: что делает приложение и чего оно не делает

orcAI Desktop — это оболочка. Вся предметная логика (расчёт спектрограмм, нарезка сниппетов, обучение, распознавание, фильтрация) выполняется неизменённой библиотекой orcAI 3.0.2, поставляемой внутри пакета приложения. Приложение отвечает за выбор параметров, порядок запуска, наблюдаемость и представление результатов.

Приложение отвечает за	Приложение не имеет права
Ввод и валидацию параметров до запуска команды; понятные значения по умолчанию.	Пересчитывать или «улучшать» результаты orcAI после её вызова.
Порядок вызовов, очередь задач, отмену и повтор, журнал и код возврата.	Подменять алгоритм спектрограммы своей реализацией отрисовки данных.
Показ артефактов: файлов предсказаний, метрик, чекпоинтов, таблиц.	Скрывать от пользователя параметры, влияющие на научный результат.
Перевод терминов и подсказки, объясняющие смысл параметров.	Менять имена файлов и структуру каталогов, которые создаёт orcAI.

0.3 Метод, которым составлено ТЗ

Отправная точка — исходный код оригинала: `src/orcai/cli.py` (состав команд и значения по умолчанию), модули `predict.py`, `snippets.py`, `train.py`, `test_models.py`, `helpers.py` (имена артефактов и правила их формирования) и файлы значений по умолчанию `defaults/`. Каждое требование ниже привязано к конкретной команде или параметру оригинала; требований «из головы», не имеющих опоры в orcAI, в документе нет.

> **Главное правило работы**
> Если проектное решение в интерфейсе противоречит поведению оригинальной orcAI — меняется решение, а не поведение библиотеки. Спорные места вынесены в раздел 10 «Риски» с указанием, какой ответ считается правильным.

1. Контекст, архитектура и границы продукта

Кому и для чего нужно приложение, из чего оно состоит и почему устроено именно так.

1.1 Пользователи и их задачи

Роль	Что делает сегодня	Что должен дать Desktop
Полевой исследователь	Собирает многочасовые записи гидрофонов, хочет узнать, где в них крики косаток.	Выбрать папку с WAV, нажать одну кнопку, увидеть метки на спектрограмме и выгрузить их в Audacity.
Аналитик-биоакустик	Сверяет распознавание с ручной разметкой, подбирает порог, отсекает слишком короткие срабатывания.	Порог и пределы длительности с предпросмотром результата; сравнение распознанного с аннотациями.
Исследователь-методист	Обучает и дообучает модели под свою популяцию, подбирает гиперпараметры.	Мастер обучения, честные статусы конвейера, монитор эпох, метрики теста в карточке модели.
Пользователь без терминала	Не может и не должен ставить Python, TensorFlow и зависимости.	Автономное приложение: движок и Python внутри пакета, установка перетаскиванием.

1.2 Продуктовые ограничения, принятые как данность

— Оригинал не форкается. Внутри приложения лежит неизменённая orcAI 3.0.2; обновление движка — замена пакета, а не правка кода.

— Автономность. Python и все зависимости внутри `.app`; внешние менеджеры пакетов не требуются.

— Ручной старт среды. Движок и sidecar запускаются пользователем — тяжёлый TensorFlow не должен стартовать сам при открытии окна.

— Локальность. Никакой отправки записей во внешние сервисы: данные не покидают компьютер.

— Два языка интерфейса. RU и EN; в русском predict называется «распознавание», имена файлов и классов на диске не переводятся.

— Окно и масштаб. Приложение работает в обычном и полноэкранном режиме; элементы и вёрстка корректно подстраиваются под разрешение и плотность пикселей экрана (NFR-17, NFR-18).

— Лицензия. orcAI Desktop распространяется под CC BY-SA 4.0 (автор — Vadim Romanov). Встроенный движок orcAI 3.0.2 — под той же лицензией; attribution авторам оригинала обязательно (FR-11.10, приложение Г).

— Качество кода. Python-часть приложения (sidecar и связанные модули) пишется и поддерживается по Clean Code и PEP 8; регрессии стиля ловятся ruff/flake8 (NFR-19). Стабильность UI и отсутствие утечек на длинных задачах — NFR-20.

1.3 Что считается успехом

Критерий	Проверяемый признак
Полнота относительно оригинала	Все 12 команд orcAI и все их значимые параметры доступны из интерфейса; ни один сценарий не требует терминала.

Критерий	Проверяемый признак
Отсутствие противоречий	Имена выходных файлов, структура каталогов и значения по умолчанию совпадают с оригиналом; интерфейс показывает точную команду, которая будет выполнена.
Понятность без документации	Научный пользователь проходит путь «папка с записями → метки на спектрограмме» без чтения руководства.
Наблюдаемость	В любой момент видно: что выполняется, сколько осталось, какие файлы уже созданы и где они лежат.
Обратимость	Любую задачу можно отменить; повторный запуск не портит ранее полученные результаты без явного разрешения на перезапись.

1.4 Визуальная основа

Проектные решения опираются на существующий интерфейс версии 0.1.8, а не на воображаемый. Тёмная «лабораторная» тема, левое меню из девяти разделов, нижняя консоль с журналом и инспектором, серая спектрограмма с цветными наложениями — всё это сохраняется. Меняются состав экранов, расположение элементов и полнота параметров, но не язык интерфейса.

1.5 Архитектура приложения

Приложение построено как четыре слоя над каталогом с данными. Разделение выбрано не ради красоты схемы: оно вытекает из главного ограничения продукта. Предметная логика живёт в библиотеке на Python с TensorFlow, переписывать её нельзя, а интерфейс должен оставаться отзывчивым, пока эта библиотека часами занимает ускоритель. Отсюда отдельный процесс службы выполнения и общение с ним по локальному HTTP.

Слой	Технологии	Ответственность	Почему именно так	При переносе
Интерфейс	`React, TypeScript, WebView`	Экраны, формы параметров, спектрограмма, очередь, консоль, переключение языка.	Не знает ни о Python, ни о файловой системе напрямую: только вызовы к оболочке.	Переносится без изменений
Оболочка приложения	`Tauri 2, Rust`	Окно и его поведение, системные диалоги, доступ к диску, запуск и остановка службы, выбор свободного порта, скрытие окна вместо закрытия.	Единственный слой, который знает, на какой операционной системе он работает.	Платформенные ветки
Служба выполнения	`Python, FastAPI, runner.py, SQLite, DuckDB`	Очередь задач, потоковый журнал, отмена, коды возврата; по обработчику на каждую из двенадцати команд оригинала; встроенные SQLite и DuckDB для служебного состояния и индексов.	Отдельный процесс: падение TensorFlow не роняет окно, а долгая задача не блокирует интерфейс.	Переносится без изменений
Движок	`orcAI 3.0.2, TensorFlow, librosa, zarr`	Вся предметная логика: спектрограммы, нарезка, обучение, распознавание, фильтрация.	Обычный пакет Python внутри сборки. Не форкается и не правится; обновление — замена пакета.	Переносится без изменений
Данные	`Каталог проекта на диске пользователя (CSV, TSV, zarr, Keras)`	Таблица записей, параметры, спектрограммы, датасеты, модели, предсказания.	Единственный источник правды для orcAI и CLI; базы данных — только индексы и служебное состояние UI.	Не зависит от платформы

> **Три следствия, важные для остального документа**
>
> Первое: любое действие пользователя рано или поздно превращается в вызов команды оригинала — поэтому в разделе 3 у каждого требования есть опора в CLI. Второе: служба работает на локальном адресе со свободным портом, выбираемым при старте, и наружу ничего не отдаёт — отсюда требование о полной локальности данных. Третье: движок остаётся обычным пакетом Python внутри сборки, поэтому обновление orcAI не затрагивает ни интерфейс, ни службу.

1.5.1 Встроенные базы данных

Помимо файлов orcAI на диске приложение использует два встроенных файловых движка без отдельного сервера баз данных. Оба работают в процессе службы выполнения и поставляются внутри пакета вместе с Python — так же, как библиотеки `sqlite3` и `duckdb` в зависимостях sidecar.

Движок	Назначение	Типичное содержимое	Где хранится
SQLite	Оперативное состояние: частые мелкие записи и транзакции.	Очередь задач и их журналы, реестр проектов, настройки интерфейса и среды, служебные флаги.	Каталог данных приложения (не в каталоге проекта orcAI).
DuckDB	Аналитика и быстрые выборки по табличным данным.	Индексы `recording_table.csv`, сводки детекций и метрик test, агрегаты для экрана «Результаты».	Рядом с проектом или в кэше приложения; пересоздаётся из CSV и TSV при расхождении.

Источник правды для движка и командной строки — только каталог проекта: `recording_table.csv`, файлы `*_predicted.txt`, zarr, датасеты. Базы данных нужны интерфейсу и службе: фильтры, сортировка, история задач, мгновенные счётчики на «Результаты». Если индекс расходится с CSV, приоритет у файлов на диске; sidecar переиндексирует DuckDB. Правки таблицы записей вне приложения не должны «ломать» совместимость с CLI.

> **Почему гибрид SQLite + DuckDB, а не одна база**
> SQLite надёжен для частых обновлений очереди и настроек. DuckDB силён на сканировании CSV и групповых агрегатах. Полный переход только на DuckDB теоретически возможен, но не заложен в целевую версию: риск регрессий по задачам выше, чем выигрыш для пользователя.

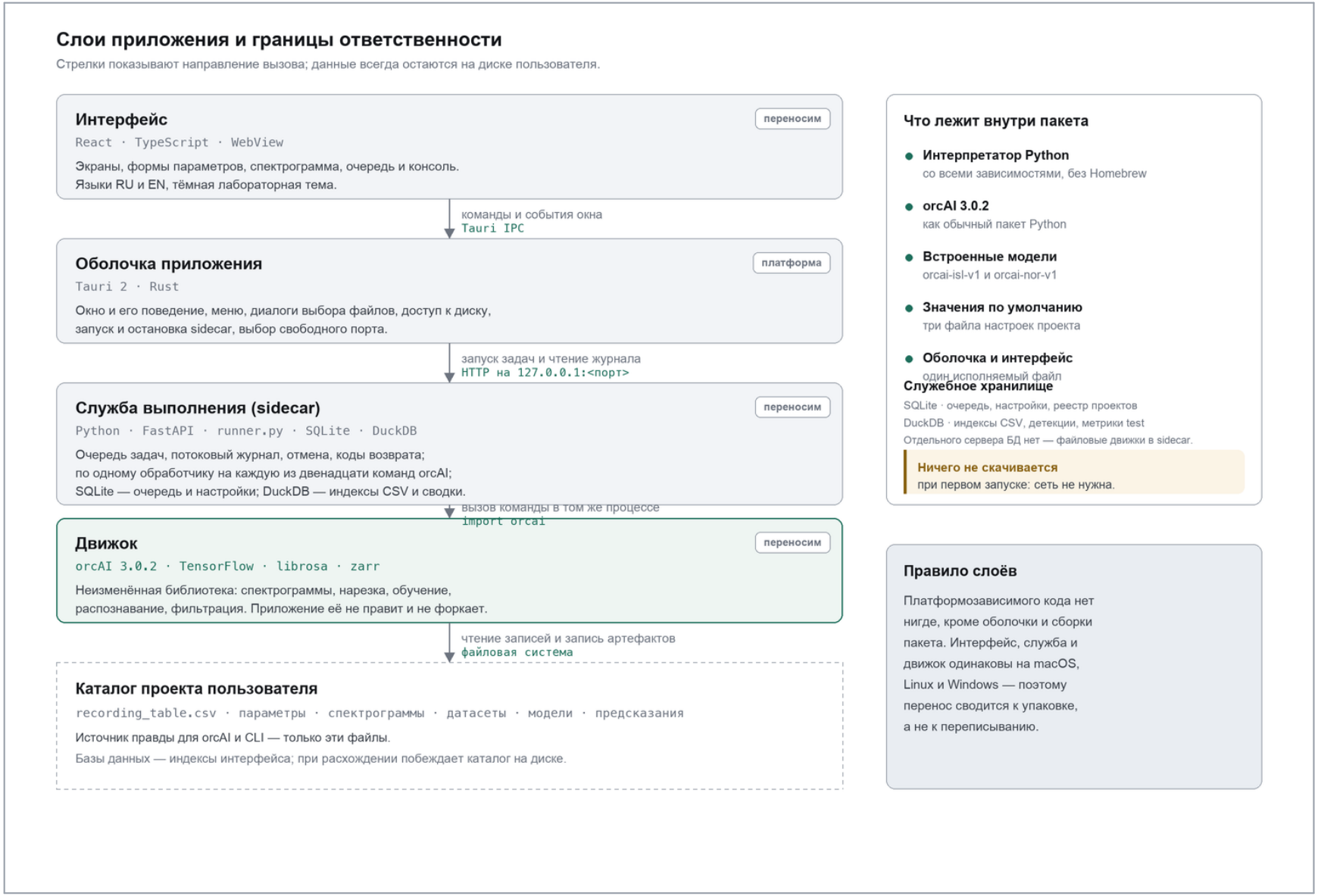


Схема 1. Слои приложения

Слева — цепочка вызовов от окна до файлов на диске с указанием способа связи между слоями. Справа — состав поставки, гибрид SQLite + DuckDB и правило переносимости: платформозависим только один слой из четырёх.

· Служба поднимается по команде пользователя, а не при открытии окна: TensorFlow стартует несколько секунд и занимает память.

- Порт выбирается динамически, поэтому занятый порт не мешает запуску.
- Обмен между шагами конвейера идёт только через файлы — так же, как в командной строке.
- SQLite — очередь и настройки; DuckDB — индексы CSV и сводки. При расхождении с файлами проекта первичны CSV и TSV на диске.

1.6 Языки интерфейса

Приложение работает на двух языках — русском и английском. Язык переключается в настройках и применяется сразу, без перезапуска; выбор запоминается между сеансами. Оба языка равноправны: не существует экрана, доступного только на одном из них.

Что переводится	Что не переводится никогда
Названия разделов, вкладок, кнопок и полей.	Имена классов сигналов: WHISTLE, BUZZ, PHS, SS, BR, HERDING, TAILSLAP.
Пояснения к параметрам и текст подсказок.	Имена файлов и каталогов, которые создаёт движок, включая суффиксы _predicted и _filtered.
Сообщения об ошибках — с сохранением исходного текста движка рядом.	Имена параметров в файлах настроек и ключи командной строки.
Состояния задач, шагов конвейера и проверок мастера.	Вывод движка в консоли: журнал показывается как есть.

Ключевое расхождение терминологии закреплено намеренно: команда оригинала называется `predict`, а в русском интерфейсе соответствующий раздел называется «Распознавание» — слово «предсказание» в биоакустике вводит в заблуждение. Чтобы это не мешало сверяться с документацией оригинала, соответствие терминов вынесено в глоссарий (приложение Б), а точная команда всегда видна в консоли и перед запуском.

> **Требование к переводу, которое легко упустить**
>
> Русские подписи в среднем на треть длиннее английских. Вёрстка экранов рассчитывается по русскому тексту как по худшему случаю: если подпись помещается по-русски, по-английски она поместится тем более. Обратный порядок приводит к обрезанным кнопкам, и это регулярная ошибка двуязычных интерфейсов.

1.7 Портирование на Linux и Windows

Версия 0.1.8 собирается только под macOS на Apple Silicon, но ни один слой архитектуры не содержит решений, привязанных к этой системе. Интерфейс, служба выполнения и движок переносятся без изменений: React работает в системном компоненте просмотра веб-страниц на всех трёх платформах, FastAPI и orcAI — обычные пакеты Python, а Tauri изначально кроссплатформенный каркас. Работа по переносу сосредоточена в двух местах: платформенные ветки в оболочке и полностью своя для каждой системы сборка пакета.



Схема 2. Готовность компонентов к переносу

Три верхних компонента переносятся без правок на обеих целевых платформах. Реальная работа приходится на упаковку, подпись и поддержку ускорителя обучения.

· Без DirectML базовый TensorFlow на Windows — CPU; DirectML в MSI — целевой GPU-путь (§1.7, П-18).

Что различается	macOS (есть сейчас)	Linux	Windows
Формат поставки	Пакет .app в образе .dmg, установка перетаскиванием	AppImage для запуска без установки и .deb для системных репозиториев	Установщик .msi или NSIS; портативная сборка как запасной вариант
Встроенный Python	Автономная сборка внутри пакета; каталог Python.app обязателен для запуска	Автономная сборка рядом с исполняемым файлом; следить за версией glibc	Автономная сборка; символические ссылки не работают, нужен запускающий exe-файл
Ускоритель обучения	Apple Silicon через Metal / tensorflow-metal — из коробки	NVIDIA GPU + CUDA/cuDNN, если драйвер и библиотеки доступны в пакете; иначе CPU. AMD ROCm — только если явно поддержана сборкой TensorFlow в поставке	Windows 10/11 x86-64: **DirectML** (tensorflow-directml-plugin или эквивалент) для GPU Intel/AMD/NVIDIA через DirectX 12. Без DirectML — CPU с явным предупреждением перед train. WSL2+CUDA — опциональный канал, не смешивать с базовым MSI.
Пути к файлам	Регистр в именах не различается; ограничений на длину нет	Регистр различается — записи «Whale.wav» и «whale.wav» станут разными строками таблицы	Ограничение в 260 символов снимается вручную; разделитель — обратная косая черта

Что различается	macOS (есть сейчас)	Linux	Windows
Поведение окна	Закрытие скрывает приложение, оно остаётся в Dock	Значка в трее нет по умолчанию; поведение задаётся окружением рабочего стола	Ожидается сворачивание в область уведомлений, а не в панель задач
Доверие системы	Подпись разработчика и нотаризация Apple обязательны	Подпись не требуется; достаточно контрольной суммы	Сертификат Authenticode: без него SmartScreen предупредит о неизвестном издателе
Звук и кодеки	Библиотеки чтения WAV входят в сборку Python	Нужно включить в пакет системную библиотеку чтения звука	Библиотеки чтения WAV входят в сборку Python

1.8 Правила, которые сохраняют переносимость

Перенос дешёв ровно до тех пор, пока платформенные допущения не расползлись по коду. Пять правил ниже соблюдаются с самого начала, даже пока существует только сборка под macOS: их стоимость сейчас близка к нулю, а стоимость возврата к ним потом измеряется неделями.

Правило	Что оно означает на практике	Что случится без него
Пути только через API оболочки	Никаких строк с косой чертой в коде интерфейса и службы; каталоги пользователя запрашиваются у Tauri.	Иначе перенос на Windows превращается в поиск сотен мест склейки путей.
Регистр имён считается значимым	Сопоставление записей и аннотаций по имени файла выполняется с учётом регистра уже сейчас.	На Linux иначе появятся дубликаты строк в таблице записей.
Служба общается только по HTTP	Никаких сигналов процессам и абсолютных путей к интерпретатору в коде интерфейса.	Управление процессами устроено по-разному на всех трёх системах.
Шрифты и метрики не привязаны к системе	Интерфейс не полагается на шрифты, которые есть только в macOS.	Иначе вёрстка поедет на первой же сборке под Linux.
Наличие ускорителя — свойство среды, а не платформы	Экран среды показывает найденный ускоритель и способ его использования, каким бы он ни был.	Позволяет не делать отдельных экранов под каждую систему.

Когда браться за перенос

Подготовка (П-15 / ЭК-2) начинается сразу после sidecar: пути, регистр, абстракция ускорителя. **Релизные сборки** Linux/Windows — после этапа 0.4, когда состав экранов стабилен. Последовательность: Linux первым (CUDA/CPU), Windows с **DirectML** в базовом MSI.

Ускорители: что обещать пользователю

Платформа	Типичный ускоритель	Что показывает UI	Оценка времени обучения
macOS Apple Silicon	Metal (tensorflow-metal)	«Apple GPU · Metal» / устройство из TF	По факту GPU; запас по RAM при больших batch.
Linux + NVIDIA	CUDA + cuDNN в составе пакета или системные	«NVIDIA · CUDA» и имя GPU, если обнаружен	По GPU; при отсутствии драйвера — CPU + предупреждение.

Платформа	Типичный ускоритель	Что показывает UI	Оценка времени обучения
Linux без NVIDIA	CPU	«Процессор · без GPU»	В разы дольше; мастер показывает порядок часов/суток.
Windows (базовый .msi)	DirectML (tensorflow-directml-plugin)	«DirectML · GPU» при успехе; иначе «Процессор» + предупреждение до train	GPU через DX12; без DirectML — CPU по явному согласию пользователя.
Windows + WSL2/CUDA (опц.)	GPU через WSL2	Отдельный профиль поставки или инструкция	Как на Linux CUDA; не смешивать с базовым MSI.

2. Источник правды: оригинальная orcAI 3.0.2

Всё, что приложение обязано уметь, и всё, что оно обязано не сломать.

orcAI — инструмент для обучения, проверки и применения нейросетевых моделей, обнаруживающих акустические сигналы на спектрограммах аудиозаписей. Командный интерфейс состоит из двенадцати команд, разбитых автором на три группы: распознавание, обучение и вспомогательные операции. В поставку входят две обученные модели: `orcai-isl-v1` (исландская популяция, модель по умолчанию) и `orcai-nor-v1` — это та же исландская модель, дообученная на норвежских данных в течение тридцати эпох. Обе работают с одним и тем же набором из семи классов.

2.1 Двенадцать команд, их аргументы и результат

Команда	Группа	Обязательные аргументы	Параметры (со значениями по умолчанию)	Что появляется на диске
`init`	Помощник	`PROJECT_DIR PROJECT_NAME`	`-p параметры`	Каталог проекта и три файла настроек: <имя>_orcai_parameter.json, <имя>_call_duration_limits.json, <имя>_hps_parameter.json
`create-recording-table`	Помощник	`BASE_DIR_RECORDING`	`-o -bda -dc 1 -p -ut -up -ep -rdf`	recording_table.csv: recording (ключ), channel, duplicate, base_dir_recording, rel_recording_path, base_dir_annotation, rel_annotation_path и по столбцу на каждый класс (пусто = класс замаскирован)
`predict`	Распознавание	`RECORDING_PATH (WAV или .csv)`	`-c 1 -m orcai-isl-v1 -md -o default -th 0.5 -ow -sp -bdr -cdl -ls * -v 2`	<запись>_c<канал>_<модель>_predicted.txt — TSV со столбцами start, stop, label, mean_p, label_source
`filter-predictions`	Распознавание	`PREDICTED_LABELS`	`-cdl default_call_duration_limits.json -o default -ow -ls * -v 2`	<имя>_filtered.txt рядом с исходным файлом; исходный файл не изменяется
`create-spectrograms`	Обучение	`RECORDING_TABLE_PATH OUTPUT_DIR`	`-bdr -p -en -enp -ow -v 2`	На запись: spectrogram/spectrogram.zarr, spectrogram/times.json, spectrogram/frequencies.json
`create-label-arrays`	Обучение	`RECORDING_TABLE_PATH OUTPUT_DIR`	`-bda -p -ce -ow -v 2`	На запись: labels/labels.zarr и labels/label_list.json
`create-snippet-table`	Обучение	`RECORDING_TABLE_PATH RECORDING_DATA_DIR`	`-o -p -v 2`	all_snippets.csv.gz, failed_snippets.csv в каталоге tvt_data
`create-tvt-snippet-tables`	Обучение	`OUTPUT_DIR`	`-st -p -uts -n_uts -ow -v 2`	train.csv.gz, val.csv.gz, test.csv.gz, при -uts test_unfiltered.csv.gz, сводки длительностей
`create-tvt-data`	Обучение	`TVT_DIR`	`-p -ow -ct GZIP -v 2`	train_dataset/, val_dataset/, test_dataset/, dataset_shapes.json, call_weights.json
`hpsearch`	Обучение	`DATA_DIR OUTPUT_DIR`	`-p -hp default_hps_parameter.json -pl -ct GZIP -v 2`	hps_logs/best_hyperparameters.json, hps_logs/all_trials.csv, лучший чекпоинт в <имя>/hps/

Команда	Группа	Обязательные аргументы	Параметры (со значениями по умолчанию)	Что появляется на диске
`train`	Обучение	`DATA_DIR OUTPUT_DIR`	`-p -ct GZIP -lm -lmf -v 2`	OUTPUT_DIR/<имя>/: <имя>.keras, orcai_parameter.json, model_shape.json, training_history.json
`test`	Обучение	`MODEL_DIR DATA_DIR`	`-tu -o -ct GZIP -v 2`	MODEL_DIR/test/: <датасет>_metrics.json, <датасет>_confusion_table.csv и две таблицы ошибок (true→pred и pred→true)

Что из этого критично для интерфейса

Имена выходных файлов формируются жёстко: предсказания — «имя записи + _c + номер канала + _ + имя модели + _predicted.txt», фильтрация добавляет «_filtered». Приложение обязано показывать эти имена, а не придумывать свои: по ним пользователь находит файлы в Finder и открывает их в Audacity. Исключение, о котором нужно знать: при пакетном распознавании с указанием каталога вывода номер канала в имя не попадает — файл называется «имя записи + _ + имя каталога модели + _predicted.txt». Если в проекте распознаются разные каналы одной записи, результаты перезапишут друг друга, поэтому приложение обязано предупредить об этом до запуска.

2.2 Параметры проекта: orcai_parameter.json

Один файл управляет и подготовкой данных, и архитектурой модели, и распознаванием. Он копируется в проект при инициализации, передаётся почти каждой команде обучения и сохраняется рядом с обученными весами — именно поэтому модель всегда знает, на каких параметрах спектрограммы её обучали.

Поле	Значение по умолчанию	Смысл
`name`	`orcai-v1`	Имя модели; используется как имя папки и файла весов
`architecture`	`ResNetLSTM`	Доступны ResNetLSTM и ResNet1DConv
`model.epochs / batch_size`	`30 / 64`	Максимум эпох и размер батча
`model.filters`	`[30, 40, 50, 60]`	Число фильтров по блокам свёрточной части
`model.kernel_size / dropout_rate`	`3 / 0.5`	Размер ядра и доля отсева
`model.lstm_units`	`128`	Размер рекуррентного слоя
`model.n_batch_train / val / test`	`3750 / 375 / 375`	Батчей в каждой выборке; вместе с batch_size задаёт её размер: 240 000 / 24 000 / 24 000 сниппетов
`model.learning_rate`	`1e-04`	Начальная скорость обучения
`model.EarlyStopping_patience`	`10`	Эпох без улучшения до остановки
`model.ReduceLROnPlateau_*`	`3 / 0.5 / 1e-07`	Терпение, множитель и нижняя граница скорости обучения
`model.call_weights / monitor`	`null / val_MBA`	Веса классов и отслеживаемая метрика
`spectrogram.sampling_rate`	`48000`	Записи приводятся к этой частоте
`spectrogram.nfft / n_overlap`	`512 / 256`	Окно БПФ и перекрытие
`spectrogram.freq_range`	`[0, 16000]`	Полоса частот, попадающая в модель
`spectrogram.quantiles`	`[0.01, 0.999]`	Нормировка яркости спектрограммы
`spectrogram.duration`	`4`	Длительность окна, с
`calls`	`BR, BUZZ, HERDING, PHS, SS, TAILSLAP, WHISTLE`	Классы модели; задаёт размер выхода сети
`snippets.segment_duration`	`200`	Шаг чтения записи, с
`snippets.snippets_per_sec`	`3`	Плотность нарезки
`snippets.snippet_duration`	`4`	Длительность обучающего сниппета, с
`snippets.fraction_removal`	`0.99`	Доля отбрасываемых пустых сниппетов

Поле	Значение по умолчанию	Смысл
`snippets.train / val / test`	`0.8 / 0.1 / 0.1`	Доли времени каждого сегмента, отдаваемые обучающей, проверочной и тестовой выборкам; сумма меньше 1 означает неиспользованные участки записи
`seed`	`0`	Зерно генератора для воспроизводимости

Форма входа сети выводится из этих значений, а не задаётся отдельно

Частота 48 кГц, окно 512 и перекрытие 256 дают шаг по времени; полоса 0–16 кГц даёт число частотных полос. Для встроенных моделей это 736 отсчётов по времени и 171 полоса на сниппет — именно эта форма записана в model_shape.json. Отсюда следует практическое правило: изменить любой из параметров спектрограммы означает пересобрать все данные и обучить модель заново; применить старую модель к новым спектрограммам нельзя.

2.3 Инварианты: то, что нельзя нарушить ни при каких улучшениях интерфейса

ID	Инвариант	Почему
INV-1	Спектрограмма в интерфейсе строится тем же кодом, что и спектрограмма для модели (make_spectrogram, полоса 0–16 кГц, квантили 0.01–0.999).	Иначе пользователь видит не то, что «видит» сеть, и делает неверные выводы о качестве распознавания.
INV-2	Набор классов модели фиксируется полем calls в момент обучения и определяет размер выхода сети.	Дообучение с другим набором классов невозможно: веса не совпадут по форме. Сама orcAI этого не проверяет — при дообучении она берёт из каталога модели только веса, а список классов читает из переданных параметров, поэтому расхождение проявится как сбой формы тензора уже в ходе обучения. Проверка целиком ложится на приложение.
INV-3	Параметры спектрограммы обученной модели неизменяемы при её применении и дообучении.	Модель обучена на конкретном представлении сигнала; смена nfft или полосы делает вход несовместимым.
INV-4	Имена и расположение артефактов задаёт orcAI, а не приложение.	Пользователи и внешние инструменты (Audacity, скрипты) опираются на эти имена.
INV-5	Фильтрация по длительности — отдельная операция над файлом предсказаний; исходный файл остаётся нетронутым.	Позволяет пересматривать пределы, не теряя исходное распознавание.
INV-6	Значения по умолчанию в интерфейсе совпадают со значениями по умолчанию CLI (порог 0.5, канал 1, суффикс *, verbosity 2, сжатие GZIP).	Одинаковые настройки должны давать одинаковый результат в приложении и в терминале.
INV-7	Записи приводятся к 48 кГц самой orcAI; приложение только предупреждает об этом.	Передискретизация — часть предметной логики, дублировать её нельзя.
INV-8	Порядок шагов конвейера обучения не переставляется и не сокращается.	Каждый следующий шаг читает артефакты предыдущего; пропуск ведёт к ошибке или к молча испорченным данным.
INV-9	Метки, отсутствующие в списке классов, игнорируются при построении массивов меток; свести их можно только через файл синонимов.	Это поведение оригинала; интерфейс обязан показать, сколько меток отброшено, но не менять правило.
INV-10	Приложение не изменяет содержимое встроенных моделей и не удаляет их.	Они входят в поставку и служат отправной точкой для дообучения.

3. Техническое задание

121 функциональное требование, сгруппированное по разделам приложения, и 20 нефункциональных. Приоритет: 1 — без этого продукт неполон, 2 — важно, 3 — желательно.

Как связаны требования и оригинал

Столбец «опора в оригинале» указывает команду или параметр orcAI, ради которого существует требование. Требование без опоры допустимо только в разделах «Среда выполнения» и «Интерфейс» — это то, чего в командной строке нет по определению (окна, очередь, язык).

3.1 Среда выполнения (FR-1)

Приложение автономно: движок и Python поставляются внутри пакета. Пользователь управляет запуском вручную и всегда видит состояние среды.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-1.1	Ручной запуск и остановка sidecar и движка orcAI с индикацией состояния каждого компонента.	–	1	●
FR-1.2	Python и orcAI 3.0.2 внутри пакета приложения; сторонние менеджеры пакетов не требуются.	–	1	●
FR-1.3	Порт sidecar подбирается динамически; занятый порт не блокирует запуск.	–	1	●
FR-1.4	Версии движка, Python и приложения видны пользователю.	`orcai --version`	2	●
FR-1.5	Закрытие окна скрывает приложение, не прерывая выполняемые задачи.	–	2	●
FR-1.6	Признак доступности ускорителя (GPU/Metal) отображается до запуска обучения.	–	2	○
FR-1.7	Автозапуск движка при старте приложения — отключаемая опция, по умолчанию выключена.	–	3	○

3.2 Проекты и записи (FR-2)

Проект — это каталог с файлами настроек, записями и артефактами. Приложение обязано поддерживать и создание проекта, и подключение уже существующего каталога.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-2.1	Создание проекта: каталог и три файла настроек, скопированных из значений по умолчанию под именем проекта.	`init`	1	◐
FR-2.2	Открытие существующего каталога проекта, в том числе созданного в командной строке.	–	1	●
FR-2.3	Построение и перестроение таблицы записей с отображением всех её столбцов.	`create-recording-table`	1	◐
FR-2.4	Обновление существующей таблицы с сохранением ранее внесённых столбцов и обновлением путей.	`-ut, -up`	2	○

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-2.5	Исключение файлов по списку шаблонов и удаление записей с повторяющимися именами.	`-ep, -rdf`	2	○
FR-2.6	Канал по умолчанию для всех записей и переопределение канала у отдельной записи.	`-dc, столбец channel`	1	◐
FR-2.7	Список записей с частотой дискретизации, разрядностью, длительностью и размером.	—	1	●
FR-2.8	Отображение наличия файла аннотаций для каждой записи и их общего количества.	`rel_annotation_path`	1	○
FR-2.9	Предупреждение о частоте дискретизации, отличной от 48 кГц, с указанием на передискретизацию.	`sampling_rate`	2	○
FR-2.10	Отдельный каталог аннотаций, если он не совпадает с каталогом записей.	`-bda`	2	○
FR-2.11	Несколько проектов с быстрым переключением активного.	—	2	●
FR-2.12	Просмотр и редактирование параметров проекта с проверкой корректности значений.	`orcai_parameter.json`	1	○
FR-2.13	Список артефактов проекта с размерами и датами: спектрограммы, метки, сниппеты, датасеты, модели.	—	2	○
FR-2.14	Мастер создания проекта с выбором мягкого фокуса: полевое распознавание, обучение модели, полный цикл. Выбор задаёт подсказки и быстрые действия, но не блокирует команды orcAI.	`init`	1	○
FR-2.15	Предпочтение фокуса хранится только в SQLite приложения; в каталог orcAI не записывается. При открытии CLI-проекта фокус выводится из артефактов или «полный цикл».	—	2	○
FR-2.16	Панель готовности данных проекта: записи, аннотации, распознавание, датасеты, модель — с индикаторами и подсказками «можно / нужно / нет данных».	—	1	○
FR-2.17	Блокировка действий только по отсутствию входных артефактов (как FR-7.3), а не по фокусу проекта; раздел «Обучение» при отсутствии аннотаций доступен с пояснением причины.	—	2	○

3.3 Распознавание: параметры и запуск (FR-3)

Центральный сценарий продукта. Все параметры команды predict должны быть доступны, а запуск — пакетным и наблюдаемым.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-3.1	Распознавание одной записи и пакетное распознавание по таблице записей.	`predict RECORDING_PATH`	1	●
FR-3.2	Выбор встроенной модели из списка.	`-m`	1	●
FR-3.3	Выбор произвольного каталога модели, включая обученную в проекте.	`-md`	1	◐
FR-3.4	Номер канала для одиночного файла; для таблицы канал берётся из неё.	`-c`	1	●
FR-3.5	Порог вероятности с шагом и отображением текущего значения.	`-th`	1	●
FR-3.6	Выбор каталога результата. Путь вывода всегда передаётся явно: режим «не сохранять» из справки CLI в коде 3.0.2 приводит к ошибке и в приложении не предлагается.	`-o`	2	◐
FR-3.7	Перезапись существующих результатов. Без неё одиночный файл вызывает ошибку, в пакете запись пропускается с записью в журнал; интерфейс показывает это как «пропущено», а не как успех.	`-ow`	1	◐
FR-3.8	Сохранение матрицы вероятностей отдельным файлом.	`-sp`	2	○
FR-3.9	Альтернативный базовый каталог записей для таблицы, созданной на другой машине.	`-bdr`	2	○
FR-3.10	Применение пределов длительности сразу при распознавании.	`-cdl`	2	◐
FR-3.11	Суффикс, добавляемый к именам классов в выходном файле.	`-ls`	3	○
FR-3.12	Уровень подробности журнала от 0 до 3.	`-v`	3	◐
FR-3.13	Очередь файлов с индивидуальным состоянием, прогрессом и возможностью отмены.	—	1	◐
FR-3.14	Показ точной команды orcAI, которая будет выполнена с текущими параметрами.	—	1	○
FR-3.15	Сохранение набора параметров как пресета проекта.	—	3	○

3.4 Просмотр результата на спектрограмме (FR-4)

Пользователь должен проверять распознавание глазами и ушами — это главный инструмент доверия к модели.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-4.1	Серая спектрограмма, построенная тем же кодом, что и вход модели.	`make_spectrogram`	1	●
FR-4.2	Наложения детекций, окрашенные по классу, поверх серой спектрограммы.	`predicted.txt`	1	●
FR-4.3	Навигация по сегментам записи и перемещение по времени.	`segment_duration`	1	●

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-4.4	Управление масштабом времени, контрастом и громкостью.	–	2	◐
FR-4.5	Воспроизведение выбранного участка с бегунком позиции.	–	2	◐
FR-4.6	Легенда классов с числом детекций и переключением видимости каждого класса.	`calls`	1	○
FR-4.7	Значение средней вероятности у каждой детекции.	`mean_p`	1	○
FR-4.8	Таблица детекций, синхронизированная с выделением на спектрограмме.	–	1	○
FR-4.9	Наложение ручных аннотаций для визуальной сверки с распознаванием.	`rel_annotation_path`	2	○
FR-4.10	Переключение между исходными и отфильтрованными предсказаниями без перезапуска.	`_filtered.txt`	2	○

3.5 Фильтрация по длительности (FR-5)

Отдельная команда оригинала со своим файлом настроек. В интерфейсе это должен быть редактор правил с предсказуемым результатом.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-5.1	Запуск фильтрации по файлу пределов длительности.	`filter-predictions`	1	●
FR-5.2	Табличный редактор пределов по классам с правилом default и пустой верхней границей.	`call_duration_limits.json`	1	○
FR-5.3	Предпросмотр: сколько детекций останется и сколько будет удалено по каждому классу.	—	1	○
FR-5.4	Результат сохраняется рядом как _filtered.txt; исходный файл не изменяется.	`output_file=default`	1	●
FR-5.5	Перезапись ранее отфильтрованных файлов.	`-ow`	2	○
FR-5.6	Пакетное применение ко всем файлам предсказаний проекта.	—	2	○
FR-5.7	Сохранение и загрузка набора пределов как файла JSON.	`-cdl`	2	○
FR-5.8	Распределение длительностей по классу как опора для выбора границ.	—	3	○

3.6 Результаты и экспорт (FR-6)

Сводный взгляд на весь проект: не «файл за файлом», а все детекции сразу.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-6.1	Сводная таблица детекций по всем записям проекта.	`predicted.txt`	1	◐
FR-6.2	Фильтры по файлу, классу и вероятности, поиск и сортировка.	—	1	○
FR-6.3	Сводка: число детекций, распределение по классам, средняя вероятность.	—	2	○
FR-6.4	Экспорт в CSV и в формат меток Audacity.	`TSV start/stop/label`	1	○
FR-6.5	Переход от строки таблицы к этому месту на спектрограмме.	—	1	○
FR-6.6	Переключение источника: распознанное, отфильтрованное, ручные аннотации.	—	2	○

3.7 Обучение: конвейер (FR-7)

Восемь шагов оригинала, доступных и по отдельности, и как единый прогон. Главное требование — достоверные состояния и видимые артефакты.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-7.1	Доска конвейера: шаг, команда, состояние, артефакт, время последнего запуска.	`8 команд обучения`	1	◐
FR-7.2	Состояния шага: не выполнено, выполняется, готово, устарело, нет данных, ошибка.	—	1	○
FR-7.3	Блокировка шага, для которого нет входных артефактов, с объяснением причины.	`порядок команд`	1	○

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-7.4	Построение спектрограмм с включением записей без аннотаций и перезаписью.	`-en, -enp, -ow`	1	◐
FR-7.5	Построение массивов меток с файлом синонимов классов.	`-ce`	1	◐
FR-7.6	Построение таблицы сниппетов и таблиц train/val/test.	`create-snippet-table, create-tvt-snippet-tables`	1	◐
FR-7.7	Дополнительный нефильтрованный тестовый набор заданного размера.	`-uts, -n_uts`	2	○
FR-7.8	Сборка датасетов с выбором сжатия.	`create-tvt-data, -ct`	1	◐
FR-7.9	Обучение модели с сохранением весов, параметров, формы и истории.	`train`	1	◐
FR-7.10	Продолжение обучения с весов существующей модели.	`-lm, -lmf`	1	○
FR-7.11	Тестирование модели, в том числе на нефильтрованном наборе.	`test, -tu`	1	◐
FR-7.12	Поиск гиперпараметров по файлу диапазонов.	`hpsearch, -hp`	2	◐
FR-7.13	Открытие каталога любого созданного артефакта из интерфейса.	–	2	○

3.8 Мастер обучения и монитор (FR-8)

Мастер не добавляет новой предметной логики: он собирает параметры, проверяет их совместимость и запускает штатную цепочку команд.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-8.1	Пять шагов мастера: сценарий, данные и классы, спектрограммы и сниппеты, модель, проверка и запуск. На доске конвейера — 8 шагов orcAI; план «с нуля» ставит в очередь 7 команд (без hpsearch).	`7 команд с нуля / 8 на доске`	1	○
FR-8.2	Три сценария: обучение с нуля, дообучение существующей модели, поиск гиперпараметров. Сценарий hpsearch требует готовых TVT-датасетов; приёмка UI — этап 0.6.	`train, -lmf, hpsearch`	1	○
FR-8.3	Выбор классов из фактически встреченных в аннотациях с числом примеров по каждому.	`calls`	1	○
FR-8.4	Предупреждение о классах, для которых примеров недостаточно.	—	2	○
FR-8.5	Все параметры спектрограммы и нарезки с исходными значениями и возвратом к ним.	`orcai_parameter.json`	1	○
FR-8.6	Все параметры модели и расписания обучения.	`model.*`	1	○
FR-8.7	При дообучении классы, параметры спектрограммы и архитектура наследуются от базовой модели и недоступны для правки.	`INV-2, INV-3`	1	○
FR-8.8	Сверка классов базовой модели с новыми данными: где примеров достаточно, где их нет.	—	2	○
FR-8.9	Предполётная проверка: данные, место на диске, корректность параметров, оценка времени.	—	1	○
FR-8.10	План запуска: перечень команд с оценкой длительности каждой.	—	2	○
FR-8.11	Запись выбранных параметров в файл параметров проекта перед запуском.	`orcai_parameter.json`	1	○
FR-8.12	Монитор обучения: эпоха, метрики, скорость обучения, прошедшее и оставшееся время.	`training_history.json`	1	○
FR-8.13	График метрики по эпохам с отметкой лучшего значения.	`monitor=val_MBA`	2	○
FR-8.14	Остановка обучения с сохранением последнего чекпоинта и возможностью продолжить позже.	`EarlyStopping, -lmf`	1	○

3.9 Модели (FR-9)

Модель — это каталог с весами и параметрами. Карточка модели должна отвечать на вопрос «можно ли применить её к моим данным».

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-9.1	Список встроенных моделей с описанием происхождения.	`orcai-isl-v1, orcai-nor-v1`	1	●
FR-9.2	Список моделей, обученных в проекте.	`train OUTPUT_DIR`	1	◐
FR-9.3	Карточка модели: файлы, классы, параметры спектрограммы, дата, размер.	`orcai_parameter.json, model_shape.json`	1	○
FR-9.4	Метрики последнего теста в карточке, включая разбивку по классам.	`test/*_metrics.json`	1	○

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-9.5	Назначение модели активной для распознавания.	`-m, --md`	1	●
FR-9.6	Переход из карточки к дообучению и к тестированию.	`--lmf, test`	1	○
FR-9.7	Проверка совместимости модели с проектом по классам и параметрам спектрограммы.	`INV-3`	2	○
FR-9.8	Импорт каталога модели извне и открытие каталога в файловом менеджере.	`--md`	2	○

3.10 Задачи и журнал (FR-10)

Очередь — это способ доверять приложению: пользователь видит, что происходит, и может вмешаться.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-10.1	Очередь задач с состоянием, прогрессом, временем начала и длительностью.	–	1	●
FR-10.2	Полный журнал выполнения выбранной задачи.	`вывод orcai`	1	●
FR-10.3	Инспектор с параметрами задачи в исходном виде.	–	1	●
FR-10.4	Отдельный список ошибок с переходом к проблемной задаче.	–	2	○
FR-10.5	Точная команда orcAI для задачи, пригодная для копирования в терминал.	–	1	○
FR-10.6	Отмена выполняющейся и удаление ожидающей задачи.	–	1	○
FR-10.7	Повтор неудавшейся задачи с теми же параметрами.	–	2	○
FR-10.8	Сохранение журнала в файл.	–	2	○
FR-10.9	История задач сохраняется между запусками приложения.	–	2	◐

3.11 Настройки, язык и справка (FR-11)

Немногочисленные, но обязательные настройки: то, что влияет на поведение всех команд.

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-11.1	Переключение языка интерфейса между русским и английским без перезапуска, с запоминанием выбора.	–	1	●
FR-11.2	Уровень подробности журнала, применяемый ко всем командам.	`-v`	2	○
FR-11.3	Каталог проектов, каталог журналов и путь к кэшу.	–	2	◐
FR-11.4	Поведение при закрытии окна и число параллельных задач.	–	2	○
FR-11.5	Тема оформления.	–	3	◐
FR-11.6	Справка с описанием разделов, ссылкой на репозиторий и на статью.	`docs/manual/`	2	◐
FR-11.7	Соответствие русских терминов именам оригинала доступно из интерфейса: «распознавание» — это predict.	`docs/manual/ (глоссарий)`	2	◐
FR-11.8	Масштабирование интерфейса по системному DPI: спектрограмма, таблицы и студия остаются читаемыми при 125 % и 150 %.	–	1	○
FR-11.9	Полноэкранный режим без потери сайдбара, консоли и контекстной строки; минимальный комфортный размер окна 1280×800.	–	1	○

ID	Требование	Опора в оригинале	Пр.	0.1.8
FR-11.10	В поставке LICENSE, NOTICE и THIRD_PARTY_LICENSES; attribution orcAI Desktop (Vadim Romanov) и orcAI 3.0.2 по CC BY-SA 4.0.	–	1	◐
FR-11.11	Формулировка «no endorsement»: ETH Zürich и авторы orcAI не поддерживают этот desktop-клиент.	–	1	○
FR-11.12	В поставке bundled руководство пользователя (Markdown ru/ en/) и вкладка «Справка → Руководство» с оглавлением и чтением глав.	`docs/manual/`	2	◐
FR-11.13	Кнопка «Справка» в chrome рабочих экранов (не в разделе «Справка») открывает главу/якорь руководства (manifest.json → help_topics).	–	2	○
FR-11.14	Язык руководства следует языку интерфейса (FR-11.1) без перезапуска; юридические вкладки — отдельно от manual.	–	2	○

3.12 Нефункциональные требования

ID	Требование	Проверка
NFR-1	Приложение работает без внешних зависимостей. На macOS устанавливается перетаскиванием пакета .app; на Linux и Windows — своим форматом поставки (раздел 1.7).	Чистая система без Python и Homebrew: приложение запускается и распознаёт запись.
NFR-2	Первый экран появляется не позднее двух секунд после запуска; движок стартует по команде пользователя.	Секундомер на чистом профиле.
NFR-3	Интерфейс остаётся отзывчивым во время длительных задач.	Прокрутка и переключение разделов во время обучения.
NFR-4	Прогресс длительной задачи обновляется не реже одного раза в две секунды.	Наблюдение за конвейером на записи длительностью более часа.
NFR-5	Журнал задачи сохраняется полностью, включая вывод после аварийного завершения.	Принудительное завершение процесса движка.
NFR-6	Отмена задачи не оставляет частично записанных артефактов, выдаваемых за готовые.	Отмена на середине шага create-tvt-data.
NFR-7	Повторный запуск с теми же параметрами даёт тот же результат.	Совпадение файлов предсказаний побайтно при одинаковом зерне.
NFR-8	Ошибки движка показываются человеческим текстом с сохранением исходного сообщения.	Отсутствующий файл, занятый порт, нехватка места.
NFR-9	Ни один сценарий не требует ручного редактирования файлов вне приложения.	Прохождение всех сценариев раздела 4 только мышью.
NFR-10	Все подписи интерфейса переведены на оба языка; имена файлов и классов не переводятся.	Переключение языка на каждом экране.
NFR-11	Приложение не отправляет данные во внешнюю сеть.	Мониторинг сетевой активности во время полного прогона.
NFR-12	На каждой платформе при наличии ускорителя (Metal / CUDA / DirectML) он используется; иначе CPU с предупреждением до train.	Сравнение времени обучения с ускорителем и без на macOS, Linux и Windows.
NFR-13	Свободное место проверяется до запуска шагов, создающих крупные артефакты.	Запуск конвейера на почти заполненном диске.
NFR-14	Обновление движка не требует изменения проектов пользователя.	Замена версии orcAI при сохранении структуры каталога проекта.
NFR-15	Платформозависимый код существует только в слое оболочки и в сборке пакета; интерфейс, служба и движок одинаковы на всех платформах.	Сборка под Linux не требует изменений в коде интерфейса и службы.
NFR-16	Целевые платформы — macOS 12 и новее, Linux x86-64 и Windows 10 и новее; сценарии раздела 4 проходятся одинаково на каждой из них.	Полный прогон распознавания и обучения на всех трёх системах.
NFR-17	Интерфейс корректно масштабируется под разрешение и плотность пикселей экрана (логические пиксели, HiDPI).	1280×800, 1920×1080, масштаб 125 % и 150 %: без обрезанных кнопок и клиппинга спектрограммы.
NFR-18	Полноэкранный режим сохраняет доступ ко всем разделам и консоли.	Переключение Full Screen на каждом экране.
NFR-19	Python-код sidecar соответствует PEP 8; регрессии стиля ловятся ruff или flake8.	Линтер на каждом PR; без правок bundled orcAI.
NFR-20	UI остаётся отзывчивым при длительных задачах; нет утечек памяти на многочасовом train; отмена задачи освобождает ресурсы.	8+ часов train с переключением экранов; отмена create-tvt-data / train.

Сводка по объёму работ

Из 121 функционального требования в версии 0.1.8 полностью выполнено 24, частично — 26, не выполнено — 71. Основная масса невыполненного сосредоточена в трёх местах: просмотр и разбор результатов, редакторы параметров (пределы длительности, параметры проекта) и весь блок обучения — конвейер со честными статусами, мастер и монитор.

4. Сценарии использования

24 сценария покрывают весь функционал оригинала. Столбец «команды orcAI» показывает, во что сценарий превращается на уровне движка.

ID	Сценарий	Актор	Суть	Команды orcAI	0.1.8
UC-01	**Подготовить среду к работе**	Любой	Запустить sidecar и движок, убедиться, что версия и ускоритель определились	–	●
UC-02	**Создать новый проект**	Аналитик	Выбрать каталог и имя, указать мягкий фокус работы, получить файлы настроек проекта	`init`	◐
UC-03	**Открыть существующий проект**	Аналитик	Подключить каталог, созданный ранее или в терминале	–	●
UC-04	**Подключить записи и построить таблицу**	Аналитик	Указать каталог WAV, получить таблицу записей с каналами и аннотациями	`create-recording-table`	◐
UC-05	**Обновить таблицу после пополнения записей**	Аналитик	Добавить новые файлы, сохранив ранее заданные каналы	`create-recording-table -ut -up`	○
UC-06	**Распознать одну запись**	Исследователь	Выбрать WAV, модель и порог, получить файл меток	`predict`	●
UC-07	**Распознать все записи проекта**	Исследователь	Отметить файлы, запустить пакет, следить за очередью	`predict recording_table.csv`	◐
UC-08	**Проверить результат на спектрограмме**	Исследователь	Найти детекции глазами, послушать участок, увидеть вероятность	–	◐
UC-09	**Подобрать порог и перезапустить**	Аналитик	Изменить порог, перезаписать результаты, сравнить количество детекций	`predict -th -ow`	◐
UC-10	**Отфильтровать по длительности**	Аналитик	Задать пределы по классам, увидеть предпросмотр, получить _filtered.txt	`filter-predictions`	◐
UC-11	**Сохранить и переиспользовать пределы**	Аналитик	Сохранить набор пределов в JSON и применить в другом проекте	`-cdl`	○
UC-12	**Свести результаты по проекту**	Аналитик	Увидеть все детекции разом, отфильтровать по классу и вероятности	–	○
UC-13	**Выгрузить результаты**	Исследователь	Экспортировать CSV или метки Audacity для дальнейшей работы	–	○
UC-14	**Сверить распознавание с аннотациями**	Аналитик	Наложить ручную разметку на распознанное и найти расхождения	–	○
UC-15	**Диагностировать ошибку прогона**	Любой	Понять причину сбоя по журналу и повторить задачу	–	◐
UC-16	**Подготовить обучающие данные**	Методист	Пройти шаги от спектрограмм до датасетов, видя артефакты	`5 команд подготовки`	◐

ID	Сценарий	Актор	Суть	Команды orcAI	0.1.8
UC-17	**Задать параметры проекта**	Методист	Изменить спектрограмму, нарезку и классы с проверкой значений	`orcai_parameter.json`	○
UC-18	**Обучить модель с нуля через мастер**	Методист	Пройти пять шагов и запустить полную цепочку	`цепочка до train`	○
UC-19	**Следить за обучением**	Методист	Видеть эпохи и метрики, понимать, когда сработает ранняя остановка	`train`	○
UC-20	**Остановить и продолжить обучение**	Методист	Прервать прогон и позже продолжить с сохранённого чекпоинта	`train –lm –lmf`	○
UC-21	**Дообучить модель под свою популяцию**	Методист	Взять встроенную модель как основу и обучить на своих записях	`train –lmf`	○
UC-22	**Подобрать гиперпараметры**	Методист	Перебрать конфигурации и перенести лучшую в параметры проекта	`hpsearch`	◐
UC-23	**Протестировать модель**	Методист	Получить метрики на тестовой выборке, в том числе нефильтрованной	`test –tu`	◐
UC-24	**Применить обученную модель**	Исследователь	Выбрать модель проекта для распознавания новых записей	`predict –md`	◐

4.1 Ключевые сценарии подробно

Шесть сценариев разобраны по шагам: это те пути, по которым пользователь идёт чаще всего или где ошибка стоит дороже всего.

UC-07 Распознать все записи проекта	покрытие в 0.1.8: ◐
Актор	Исследователь
Предусловия	Проект открыт, таблица записей построена, движок запущен.
Основной поток	1. Открыть «Распознавание», вкладку «Параметры». 2. Выбрать модель; при выборе проверяется совместимость классов и параметров спектрограммы с проектом. 3. Задать порог; значение по умолчанию 0.5 совпадает с оригиналом. 4. При необходимости включить перезапись готовых результатов и пределы длительности на лету. 5. В правой панели отметить записи: все, только новые или только с ошибками. 6. Сверить строку команды orcAI внизу экрана и запустить. 7. Наблюдать очередь в консоли; при необходимости отменить отдельный файл.
Результат	Для каждой записи создан файл «имя_c<канал>_<модель>_predicted.txt» рядом с WAV; вкладка «Результаты» показывает число найденных детекций.
Команды orcAI	predict recording_table.csv -m <модель> -th 0.5 [-ow] [-cdl limits.json]
Отклонения	Если файл результата существует и перезапись выключена — запись помечается как пропущенная, а не как ошибка. Записи с частотой, отличной от 48 кГц, обрабатываются после передискретизации внутри orcAI.

UC-10 Отфильтровать распознанное по длительности	покрытие в 0.1.8: ◐
Актор	Аналитик
Предусловия	Есть хотя бы один файл предсказаний.
Основной поток	1. Открыть вкладку «Фильтрация». 2. Выбрать источник — конкретный файл предсказаний или все файлы проекта. 3. Задать минимальную и максимальную длительность по каждому классу; пустая верхняя граница означает отсутствие ограничения. 4. Оценить предпросмотр: сколько детекций останется и сколько будет удалено по каждому классу. 5. Применить фильтр; при необходимости сохранить набор пределов в файл JSON.
Результат	Рядом с исходным файлом появляется результат filter-predictions с суффиксом _filtered (как формирует orcAI 3.0.2, обычно …_predicted_filtered.txt); исходный файл не изменяется.
Команды orcAI	filter-predictions <файл>_predicted.txt -cdl limits.json [-ow]
Отклонения	Правило default применяется к классам, для которых не задано собственное. Повторное применение к уже отфильтрованному файлу требует явного разрешения на перезапись.

UC-18 Обучить модель с нуля через мастер	покрытие в 0.1.8: ○
Актор	Методист
Предусловия	В проекте есть записи с аннотациями; на диске достаточно места.
Основной поток	1. Открыть «Обучение» и нажать «Мастер обучения». 2. Шаг 1: выбрать сценарий «обучить с нуля». 3. Шаг 2: указать таблицу записей и каталог аннотаций, при необходимости файл синонимов меток; выбрать классы из фактически встреченных, отключив те, где примеров недостаточно. 4. Шаг 3: проверить параметры спектрограммы и нарезки сниппетов; мастер показывает оценку объёма и времени подготовки. 5. Шаг 4: задать имя модели и параметры обучения — архитектуру, эпохи, батч, скорость обучения, правила ранней остановки. 6. Шаг 5: пройти предполётную проверку и запустить; план из семи команд выполняется последовательно.
Результат	В каталоге моделей проекта появляется папка с весами, параметрами, формой модели и историей обучения; выбранные параметры записаны в файл параметров проекта.
Команды orcAI	create-spectrograms → create-label-arrays → create-snippet-table → create-tvt-snippet-tables → create-tvt-data → train → test
Отклонения	Если артефакты предыдущих шагов уже готовы и параметры не менялись — шаг пропускается. Если параметры нарезки изменились, зависящие шаги помечаются как устаревшие и выполняются заново.

UC-21 Дообучить модель под свою популяцию	покрытие в 0.1.8: ○
Актор	Методист
Предусловия	Есть базовая модель (встроенная или обученная ранее) и новые записи с аннотациями.
Основной поток	1. Открыть карточку базовой модели и нажать «Дообучить» — либо выбрать сценарий дообучения в мастере. 2. Мастер показывает унаследованные и заблокированные параметры: классы, спектрограмму, форму входа, архитектуру. 3. Указать новые данные; мастер сверяет классы базовой модели с новыми аннотациями и показывает, по каким классам примеров нет. 4. Задать имя новой модели, число эпох и скорость обучения — как правило, меньшую, чем при обучении с нуля. 5. Пройти проверку и запустить: подготовка данных выполняется по параметрам базовой модели, затем обучение продолжается с её весов.
Результат	Новая модель в каталоге проекта; исходная модель не изменяется.
Команды orcAI	create-* (по параметрам базовой модели) → train <данные> <выход> --load_model --load_model_from <базовая модель>
Отклонения	Попытка изменить набор классов или параметры спектрограммы блокируется: это не дообучение, а обучение с нуля, поскольку форма входа и выхода сети зафиксирована при первом обучении.

UC-08 Проверить результат на спектрограмме	покрытие в 0.1.8: ◐
Актор	Исследователь
Предусловия	Есть файл предсказаний для выбранной записи.
Основной поток	1. Открыть вкладку «Результаты» — запись открывается в просмотрщике. 2. Двигаться по сегментам записи; масштаб и контраст настраиваются на панели. 3. Скрыть лишние классы через легенду, оставив только интересующие. 4. Выделить участок и прослушать его; в таблице внизу видны детекции этого участка с вероятностями. 5. При необходимости включить показ ручных аннотаций и сравнить их с распознанным.
Результат	Пользователь понимает, какие срабатывания верны, и решает, менять ли порог или пределы длительности.
Команды orcAI	— (чтение predicted.txt и построение спектрограммы средствами orcAI)
Отклонения	Спектрограмма строится тем же кодом, что и вход модели; поэтому увиденное соответствует тому, по чему принимала решение сеть.

UC-15 Диагностировать ошибку прогона	покрытие в 0.1.8: ◐
Актор	Любой
Предусловия	Задача завершилась с ошибкой.
Основной поток	1. В консоли открыть вкладку «Ошибки» — список сбоев с указанием задачи и записи. 2. Перейти к задаче: журнал прокручен к месту ошибки, исходное сообщение сохранено полностью. 3. Во вкладке «Команда» скопировать точную строку orcAI, чтобы при необходимости воспроизвести запуск в терминале. 4. Устранить причину (например, отсутствующий файл или занятый диск) и нажать «Повторить».
Результат	Задача перезапускается с теми же параметрами; остальные задачи очереди не затронуты.
Команды orcAI	— (повтор ранее сформированной команды)

5. Оценка версии 0.1.8

Что уже сделано, что сделано наполовину, что придётся сделать. Оценка построена на живых экранах приложения и журнале реального прогона распознавания.

5.1 Покрытие требований по разделам

Столбцы «есть / част. / нет» — число функциональных требований FR с соответствующим покрытием в 0.1.8, а не число шагов конвейера или экранов.

Раздел	есть	част.	нет	Вывод	Итог
Среда выполнения	5	0	2	Сильная сторона версии: автономность и ручной старт работают как задумано.	●
Проекты и записи	3	3	11	Проект создаётся, но его содержимое почти не видно: ни таблицы записей, ни аннотаций, ни параметров.	◐
Распознавание	4	6	5	Основной путь работает; недоступна половина параметров команды и нет предсказуемости запуска.	◐
Просмотр спектрограммы	3	2	5	База отличная: серая спектрограмма с цветными наложениями и транспорт. Не хватает разбора результата.	◐
Фильтрация	2	0	6	Команда вызывается, но пределы длительности нельзя ни увидеть, ни изменить в интерфейсе.	◐
Результаты	0	1	5	Раздел присутствует в меню, но сводного взгляда на детекции и экспорта нет.	○
Обучение	0	8	5	Есть список шагов, но состояния недостоверны, артефакты не видны, мастера и монитора нет.	◐
Мастер обучения	0	0	14	Отсутствует полностью — крупнейший пробел относительно оригинала.	○
Модели	2	1	5	Встроенные модели описаны хорошо; обученные модели и метрики не представлены.	◐
Задачи и журнал	3	1	5	Журнал и инспектор — сильное решение; не хватает управления очередью.	◐
Настройки и справка	2	3	6	Язык и справка на месте; настроек, влияющих на команды, нет.	◐

Итого: 24 требования выполнено, 26 выполнено частично, 71 не выполнено. Приложение уверенно проходит основной путь «открыть проект → распознать → посмотреть спектрограмму», но останавливается там, где начинается разбор результата и работа с обучением.

5.2 Удачные решения, которые сохраняются

Это не формальная любезность: перечисленное ниже задаёт язык интерфейса, и новые экраны строятся по тем же правилам.

Решение	Почему работает	Что с ним делаем
Тёмная лабораторная тема	Даёт контраст для серой спектрограммы и не утомляет при долгой работе с записями.	Сохранить как основную; светлую тему добавить позже как опцию.

Решение	Почему работает	Что с ним делаем
Левое меню из девяти разделов	Соответствует ходу работы: проект → распознавание → обучение → модели → результаты. Порядок узнаваем.	Сохранить состав и порядок; добавить только индикаторы состояния разделов.
Нижняя консоль с журналом и инспектором	Редкое и очень уместное решение: пользователь видит настоящий вывод orcAI, а не пересказ.	Сохранить; добавить вкладки «Ошибки» и «Команда», сделать высоту регулируемой.
Серая спектрограмма, цвет только на наложениях	Цвет несёт смысл — класс сигнала; фон не соревнуется с данными за внимание.	Возвести в правило: цветом обозначается только принадлежность к классу и состояние.
Вкладки «Параметры / Результаты / Фильтрация»	Правильное разделение фаз работы внутри одного раздела.	Сохранить; наполнить содержимым — параметрами оригинала и разбором результата.
Карточка проекта со списком записей	Записи живут в проекте, отдельного раздела «Записи» не появилось — структура честная.	Сохранить; расширить таблицу столбцами аннотаций и состояния распознавания.
ЖК-индикаторы и приборная стилистика	Числовые параметры читаются мгновенно и выглядят как показания прибора, что уместно для измерительной задачи.	Расширить на порог, метрики обучения и сводку результатов.

5.3 Дефекты и пробелы

ID	Проблема	Важность	В чём вред	Решение
D-01	**Все шаги конвейера обучения помечены «ГОТОВО»**	Критично	Слово означает «можно запустить», а читается как «уже выполнено». Пользователь не понимает, что сделано, а что нет.	Шесть явных состояний с цветным индикатором и датой последнего запуска.
D-02	**Артефакты не видны**	Критично	После шага непонятно, что появилось на диске и где это лежит.	У каждого шага — имя артефакта, размер, время и кнопка «Открыть».
D-03	**Нет мастера обучения**	Критично	Обучение доступно только как ручной запуск восьми команд без проверки совместимости параметров.	Мастер из пяти шагов с предполётной проверкой (раздел 8).
D-04	**Пределы длительности недоступны**	Высокая	Фильтрация запускается, но правила скрыты в файле; результат непредсказуем.	Табличный редактор с предпросмотром «останется / удалится».
D-05	**Консоль занимает половину экрана**	Высокая	На разделе «Обучение» видно четыре шага из восьми при развёрнутой консоли.	Свёрнутая по умолчанию консоль со строкой состояния; высота регулируется.
D-06	**Половина параметров predict недоступна**	Высокая	Нет сохранения вероятностей, суффикса меток, альтернативного каталога записей, выбора каталога результата.	Полный набор параметров на вкладке «Параметры» (FR-3).
D-07	**Нет разбора результата**	Высокая	Детекции видны только как прямоугольники: нет их списка, вероятностей и счётчиков по классам.	Легенда с счётчиками и таблица детекций, связанная с выделением.
D-08	**Нет экспорта**	Высокая	Результат нельзя выгрузить, не идя в файловую систему.	Экспорт CSV и меток Audacity из раздела «Результаты».
D-09	**Задачу нельзя отменить или повторить**	Средняя	Ошибочный запуск на двенадцати часах записей приходится пережидать.	Отмена, удаление из очереди и повтор с теми же параметрами.
D-10	**Не видно, какая команда будет выполнена**	Средняя	Пользователь не может сверить свои настройки с оригиналом и воспроизвести запуск в терминале.	Строка команды на экранах запуска и вкладка «Команда» в консоли.
D-11	**Параметры проекта нельзя посмотреть**	Средняя	Файл параметров определяет всё поведение обучения, но в интерфейсе его нет.	Просмотр и редактирование с проверкой значений.
D-12	**Обученные модели не показываются**	Средняя	Раздел «Модели» перечисляет только встроенные; результата обучения в нём не видно.	Список моделей проекта и карточка с метриками теста.

Наблюдение, определившее приоритеты

Три из четырёх критичных дефектов относятся к наблюдаемости, а не к отсутствующим функциям: приложение уже умеет запускать почти всё, но не показывает, что именно произошло и в каком состоянии находится проект. Поэтому первый этап дорожной карты — это достоверные состояния, артефакты и разбор результата, а не новые команды.

6. Принципы интерфейса

Минимализм в составе элементов и скевоморфизм в их подаче: приборная панель, на которой нет ни одной лишней ручки.

6.1 Материалы

Интерфейс собран из четырёх материалов: фон, приподнятая карточка, утопленная поверхность и органы управления. Приподнятое — то, чем управляют; утопленное — то, что показывает данные. Это правило заменяет собой рамки и разделители.

Материал	Значение	Роль
Фон окна	`#0B0C0D`	Почти чёрный: даёт максимальный контраст спектрограмме.
Панель раздела	`#131415`	Подложка содержимого, отделённая от фона мягкой границей.
Карточка	`#1E1F21 → #171819`	Вертикальный градиент и светлая линия сверху — приподнятая поверхность.
Утопленная поверхность	`#0A0B0C`	Таблицы, поля ввода, спектрограмма, ЖК-индикаторы: тёмная заливка, тень сверху.
Основное действие	`#FBFBFB → #DCDCDD`	Белая кнопка с тёмным текстом — на экране всегда одна.
Вторичное действие	`#303134 → #232426`	Приподнятая тёмная кнопка со светлой кромкой.
Текст	`#E9E9EA / #9A9B9E / #6B6C6F`	Три уровня: основной, пояснение, служебная подпись.
Состояния	`#4CD07F / #E0B64A / #E26A6A / #63A6E8`	Готово, внимание, ошибка, выполняется.
Классы сигналов	`по одному цвету на класс`	Единственное место, где цвет несёт предметный смысл.

6.2 Компоненты

Компонент	Где применяется	Правило
Контекстная строка	Правый верх, все экраны	Проект, модель, движок. Всегда видно, к чему относится действие.
Вкладки-таблетки	Под заголовком раздела	Фазы работы внутри раздела; активная — белая таблетка. Счётчик в таблетке показывает объём.
Карточка	Группировка параметров	Один смысловой блок — одна карточка с заголовком-подписью прописными.
ЖК-индикатор	Числовые показания	Порог, позиция, длительность, метрики обучения. Только чтение или связка с регулятором.
Регулятор и шаговик	Непрерывные и целые значения	Ползунок для порога и масштаба, шаговик для канала и эпох.
Индикатор состояния	Списки шагов и задач	Светодиод плюс слово: цвет без текста не используется.
Утопленная таблица	Списки записей, детекций, задач	Заголовок приподнят, строки чередуются, выбранная строка помечена слева.

Компонент	Где применяется	Правило
Строка запуска	Низ экранов с действием	Слева команда orcAI, справа основная кнопка. Действие всегда в одном месте.
Консоль	Низ окна	Вкладки «Лог», «Инспектор», «Ошибки», «Команда». Свёрнутая показывает текущую задачу и прогресс.

6.3 Правила размещения

Правило	Как применяется
Один экран — одна задача	На экране одно основное действие и одна белая кнопка. Всё остальное — вторичные кнопки.
Действие внизу справа	Запуск всегда в правом нижнем углу рабочей области, рядом со строкой команды. Пользователь не ищет кнопку.
Параметры слева, объекты справа	Слева то, что настраивают, справа то, к чему это применяют: модель и порог слева, очередь записей справа.
Сначала состояние, потом действие	В списках сперва читается, что уже сделано, и лишь затем предлагается кнопка запуска.
Цвет только по делу	Класс сигнала и состояние. Оформительского цвета нет.
Скевоморфизм умеренный	Градиент, кромка, тень — не толще одного пикселя. Приборный характер создаётся точностью, а не объёмом.
Плотность вместо простора	Научный пользователь работает с сотнями строк: строка таблицы 26–32 пикселя, поля подписаны кратко.
Термины оригинала не переводятся	Имена классов, файлов и параметров остаются как в orcAI; переводится только пояснение к ним.
Окно живёт в любом размере	Базовый макет — 1280×800. Полноэкранный режим и большие мониторы растягивают панели пропорционально; HiDPI не ломает спектрограмму и LCD. Минимальная ширина сайдбара и высота консоли фиксированы в логических пикселях.

6.4 Чего в интерфейсе быть не должно

Антипаттерн	Почему
Слово-состояние без светодиода	«ГОТОВО» на всех строках конвейера читается как «выполнено». Состояние — это цвет плюс слово плюс дата.
Скрытые значения по умолчанию	Если параметр не показан, пользователь не знает, что порог 0.5, а сжатие GZIP. Значения по умолчанию видны всегда.
Модальные окна для длительных операций	Запуск не блокирует интерфейс: работа уходит в очередь, а пользователь продолжает работать.
Кнопка без последствий	Если действие недоступно, кнопка неактивна и рядом написано, чего не хватает.
Прогресс без содержания	Полоса без числа файлов и оставшегося времени бесполезна на многочасовых задачах.

> **Проверка любого нового экрана**
> Четыре вопроса: видно ли, к какому проекту и модели относится происходящее; понятно ли, что уже сделано; ясно ли, что произойдёт по нажатию основной кнопки; можно ли увидеть точную команду orcAI. Если хотя бы на один ответ «нет» — экран не готов.

6.5 Реализуемость макетов в приложении

Макеты сверстаны под окно `1280×800` — тот же размер, что и живые скриншоты 0.1.8. Хром (левое меню, белая таблетка активного пункта, консоль «Лог / Инспектор», чипы проекта и модели) совпадает с текущим приложением; добавлены только чип движка и строка предпросмотра команды orcAI.

Что на макете	Реализация	Оговорка
Chrome и токены	**100%**	Те же CSS-токены Lab: фон, карточки, wells, LCD, белая primary, pill-навигация.
Главная, Проекты, Параметры, Фильтрация, Результаты, Модели, Задачи, Настройки	**100%**	Плотность рассчитана при свёрнутой консоли. При развёрнутой консоли таблицы сохраняют прокрутку — макет не требует уменьшать контент.
Студия распознавания (спектрограмма + легенда + таблица)	**100% при свёрнутой консоли**	Таблица на макете — детекции текущего сегмента, не всего проекта. Полный список — в разделе «Результаты» или по кнопке «В таблицу». Спектрограмма только через make_spectrogram / display_kind=orcai (INV-1).
Конвейер обучения и монитор	**100%**	Восемь шагов и шесть состояний — React-список; артефакты читаются с диска проекта.
Мастер обучения (5 шагов + ветка дообучения)	**100%**	Stepper + формы; команда внизу собирается из состояния мастера. Дообучение проверяет совместимость классов до запуска (INV-2).
Одновременно студия + развёрнутая консоль + полная таблица проекта	**Не целится**	При развёрнутой консоли студия оставляет спектрограмму и легенду; таблица сворачивается или уходит в отдельный вид. Это осознанное упрощение плотности.

Верстка макетов относительно 0.1.8

Убрана вторая полноширинная панель под заголовком: бренд и «Справка» остаются в сайдбаре, как в живом 0.1.8. Контент лежит на тёмном поле без лишней обрамляющей «карточки окна». Чипы проекта / модели / движка — приподнятые, как нынешний бейдж проекта. Плотность студии и конвейера рассчитана на свёрнутую консоль; развёрнутая консоль — режим журнала, не режим макета.

7. Разделы приложения: как есть и как будет

Для каждого раздела: разбор текущего экрана, проектное решение и макет. Макеты выполнены в стилистике существующего интерфейса — меняется наполнение, а не язык.

Что не меняется ни в одном разделе

Состав и порядок пунктов левого меню; тёмная тема; нижняя консоль с журналом и инспектором; серая спектрограмма с цветными наложениями; ручной запуск движка. Изменения касаются наполнения экранов, полноты параметров и достоверности состояний.

Нумерация макетов

Подписи «Макет N» в этом разделе — порядок повествования в PDF (1…17). Имена файлов — m01…m38 плюс d01–d02 (всего 40 PNG); полный список в INDEX.md.

Макеты реализуемы в React Lab как есть

Все 38 макетов разделов (m01–m38) укладываются в существующий chrome 0.1.8 без смены языка интерфейса. Подпись v0.2.x на макетах — целевой chrome T3; код остаётся 0.1.8 до явного bump. Единственная оговорка по плотности: студия распознавания на макете показана со свёрнутой консолью; таблица детекций — по текущему сегменту. Подробности — §6.5.

Сводка изменений по разделам

Раздел	Главное изменение	Новые элементы
Главная	Экран запуска работы вместо простого статуса	Панель среды, карточка проекта с показателями, быстрые действия, последние задачи
Проекты	Проект как каталог артефактов, а не только список WAV	Вкладки «Записи / Аннотации / Артефакты / Параметры», столбцы аннотаций и распознавания
Распознавание · Параметры	Полный набор параметров команды и предсказуемый запуск	Очередь файлов, строка команды orcAI, сохранение пресета
Распознавание · Результаты	Из просмотрщика — в рабочее место разбора	Легенда кодов классов со счётчиками; типы — в таблице; вероятности, аннотации
Распознавание · Фильтрация	Правила стали видимыми и проверяемыми	Редактор пределов, предпросмотр «останется / удалится», распределение длительностей
Результаты	Сводный взгляд на проект целиком	Показатели, распределение по классам, фильтры, экспорт CSV и Audacity
Обучение	Достоверный конвейер вместо списка кнопок	Шесть состояний шага, артефакты, готовность данных, вход в мастер
Модели	Карточка модели с метриками	Модели проекта, файлы, результаты теста по классам, дообучение из карточки
Задачи	Управляемая очередь	Отмена, повтор, вкладки «Ошибки» и «Команда», экспорт журнала
Справка	Юридический и справочный дом приложения	Пять вкладок: Руководство (manual RU/EN), О Desktop, О orcAI (7 авторов), Лицензия, Сторонние

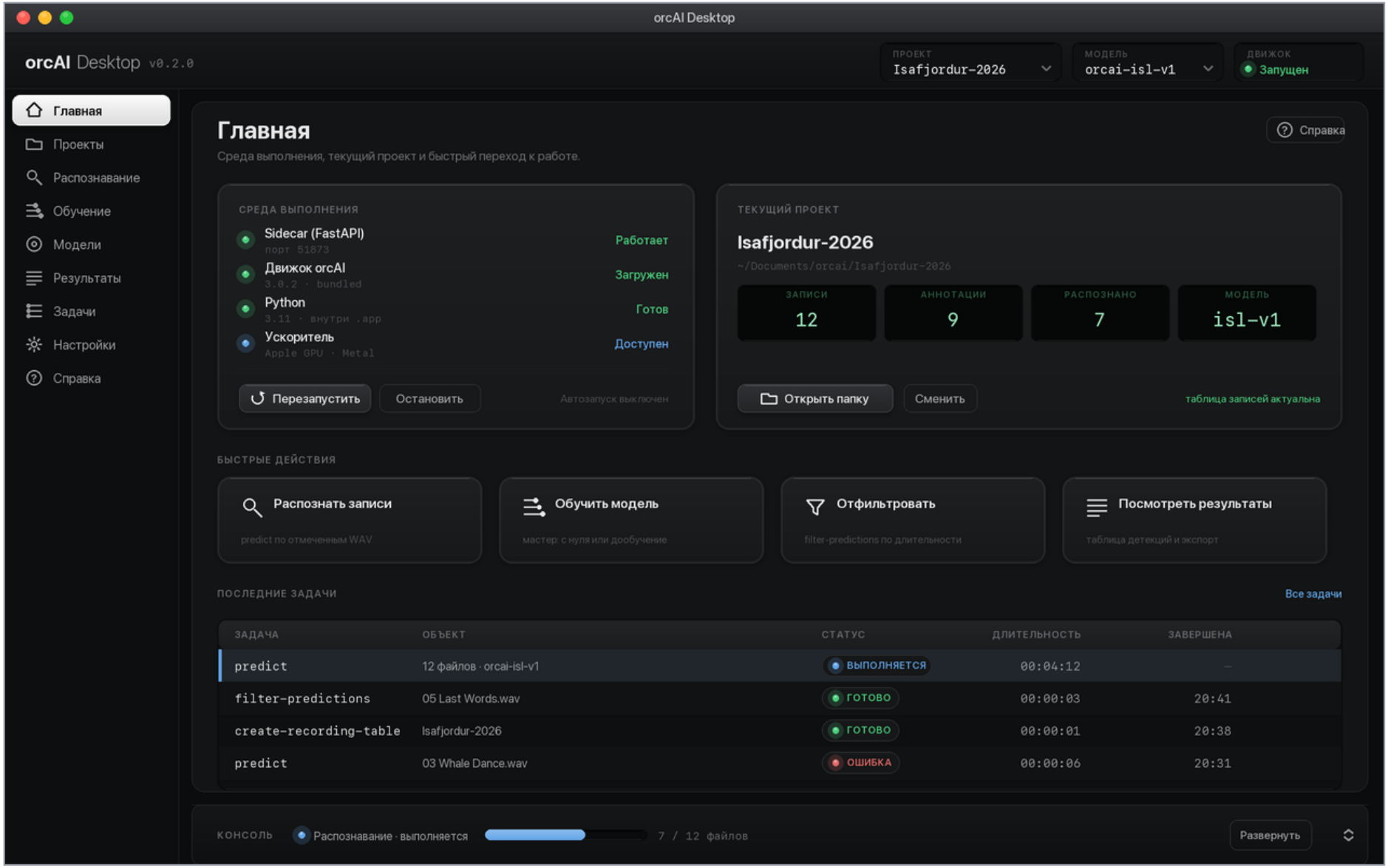
Раздел	Главное изменение	Новые элементы
Настройки	Настройки, влияющие на команды + масштаб/Full Screen	Подробность журнала, порт, DPI, полноэкранный режим, «О программе» → Справка

7.1 Главная

Сейчас экран сообщает, что среда запущена, и на этом останавливается. Он должен отвечать на три вопроса сразу: готова ли среда, с каким проектом мы работаем и что делать дальше.

Как сейчас	Что мешает	Что делаем
Состояние среды показано общей строкой.	Непонятно, что именно не готово, если запуск не удался.	Отдельная строка на компонент: sidecar, движок, Python, ускоритель — со светодиодом и версией.
Текущий проект не выведен на первый экран.	Пользователь не видит, к чему приведёт нажатие кнопок в других разделах.	Карточка проекта с показателями и панелью готовности данных (записи → модель).
Нет входа в основные сценарии.	Каждый путь начинается с блуждания по меню.	Четыре быстрых действия: распознать, обучить, отфильтровать, посмотреть результаты — с подсказками по состоянию каталога.
История запусков не видна.	После перезапуска приложения непонятно, чем всё закончилось вчера.	Последние задачи с состоянием и длительностью, переход в раздел «Задачи».

Требования: FR-1.1 – FR-1.7, FR-2.11, FR-2.16



Макет 1. Главная

Слева — среда выполнения с индикаторами по каждому компоненту и кнопками управления. Справа — карточка активного проекта с четырьмя показателями на ЖК-индикаторах и панелью готовности (m37-home-readiness.png). Ниже — быстрые действия и последние задачи с достоверными состояниями.

· Контекстная строка вверху справа появляется на всех экранах: проект, модель, движок.

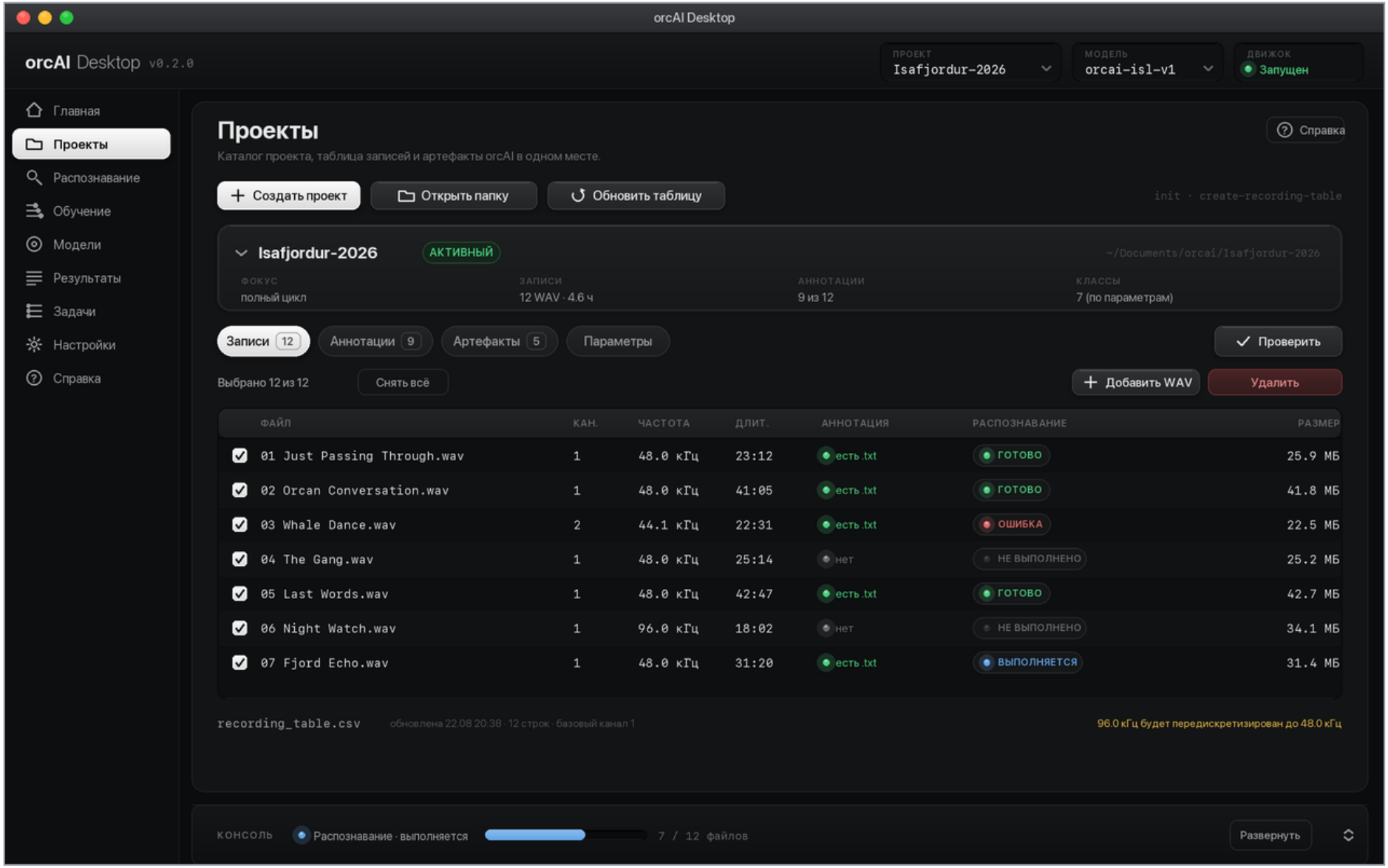
- Свёрнутая консоль показывает текущую задачу и прогресс, не отнимая места у содержимого.
- Показатель «таблица записей актуальна» предупреждает о рассинхронизации до, а не после запуска.
- Готовность данных (FR-2.16) строится по артефактам на диске, не по «типу проекта».

7.2 Проекты

Проект в orcAI — единый каталог с настройками, записями, аннотациями и артефактами. Desktop не вводит «тип проекта» и не блокирует команды: вместо этого — мягкий фокус при создании (подсказки UI) и панель готовности данных по фактическому содержимому каталога.

Как сейчас	Что мешает	Что делаем
Карточка проекта показывает имя, путь и число файлов.	Не видно ни аннотаций, ни артефактов, ни параметров проекта.	Четыре вкладки: «Записи», «Аннотации», «Артефакты», «Параметры»; шапка — фокус и готовность.
Создание проекта — только init без выбора сценария.	Пользователь не понимает, с чего начать: predict или обучение.	Мастер с тремя карточками фокуса: полевое распознавание, обучение, полный цикл — без hard lock.
Таблица записей строится автоматически и скрыта.	Пользователь не может задать канал записи, исключить файл, обновить пути.	Таблица записей показывается целиком, с каналом и признаком аннотации; обновление — явная кнопка.
Нет признака, распознана ли запись.	При повторном запуске непонятно, что будет пропущено, а что обработано.	Столбец «распознавание» с шестью состояниями, синхронный с файлами предсказаний.
Не видно, готов ли проект к обучению или predict.	Раздел «Обучение» выглядит заблокированным без объяснения.	Панель готовности: записи, аннотации, predict, датасеты, модель — с «можно / нужно / нет данных».
Удаление файлов — единственное массовое действие.	Полезные операции оригинала недоступны: исключения, дубликаты, обновление таблицы.	Кнопка «Проверить» — сверка каталога с таблицей: новые, пропавшие, повторяющиеся имена.

Требования: FR-2.1 – FR-2.17



Макет 2. Проекты, вкладка «Записи» (полный цикл)

Шапка проекта: мягкий фокус «полный цикл», сводка готовности, ключевые факты. Вкладки разделяют Дополнительные макеты состояний: m32 (мастер фокуса), m33 (пустой), m34 (полевой), m35 (лабораторный), m36 (готовность развёрнута).

· Фокус хранится только в SQLite Desktop; в JSON orcAI не пишется — CLI-проект открывается полностью.

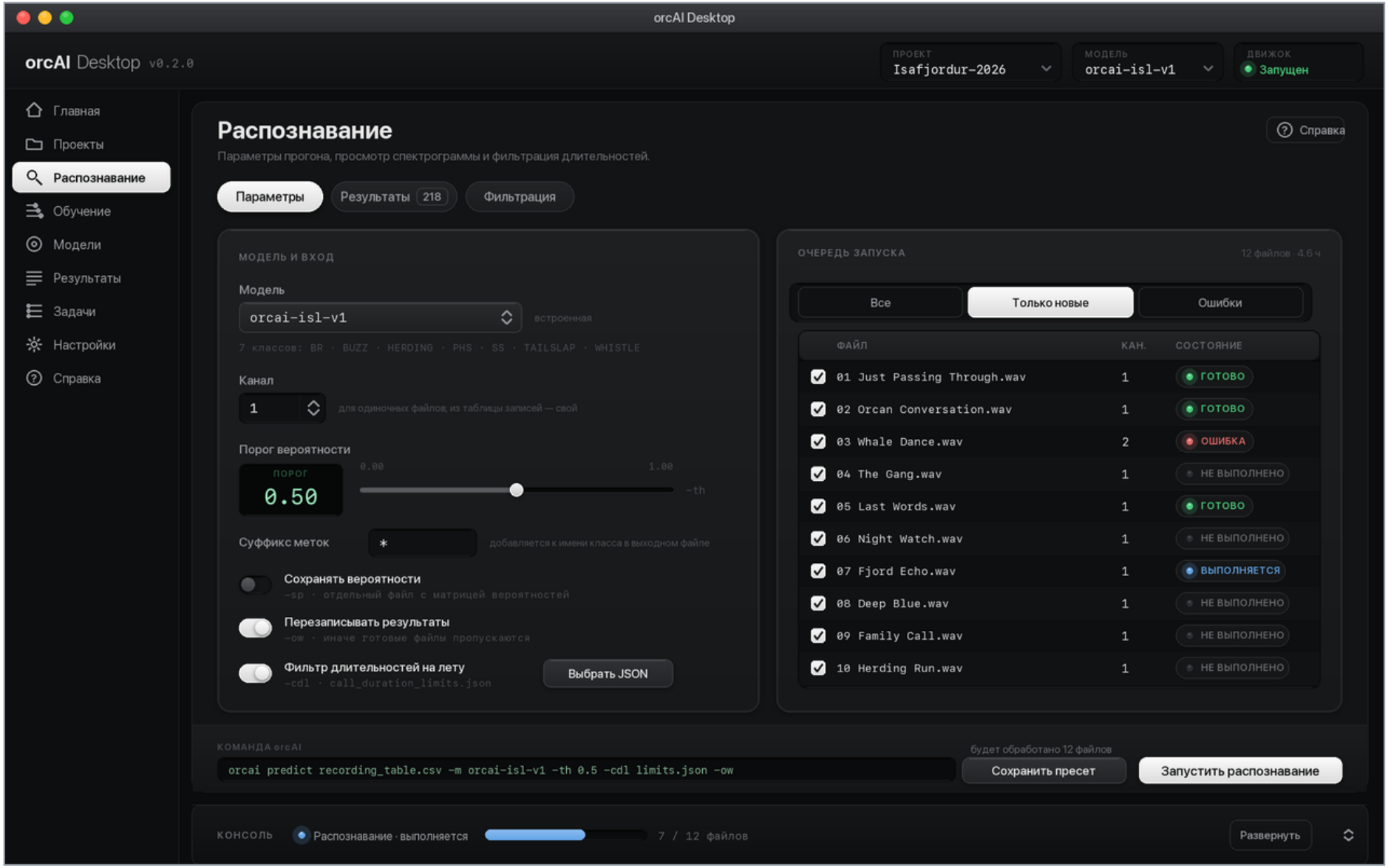
- Столбцы таблицы соответствуют полям recording_table.csv, включая канал и путь к аннотации.
- Блокировка действий — только по отсутствию артефактов (FR-7.3), не по выбранному фокусу.

7.3 Распознавание · Параметры

Основной сценарий продукта. Сейчас доступны модель, канал и порог; остальные параметры команды predict недоступны, а состав запуска неочевиден до его начала.

Как сейчас	Что мешает	Что делаем
Параметры идут одним столбцом с прокруткой.	Порог оказывается ниже сгиба, очередь запуска не видна одновременно с настройками.	Две колонки: слева модель и вход, справа очередь записей — настройка и объект видны вместе.
Нет сохранения вероятностей, суффикса меток, каталога результата.	Часть возможностей оригинала недостижима из интерфейса.	Все параметры predict на одном экране, сгруппированные по смыслу, с подписями коротких ключей.
Непонятно, сколько файлов будет обработано.	Пользователь запускает пакет вслепую.	Очередь с состоянием каждой записи и фильтрами «все / только новые / ошибки».
Не видно, что именно будет выполнено.	Нельзя сверить настройки с оригиналом и воспроизвести запуск в терминале.	Строка команды orcAI внизу экрана, обновляемая при каждом изменении параметра.
Настройки не переносятся между проектами.	Повторяющаяся ручная работа при однотипных прогонах.	Сохранение набора параметров как пресета.

Требования: FR-3.1 – FR-3.15



Макет 3. Распознавание, вкладка «Параметры»

Слева — модель с перечнем её классов, канал, порог на ЖК-индикаторе с ползунком, суффикс меток и три переключателя: вероятности, перезапись, пределы длительности на лету. Справа — очередь запуска. Внизу — закреплённая строка запуска с точной командой.

· Под названием модели перечислены её классы: это первая проверка совместимости с проектом.

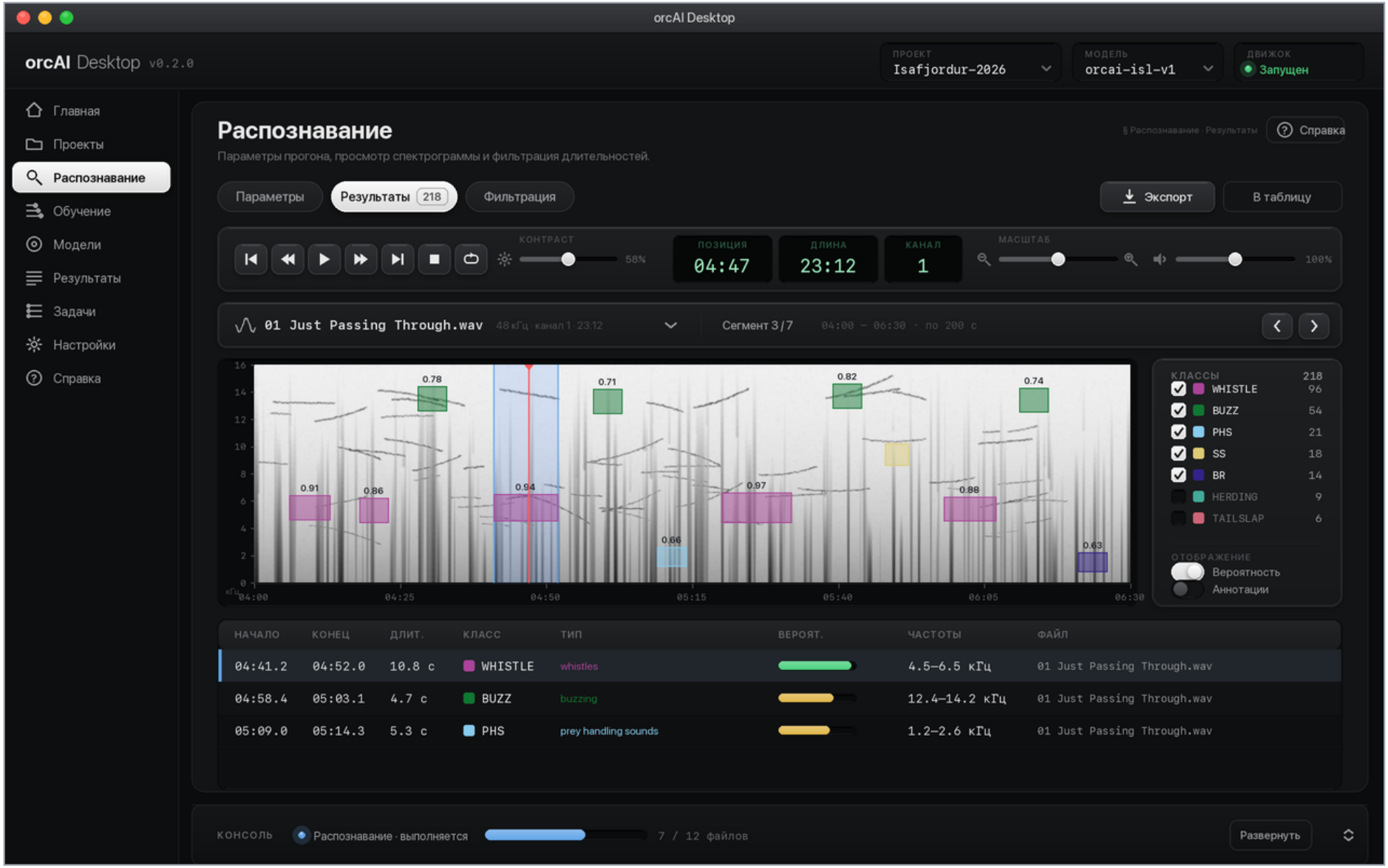
- Значения по умолчанию совпадают с CLI: канал 1, порог 0.5, суффикс «*».
- Кнопка запуска всегда в правом нижнем углу — на всех экранах приложения одинаково.

7.4 Распознавание · Результаты

Самая сильная часть текущей версии: серая спектрограмма с цветными наложениями и транспортная панель. Не хватает того, ради чего пользователь сюда приходит, — разбора: что именно найдено, с какой уверенностью и совпадает ли это с ручной разметкой.

Как сейчас	Что мешает	Что делаем
Спектрограмма и наложения детекций.	Наложения без подписей: не видно ни класса, ни вероятности.	Значение вероятности над наложением и цветовая легенда классов справа.
Транспорт, позиция, длина, канал.	Нет способа скрыть лишние классы и сосредоточиться на одном.	Легенда с кодами классов, флажками видимости и счётчиками; расшифровки типов — в колонке «Тип» таблицы.
Навигация по сегментам.	Список детекций отсутствует: нельзя перейти к нужному месту записи.	Таблица детекций под спектрограммой, связанная с выделением на ней.
Аннотации не показываются.	Сверка распознанного с ручной разметкой невозможна внутри приложения.	Переключатель «Аннотации»: ручная разметка накладывается контуром поверх наложений.
Масштаб и контраст ограничены.	На длинных записях трудно рассмотреть короткие сигналы.	Регуляторы контраста, масштаба и громкости в одной панели, как на приборе.

Требования: FR-4.1 – FR-4.10



Макет 4. Распознавание, вкладка «Результаты»

Транспортная панель с ЖК-индикаторами позиции, длительности и канала; строка выбора записи и сегмента; спектрограмма 0–16 кГц с наложениями и выделением; справа — легенда кодов классов со счётчиками и видимостью; внизу — таблица детекций (колонка «Тип» — whistles/buzzing/…).

· Спектрограмма строится тем же кодом, что и вход модели, — иначе просмотр вводил бы в заблуждение.

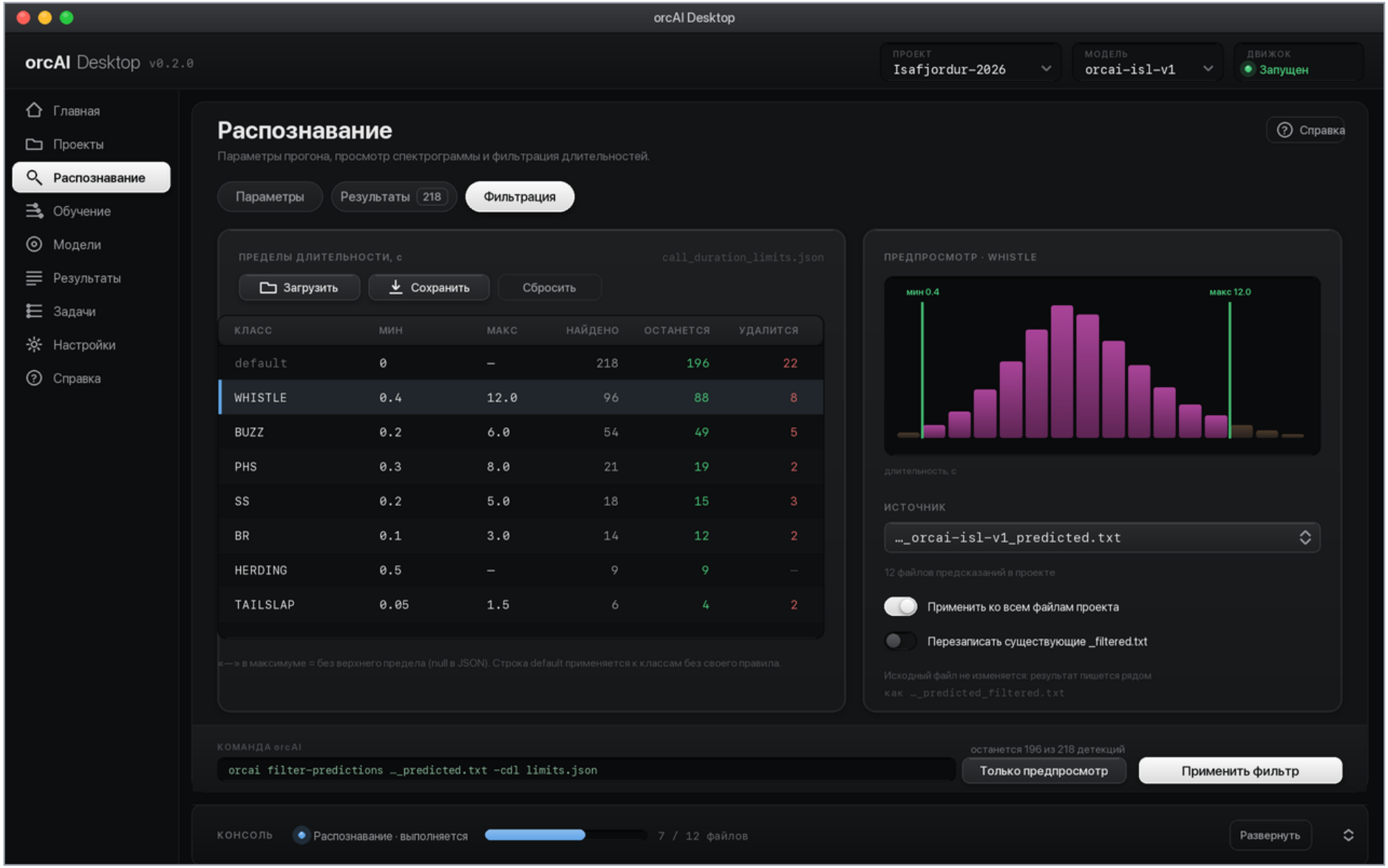
- Красная линия — позиция воспроизведения, синяя область — выделенный участок.
- Строка таблицы и наложение на спектрограмме — один и тот же объект: выделение синхронно.

7.5 Распознавание · Фильтрация

Отдельная команда оригинала, работающая над готовым файлом предсказаний. Сейчас она запускается, но правила фильтрации скрыты в файле JSON, и результат непредсказуем.

Как сейчас	Что мешает	Что делаем
Фильтрация запускается по файлу пределов.	Пределы нельзя ни увидеть, ни изменить в приложении.	Табличный редактор: класс, минимум, максимум, включая правило default.
Результат становится известен после выполнения.	Пользователь узнаёт о потере половины детекций постфактум.	Предпросмотр: сколько детекций найдено, останется и будет удалено по каждому классу.
Пустая верхняя граница не имеет объяснения.	Смысл значения null в файле неочевиден.	В таблице это «—» с подписью: без верхнего предела.
Нет выбора области применения.	Фильтровать проект по файлу приходится вручную.	Переключатель «применить ко всем файлам проекта» и отдельная перезапись.
Выбор границ ничем не подкреплён.	Пределы задаются наугад.	Распределение длительностей выбранного класса с отметками текущих границ.

Требования: FR-5.1 – FR-5.8



Макет 5. Распознавание, вкладка «Фильтрация»

Слева — редактор пределов с подсчётом последствий по каждому классу. Справа — распределение длительностей выбранного класса с отметками минимума и максимума, выбор источника и область применения. Внизу — команда и результат: останется 196 из 218 детекций.

· Исходный файл предсказаний не изменяется: результат пишется рядом с суффиксом _filtered.

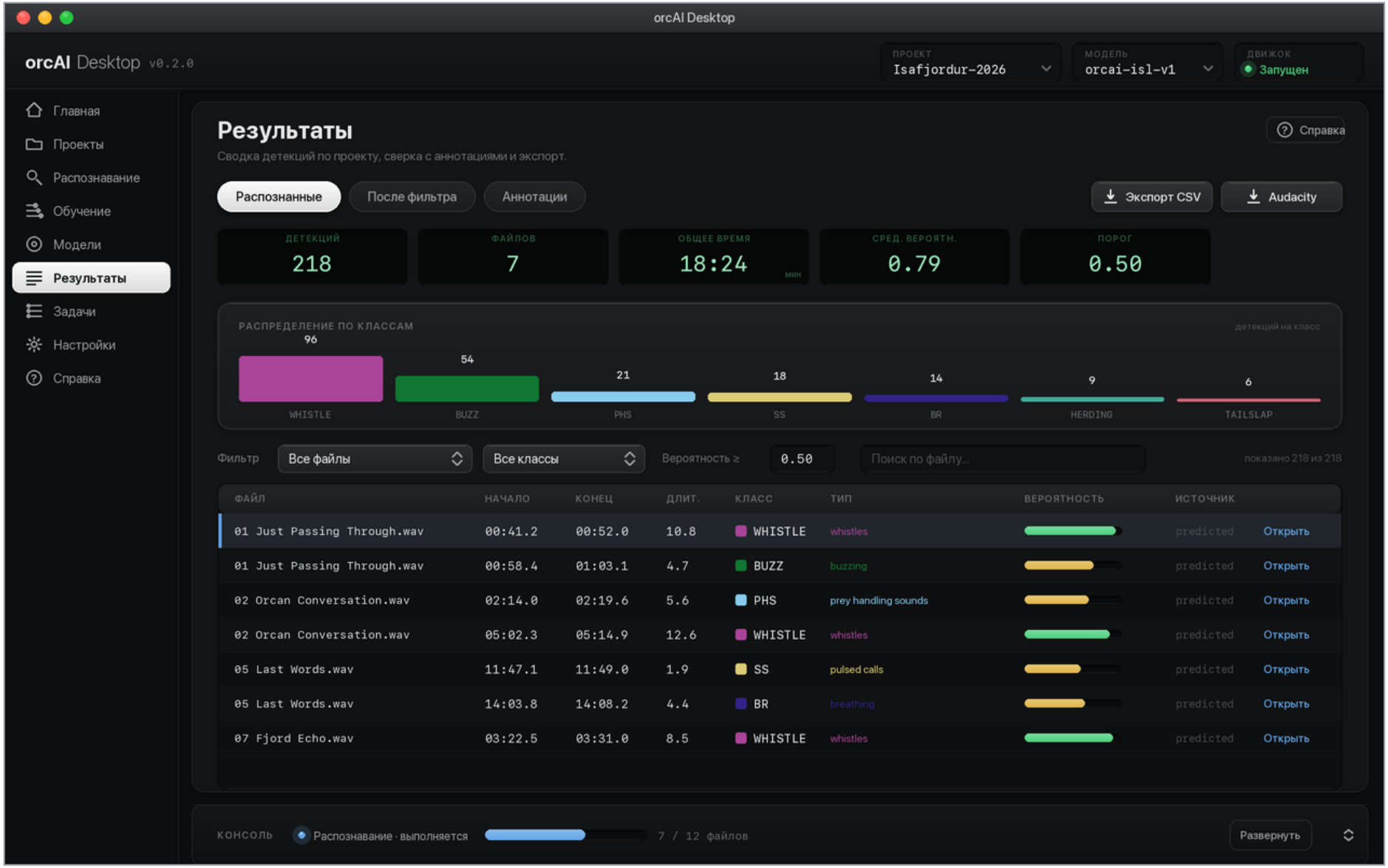
- Строка default применяется к классам, для которых не задано собственное правило.
- Набор пределов сохраняется в JSON и переносится в другой проект.

7.6 Результаты

Раздел присутствует в меню, но сводного взгляда на проект не даёт. При этом именно он отвечает на вопрос «что мы вообще нашли» и служит точкой выгрузки данных.

Как сейчас	Что мешает	Что делаем
Таблица детекций по одному файлу.	Нельзя оценить проект целиком: сколько всего найдено и чего именно.	Сводка по проекту: детекции, файлы, суммарная длительность, средняя вероятность, порог.
Нет распределения по классам.	Неясно, преобладает ли один класс и не пропущены ли остальные.	Столбиковая диаграмма по классам с абсолютными числами.
Нет фильтров и поиска.	На сотнях строк работать неудобно.	Фильтры по файлу, классу и вероятности, поиск и сортировка по любому столбцу.
Нет экспорта.	Результат приходится искать в файловой системе.	Экспорт в CSV и в формат меток Audacity — прямо из таблицы.
Источник данных не выбирается.	Распознанное, отфильтрованное и ручные аннотации нельзя сравнить.	Переключатель источника: распознанные, после фильтра, аннотации.

Требования: FR-6.1 – FR-6.6



Макет 6. Результаты

Сводка проекта на ЖК-индикаторах, распределение по классам, фильтры и таблица всех детекций с полосой вероятности. Каждая строка ведёт к соответствующему месту на спектрограмме.

- Полоса вероятности окрашивается по порогу уверенности: наглядно видно слабые срабатывания.
- Переключатель источника показывает, чем распознанное отличается от отфильтрованного.

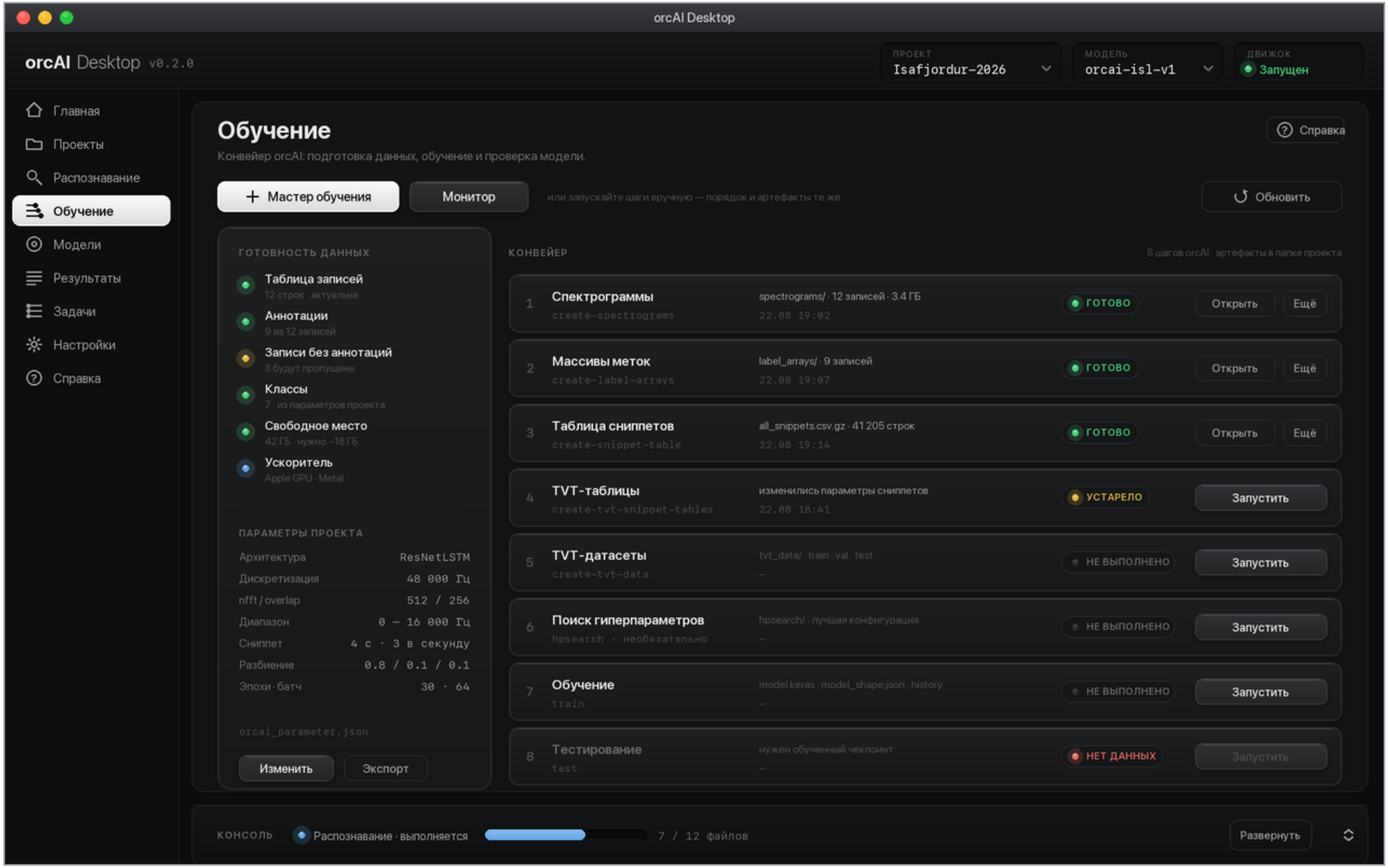
· Экспорт Audacity даёт файл, который открывается как дорожка меток поверх записи.

7.7 Обучение · конвейер

Здесь сосредоточен самый серьёзный дефект версии: все двенадцать шагов помечены словом «ГОТОВО». Оно означает «можно запустить», а читается как «уже выполнено» — интерфейс систематически вводит пользователя в заблуждение о состоянии проекта.

Как сейчас	Что мешает	Что делаем
Список из двенадцати шагов, включая распознавание и фильтрацию.	В конвейер обучения попали команды, к обучению не относящиеся.	Восемь шагов подготовки и обучения; распознавание и фильтрация остались в своём разделе.
У каждого шага надпись «ГОТОВО».	Состояние недостоверно — главная причина недоверия к разделу.	Шесть состояний: не выполнено, выполняется, готово, устарело, нет данных, ошибка.
Артефакты не показаны.	Непонятно, что появилось на диске и сколько это занимает.	У каждого шага имя артефакта, объём, время последнего запуска и кнопка «Открыть».
Готовность данных не проверяется.	Запуск возможен там, где данных заведомо нет.	Панель готовности: таблица записей, аннотации, классы, место на диске, ускоритель.
Обучение доступно только как ручной запуск шагов.	Пользователь должен сам знать порядок и совместимость параметров.	Кнопка «Мастер обучения» вверху; ручной режим сохраняется для опытных пользователей.

Требования: FR-7.1 – FR-7.13



Макет 7. Обучение, доска конвейера

Слева — готовность данных и действующие параметры проекта. Справа — восемь шагов оригинала: команда, артефакт, время, честное состояние. Шаг 4 помечен как устаревший, потому что изменились параметры нарезки; шаг 8 в состоянии «нет данных» до появления обученного чекпоинта.

· Состояние «устарело» появляется, когда изменились параметры, от которых зависит артефакт.

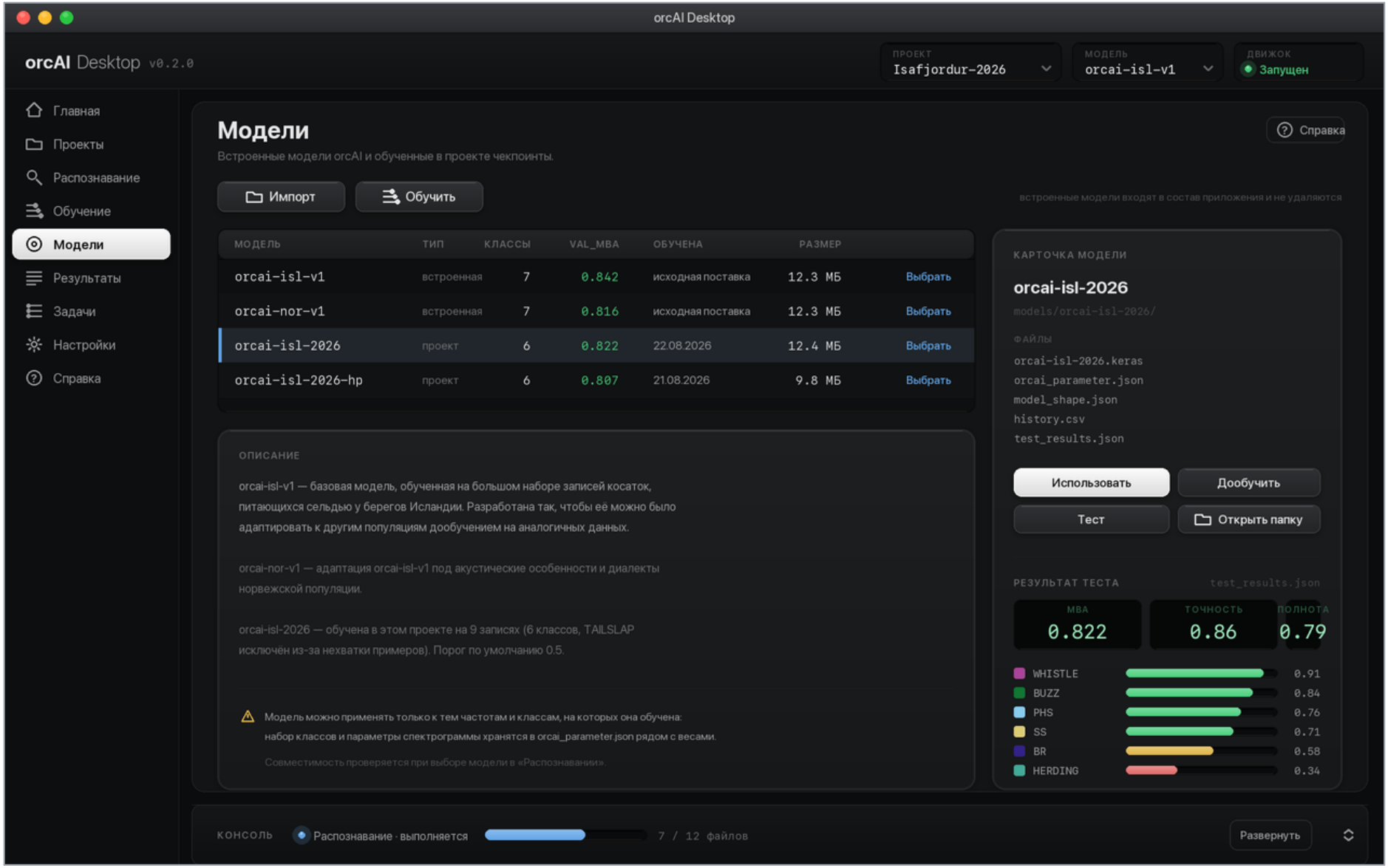
- Шаг «нет данных» всегда с текстом причины (FR-7.3), а не молча неактивная кнопка.
- Панель параметров проекта показывает то, с чем реально будет работать конвейер.

7.8 Модели

Раздел хорошо описывает две встроенные модели, но ничего не знает о моделях, обученных пользователем, и не отвечает на главный вопрос — можно ли применить модель к этим данным.

Как сейчас	Что мешает	Что делаем
Таблица встроенных моделей с описанием.	Обученные в проекте модели не показываются.	Единый список: встроенные и модели проекта, с датой обучения и метрикой.
Действие только одно — использовать в распознавании.	Дообучение, тест и открытие каталога недоступны.	Карточка модели с действиями: использовать, дообучить, протестировать, открыть каталог.
Метрики модели неизвестны.	Качество обученной модели нельзя оценить внутри приложения.	Результаты теста в карточке: метрика val_MBA и по каждому классу точность, полнота и F1.
Состав модели скрыт.	Непонятно, из чего состоит каталог модели.	Список файлов каталога: веса, параметры, форма модели, история обучения и подкаталог test с метриками и таблицами ошибок.
Совместимость не проверяется.	Модель с другой полосой частот молча даст плохой результат.	Проверка классов и параметров спектрограммы при выборе модели для проекта.

Требования: FR-9.1 – FR-9.8



Макет 8. Модели

Слева — список моделей и описание происхождения встроенных. Справа — карточка выбранной модели: файлы каталога, действия и метрики последнего теста с разбивкой по классам, где сразу видно слабое место — класс HERDING.

· Метрики берутся из файлов, которые пишет команда test, и не пересчитываются приложением.

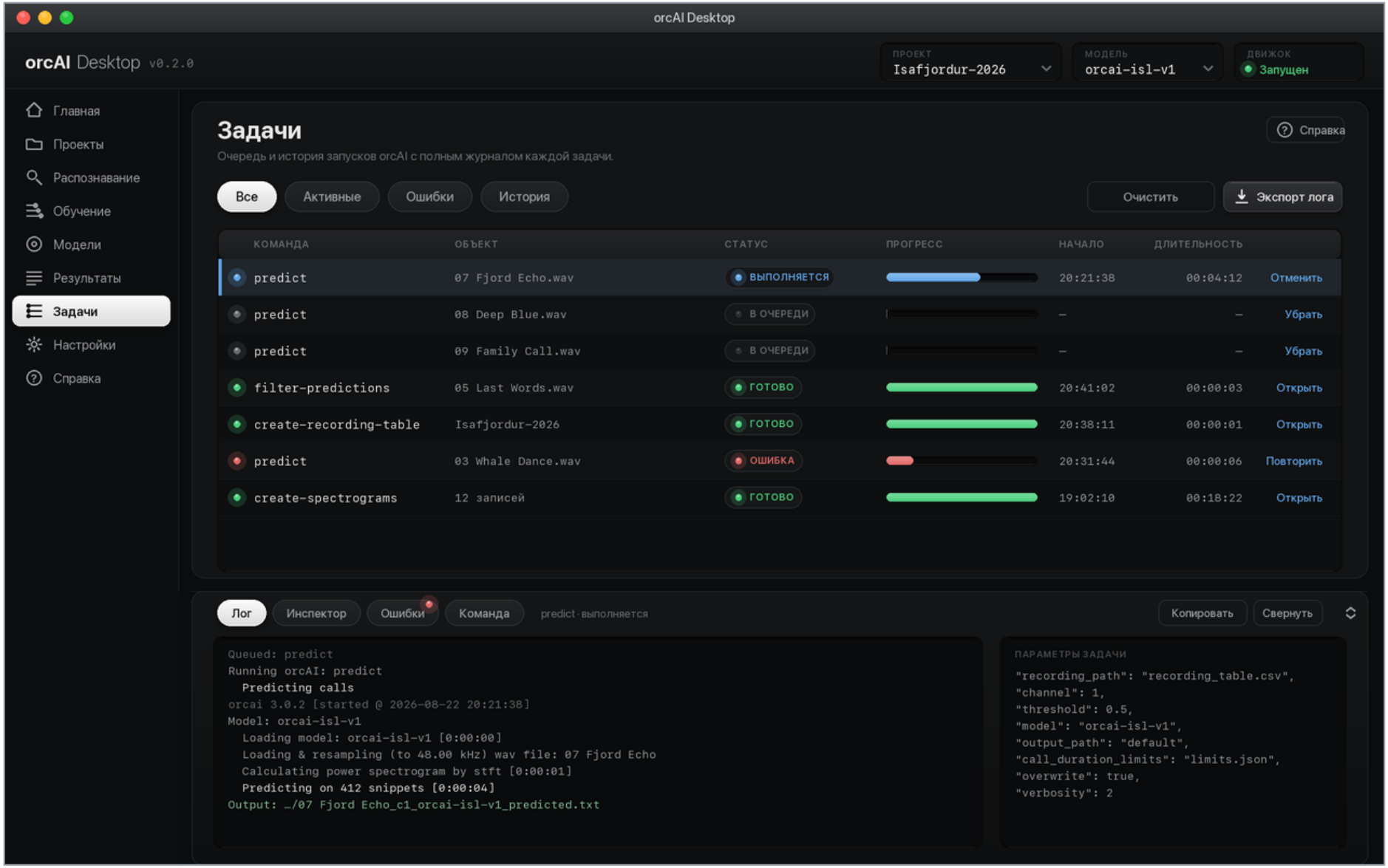
- «Дообучить» из карточки открывает мастер сразу в режиме дообучения с этой базовой моделью.
- Встроенные модели нельзя удалить или изменить — они часть поставки.

7.9 Задачи

Очередь с журналом и инспектором — одно из лучших решений текущей версии. Ему не хватает управления: задачу нельзя ни отменить, ни повторить, а найти ошибку среди множества строк трудно.

Как сейчас	Что мешает	Что делаем
Список задач с состоянием и журналом.	Нет отмены выполняющейся и удаления ожидающей задачи.	Действие в каждой строке: отменить, убрать из очереди, повторить, открыть результат.
Ошибки перемешаны с остальными задачами.	Причину сбоя приходится искать в общем потоке.	Вкладка «Ошибки» в консоли с переходом к проблемной задаче.
Команда, которую выполнил движок, не показана.	Невозможно воспроизвести запуск вне приложения и сверить параметры.	Вкладка «Команда» с точной строкой orcAI, готовой к копированию.
Журнал живёт только в окне.	Нечего приложить к сообщению об ошибке.	Экспорт журнала задачи в файл.
Прогресс задан общей полосой.	На многочасовых прогонах непонятно, сколько осталось.	Прогресс по файлам и оставшееся время в строке задачи и в свёрнутой консоли.

Требования: FR-10.1 – FR-10.9



Макет 9. Задачи · очередь и консоль «Лог»

Очередь с прогрессом по каждой задаче и действиями в строке. Консоль развёрнута: слева настоящий вывод orcAI, справа параметры задачи в исходном виде. Вкладка «Ошибки» помечена индикатором.

· Журнал показывается как есть, без пересказа: это способ доверять приложению.

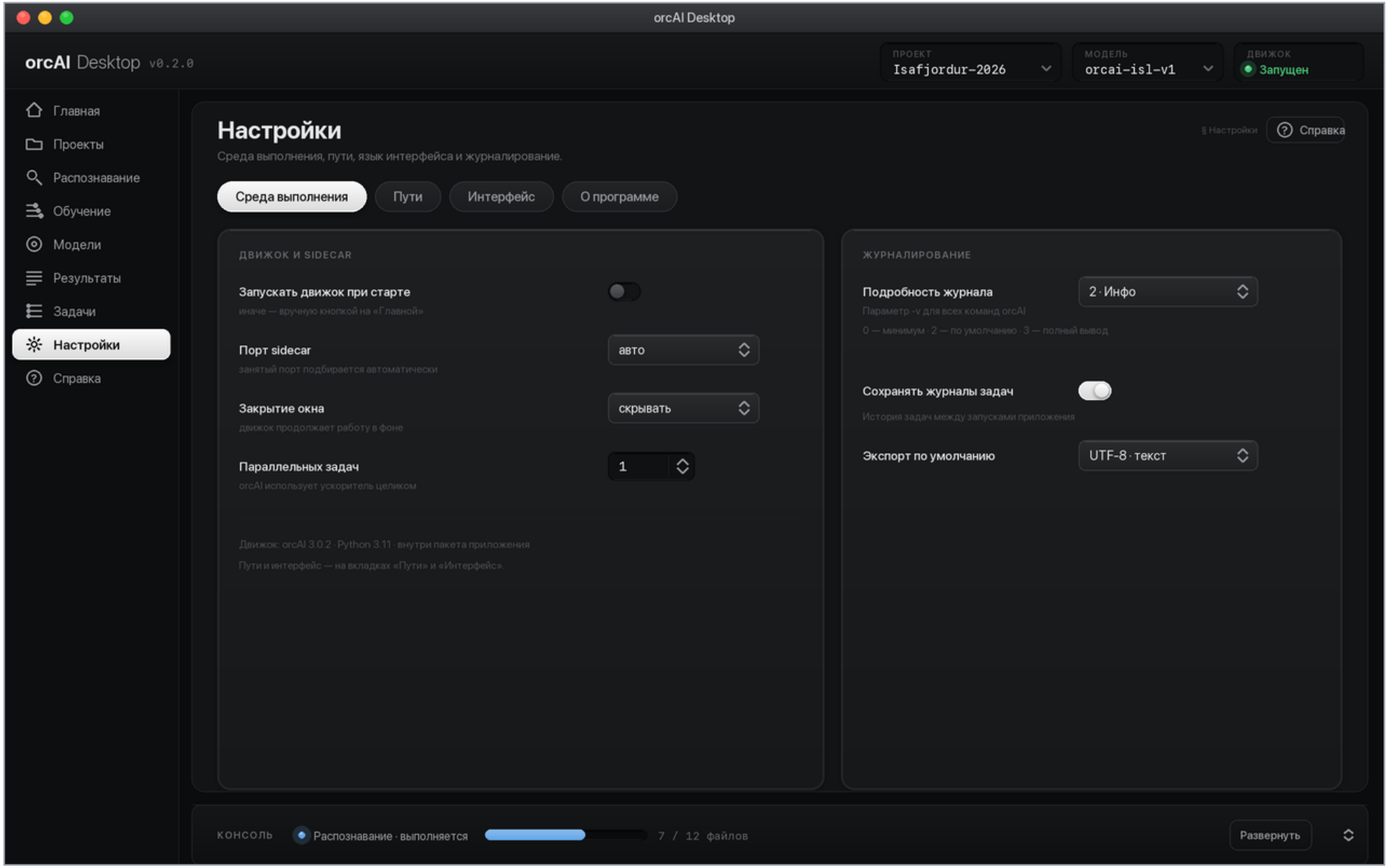
- Повтор неудавшейся задачи запускает её с теми же параметрами, не трогая остальную очередь.
- История сохраняется между запусками приложения.

7.10 Настройки

Настроек должно быть мало, но те, что влияют на поведение команд, обязаны быть доступны. Сейчас отсутствует главное — подробность журнала, поведение окна и пути.

Как сейчас	Что мешает	Что делаем
Настройки ограничены языком и темой.	Уровень подробности журнала недоступен, хотя есть у каждой команды.	Подробность журнала от 0 до 3, применяемая ко всем запускам.
Порт и поведение окна не настраиваются.	Пользователь не может изменить сценарий работы в фоне.	Порт sidecar, поведение при закрытии окна, число параллельных задач.
Пути к данным не показаны.	Непонятно, где приложение хранит проекты, кэш и журналы.	Список путей с кнопками «Обзор» и «Открыть».
Язык переключается, но термины не объяснены.	Неочевидно, что «распознавание» — это predict.	Пояснение под переключателем языка: соответствие терминов оригиналу.

Требования: FR-11.1 – FR-11.9, NFR-17, NFR-18



Макет 10. Настройки · Среда выполнения

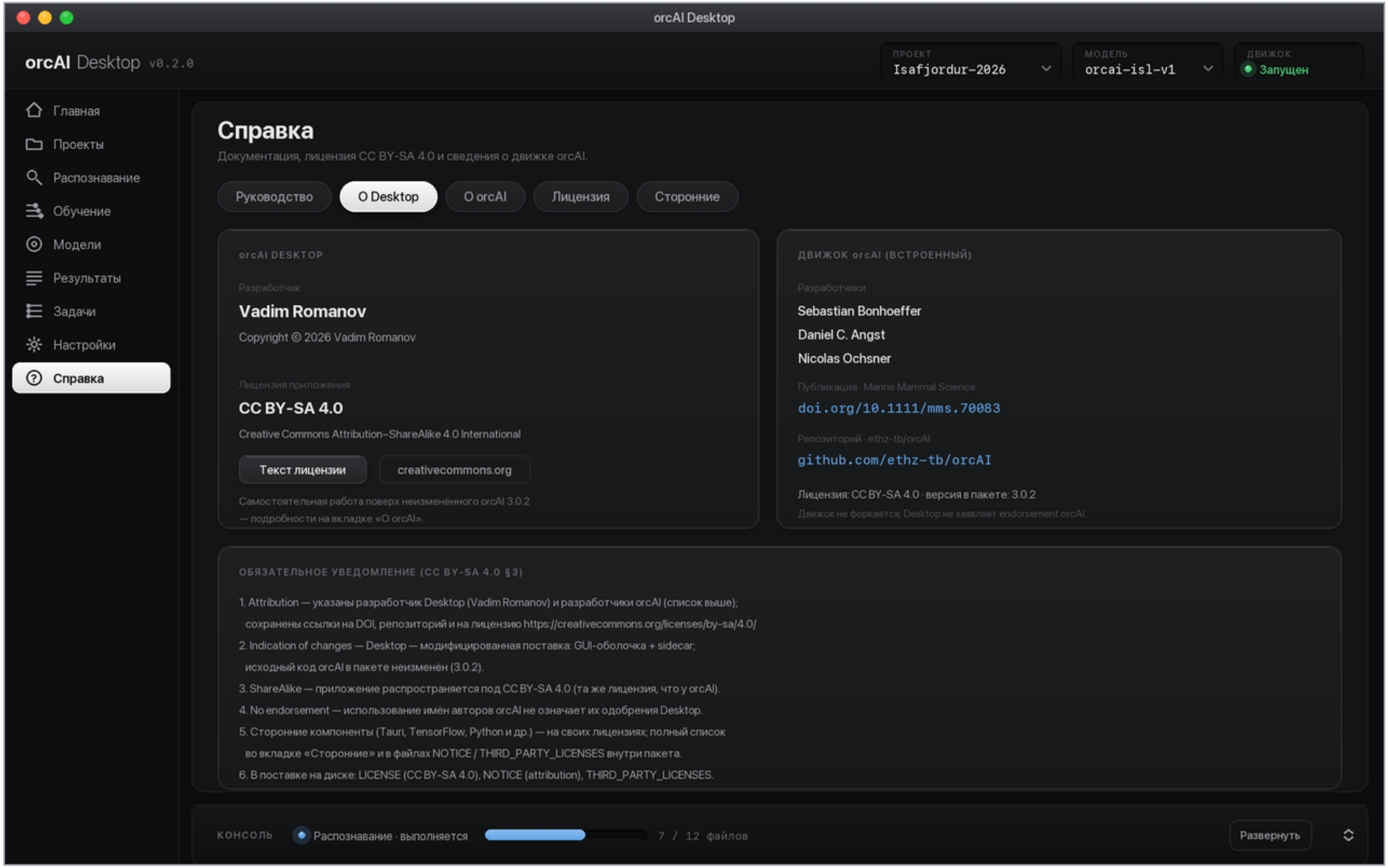
Вкладка «Среда выполнения»: движок, sidecar, -v, параллельные задачи (ЭК-10 / П-13). Интерфейс (язык, DPI, Full Screen) — раньше, ЭК-3а / П-13а, m28. Пути — m31; «О программе» — m29 (с П-16).

· Число параллельных задач по умолчанию равно единице: orcAI занимает ускоритель целиком.

- Подробность журнала передаётся командам как параметр -v и влияет на объём вывода в консоли.
- Пути показываются в том виде, в каком их видит движок.

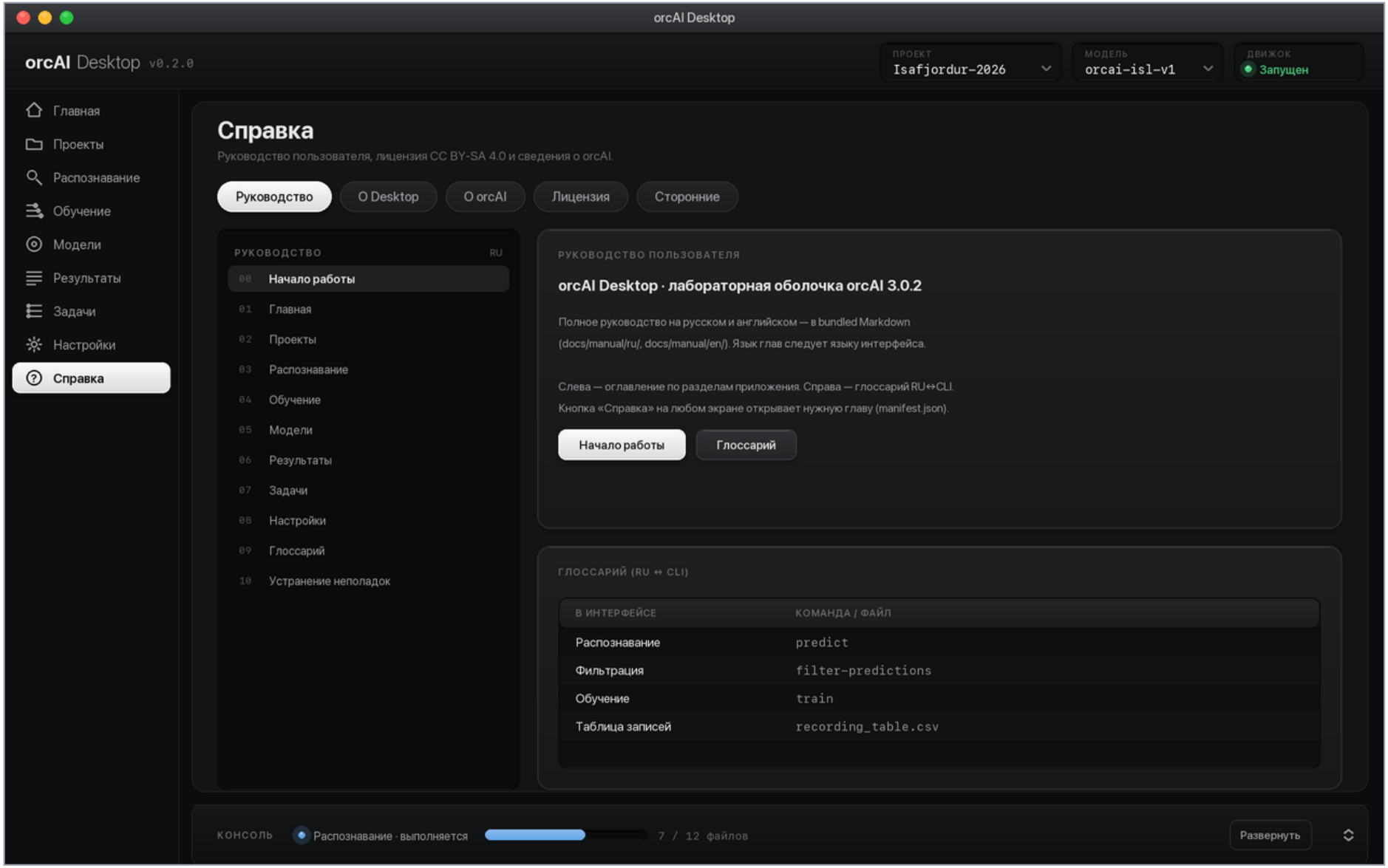
7.11 Справка

Раздел «Справка» — руководство пользователя (RU/EN), юридический и справочный дом. Вкладка «Руководство» — оглавление и чтение глав из bundled docs/manual/; кнопка «Справка» на рабочих экранах (не в самом разделе «Справка») открывает якорь текущего раздела (FR-11.13). Chrome и порядок сайдбара не меняются.



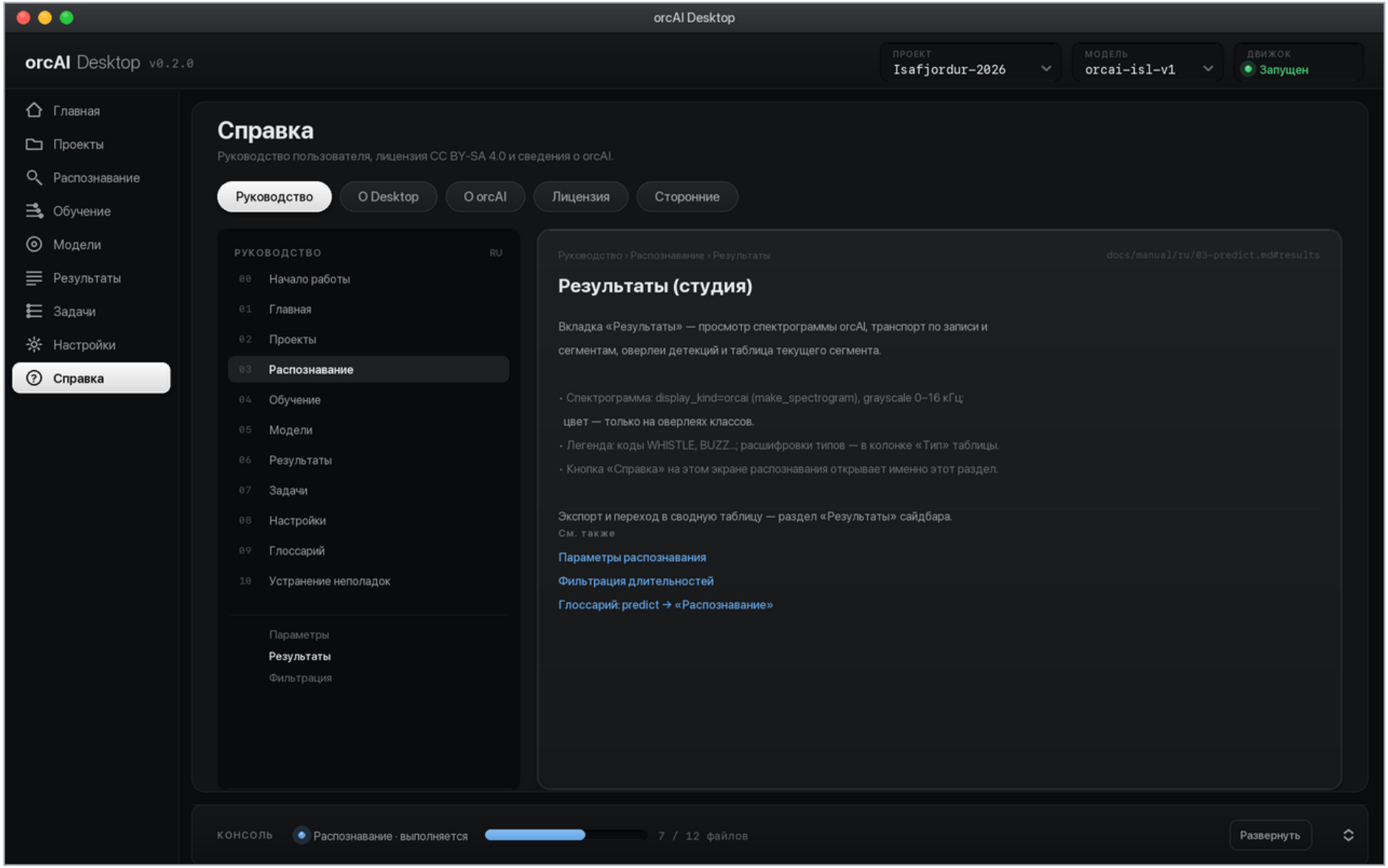
Справка · вкладка «О Desktop»

Vadim Romanov, CC BY-SA 4.0, краткое attribution orcAI.



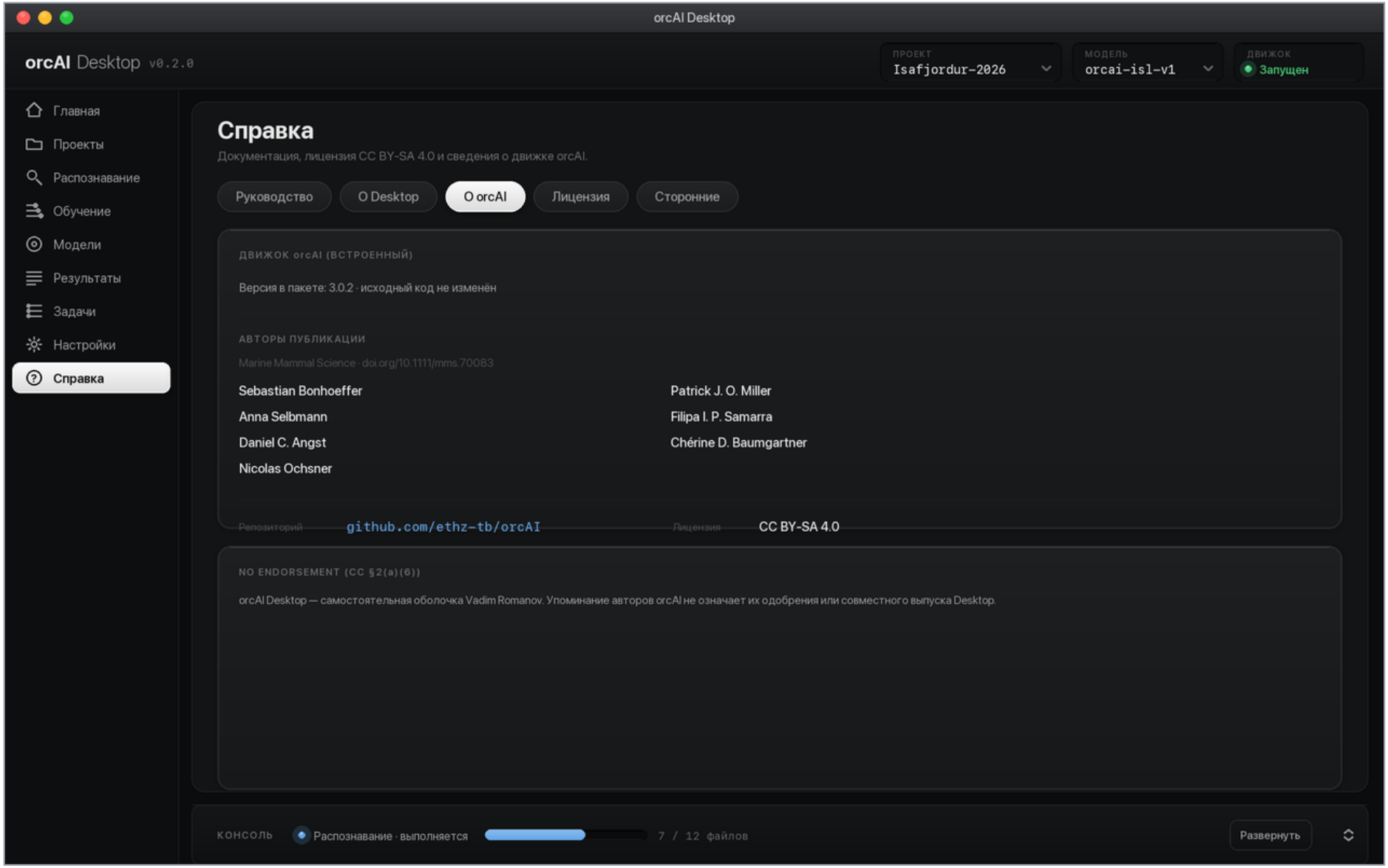
Справка · вкладка «Руководство» — оглавление

Дерево глав RU/EN, глоссарий RU↔CLI, bundled Markdown.



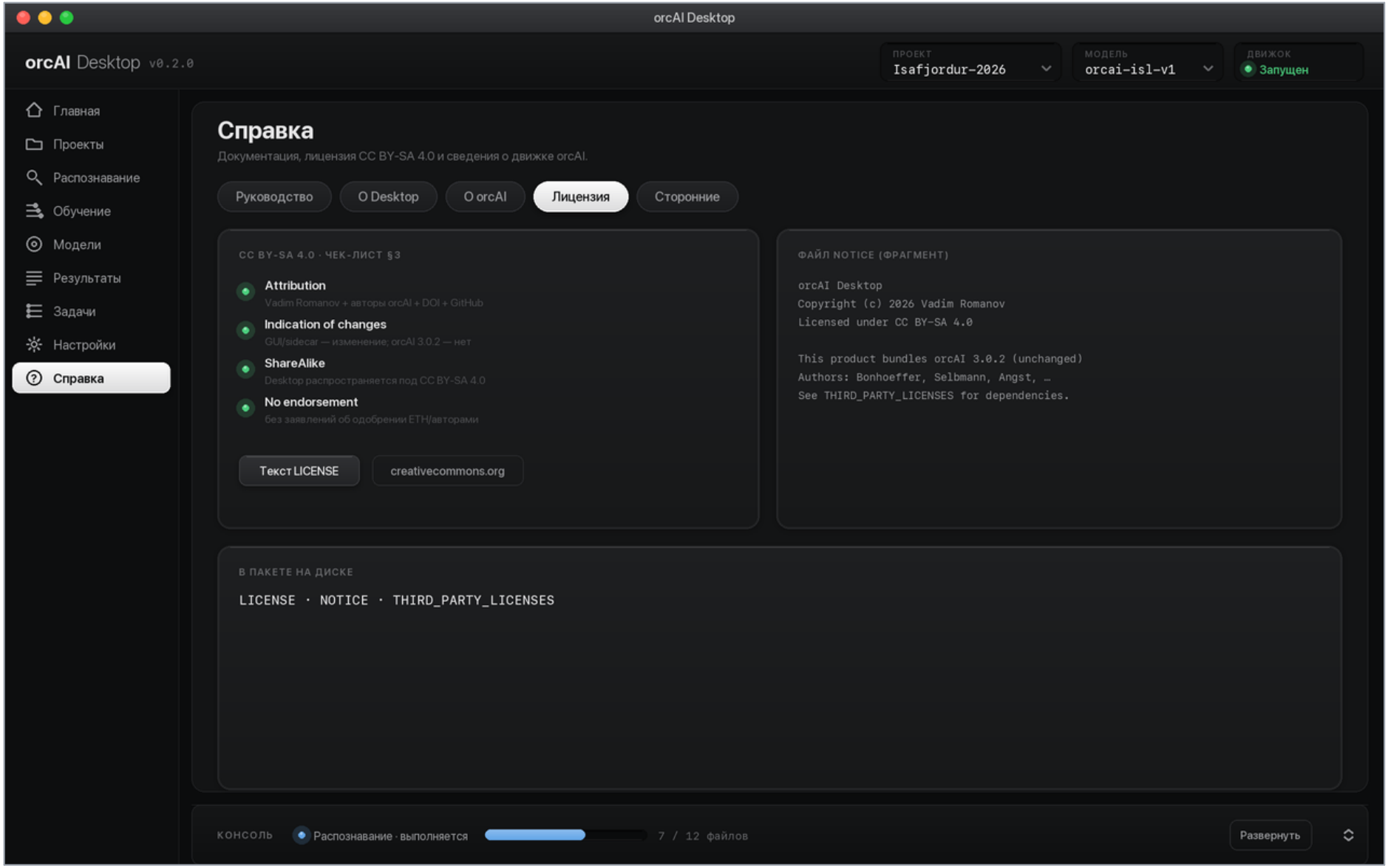
Справка · глава «Распознавание · Результаты»

Пример deep link с кнопки «Справка» на экране студии (m04).



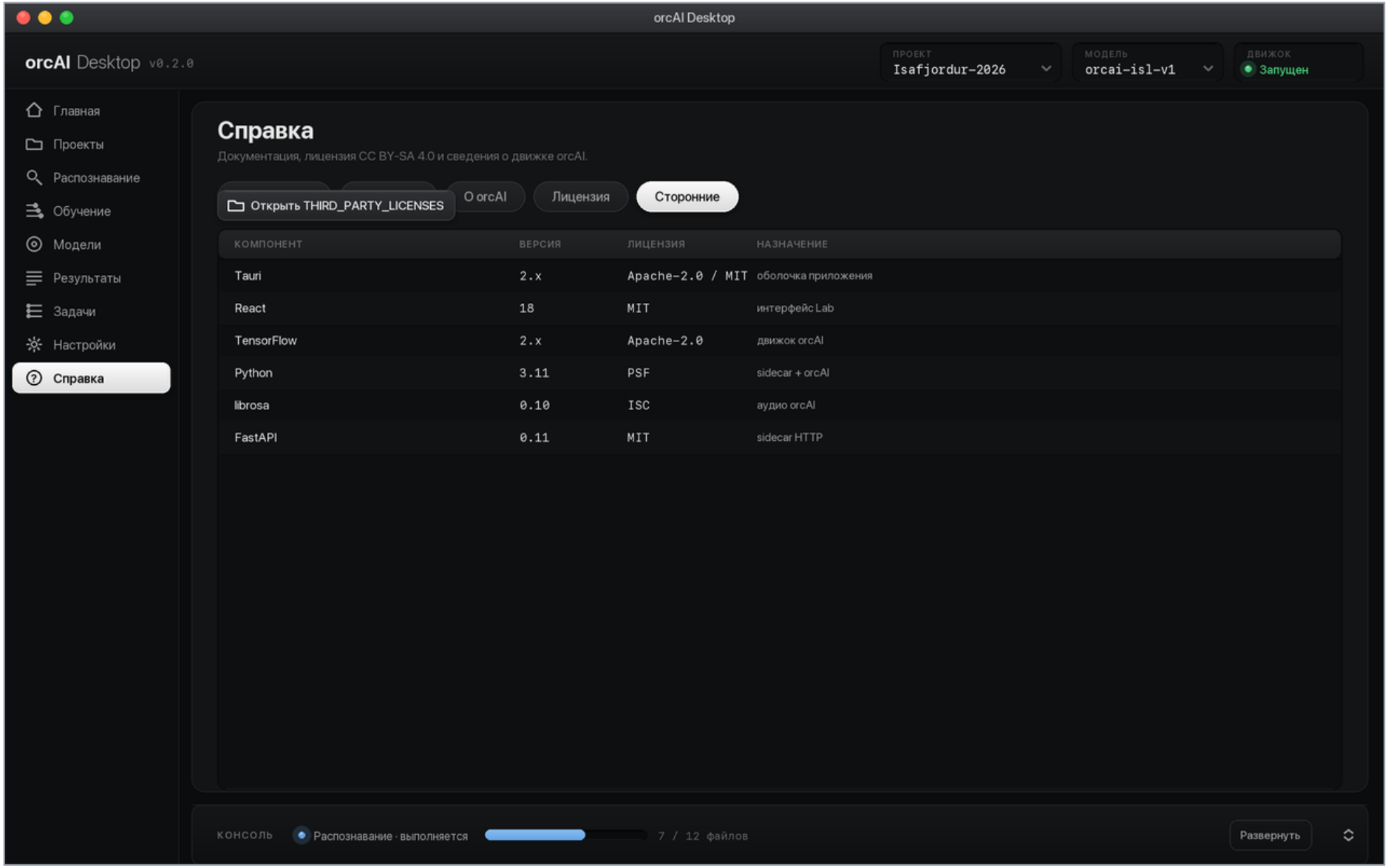
Справка · вкладка «О orcAI»

Семь авторов статьи, DOI, GitHub, формулировка no endorsement.



Справка · вкладка «Лицензия»

Чек-лист CC §3 и фрагмент NOTICE.



Справка · вкладка «Сторонние»

Таблица зависимостей и кнопка THIRD_PARTY_LICENSES.

8. Мастер обучения моделей

Пять шагов, за которыми стоит штатная цепочка команд orcAI. Мастер ничего не вычисляет сам: он собирает параметры, проверяет их совместимость и выстраивает порядок запуска.

8.1 Принцип: мастер — это оркестратор, а не новый алгоритм

Обучение в orcAI — это последовательность из пяти команд подготовки данных, команды обучения и команды тестирования. Каждая читает артефакты предыдущей и получает один и тот же файл параметров проекта. Ошибиться легко: достаточно поменять длительность сниппета и не пересобрать датасеты, чтобы получить обучение на несогласованных данных. Мастер устраняет именно этот класс ошибок — он хранит один набор параметров, записывает его в файл проекта и запускает команды в правильном порядке.

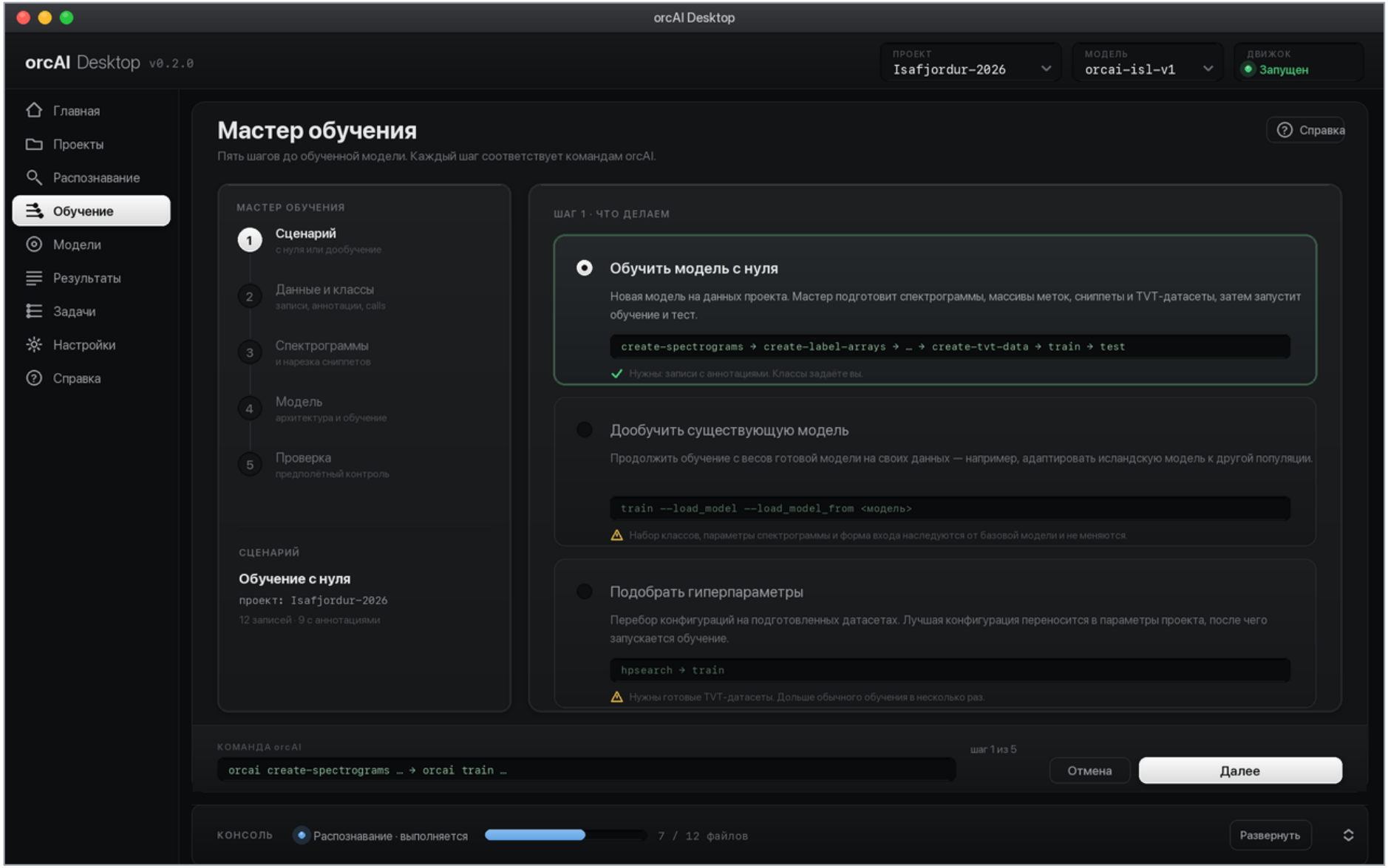
Что мастер делает	Чего мастер не делает
Собирает параметры в один согласованный набор и записывает его в файл параметров проекта.	Не вводит собственных параметров, которых нет в orcAI.
Проверяет совместимость значений между собой и с базовой моделью до запуска.	Не изменяет значения по умолчанию оригинала и не «улучшает» их.
Определяет, какие шаги уже выполнены, а какие устарели, и не запускает лишнего.	Не пропускает шаги, от которых зависит следующий, — порядок неизменен.
Показывает план из конкретных команд с оценкой времени.	Не скрывает происходящее: каждая команда попадает в очередь задач с полным журналом.
Позволяет остановиться после подготовки данных, не запуская обучение.	Не запускает ничего в фоне без ведома пользователя.

8.2 Шаги мастера и их отображение в командах

Шаг	Название	Что спрашивает	Куда попадает	Какие команды порождает
1	**Сценарий**	Обучение с нуля, дообучение существующей модели или поиск гиперпараметров.	`Определяет, какие шаги попадут в план и какие поля будут заблокированы.`	—
2	**Данные и классы**	Таблица записей, каталог аннотаций, файл синонимов меток, набор классов будущей модели.	`calls, пути к данным.`	`create-recording-table (при необходимости)`
3	**Спектрограммы и сниппеты**	Частота, окно БПФ, перекрытие, полоса частот, квантили; длительность сниппета, плотность нарезки, длина сегмента, отбрасывание пустых, разбиение train/val/test.	`spectrogram.*, snippets.*`	`create-spectrograms, create-label-arrays, create-snippet-table, create-tvt-snippet-tables, create-tvt-data`
4	**Модель**	Имя модели, архитектура, фильтры, ядро, отсев, размер рекуррентного слоя, эпохи, батч, скорость обучения, ранняя остановка, снижение скорости, метрика, веса классов, зерно.	`name, architecture, model.*, seed`	`train`
5	**Проверка и запуск**	Предполётные проверки, план команд с оценкой времени, тест после обучения, сжатие датасетов.	`Ничего нового: подтверждение ранее выбранного.`	`весь план, затем test`

Почему шагов именно пять

Границы шагов повторяют границы предметных решений: сначала выбирается стратегия, затем данные и словарь классов, затем представление сигнала, затем сама модель, и лишь потом — запуск. Разрезать иначе нельзя: параметры спектрограммы бессмысленно задавать до того, как известен набор классов, а модель — до того, как определена форма входа.



Макет 11. Мастер обучения, шаг 1 — сценарий

Три сценария с описанием, точной цепочкой команд и главным ограничением каждого. Выбранный вариант выделен зелёной кромкой; под каждым — предупреждение о том, что потребуется и чего изменить будет нельзя.

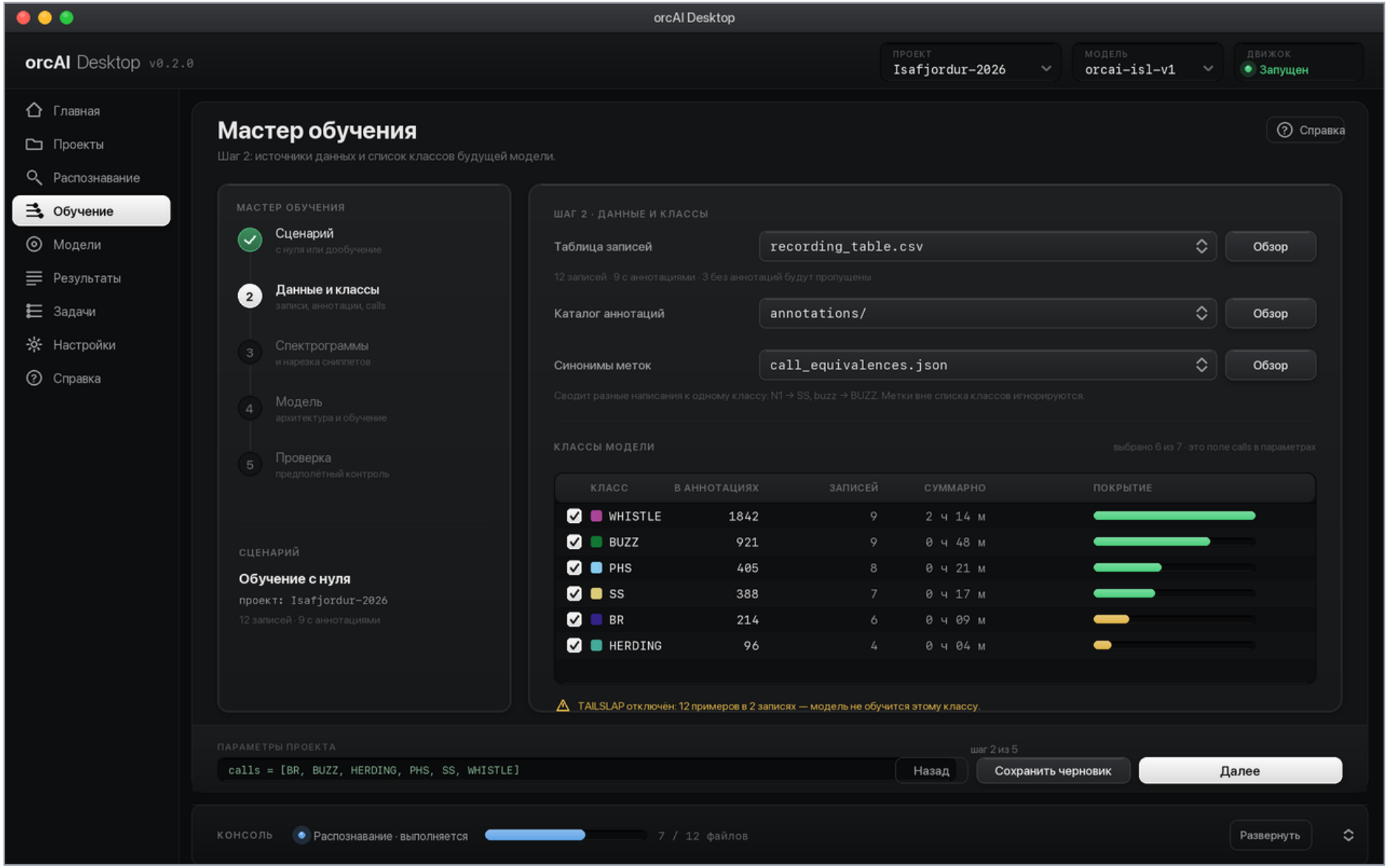
· Дообучение сразу предупреждает: классы и спектрограмма наследуются от базовой модели.

- Поиск гиперпараметров требует готовых датасетов — это указано до выбора, а не после.
- Левая колонка показывает пройденные шаги и напоминает выбранный сценарий.

8.3 Шаг 2: данные и классы

Набор классов — самое ответственное решение мастера: он фиксируется в параметрах модели и определяет размер выхода сети. Поэтому классы не вводятся вручную, а выбираются из тех, что фактически встречены в аннотациях, вместе с числом примеров по каждому. Метки, которых нет в выбранном списке, при построении массивов меток игнорируются — это поведение оригинала, и мастер обязан показать, сколько меток будет отброшено. Свести разные написания к одному классу можно только через файл синонимов.

> **Правило, которое мастер обязан объяснить пользователю**
>
> Класс, для которого в данных почти нет примеров, не станет лучше от обучения — он станет источником ложных срабатываний. Мастер не запрещает такой выбор, но показывает число примеров, покрытие и предупреждение, чтобы решение принималось осознанно.



Макет 12. Мастер обучения, шаг 2 — данные и классы

Источники данных сверху, таблица классов снизу: число примеров, в скольких записях встречается класс, суммарная длительность и покрытие. Класс TAILSLAP отключён автоматически с объяснением причины.

· Строка внизу экрана показывает итоговое значение поля calls — то, что попадёт в параметры.

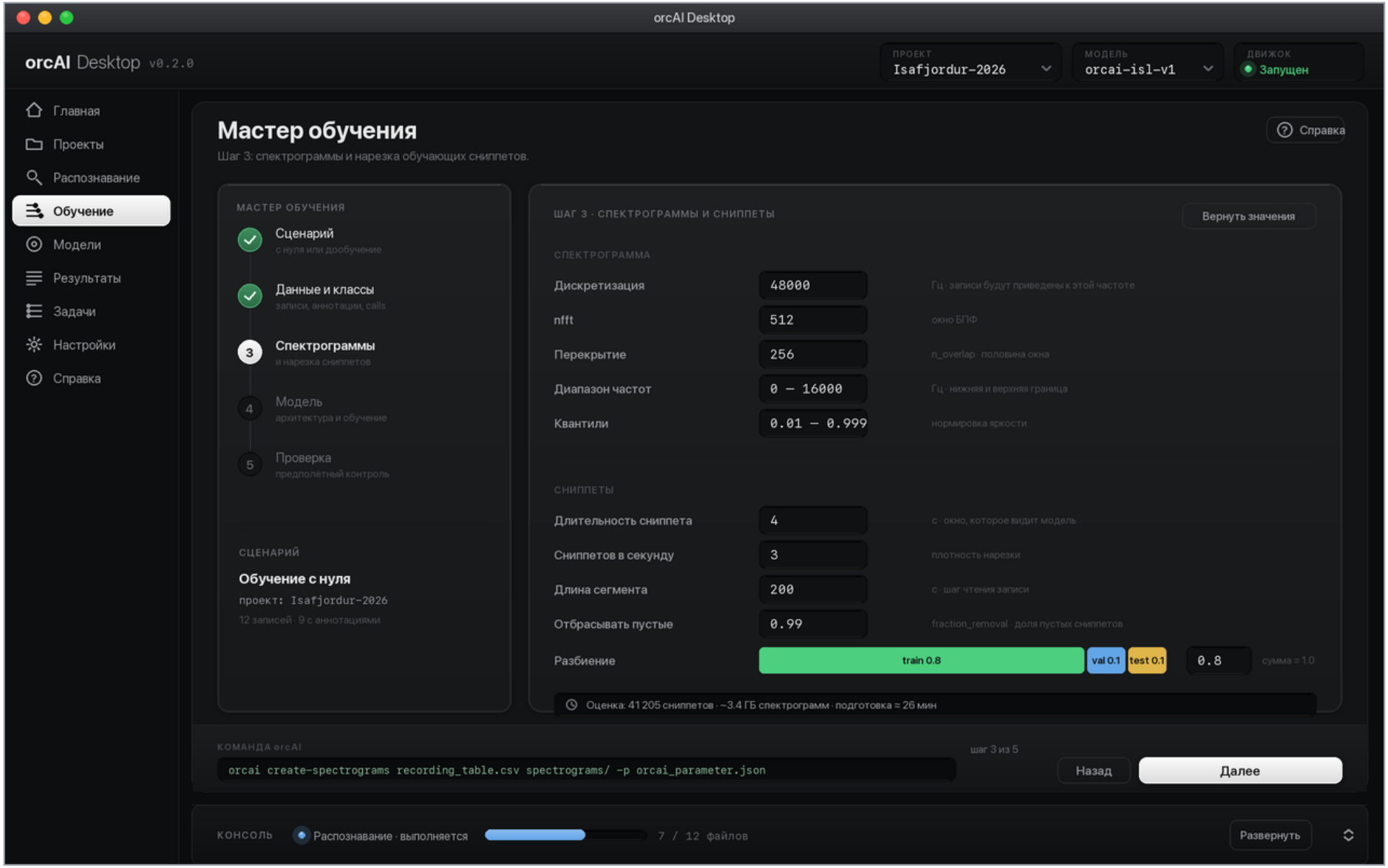
- Файл синонимов сводит разные написания меток к одному классу до построения массивов.
- Записи без аннотаций в обучении не участвуют; их число указано под выбором таблицы.

8.4 Шаг 3: спектрограммы и нарезка

Здесь задаётся представление сигнала — то, что модель будет считать «изображением». Все поля соответствуют разделам spectrogram и snippets файла параметров, значения по умолчанию взяты из оригинала. Мастер добавляет к ним только оценку: сколько сниппетов получится, сколько места займут спектрограммы и сколько времени займёт подготовка.

Оценка нужна не для красоты. Размер выборок задаётся не долями, а произведением «число батчей × размер батча»: по умолчанию это 240 000 обучающих сниппетов. Если записей мало или плотность нарезки занижена, столько сниппетов просто не наберётся, и подготовка прервётся с ошибкой — но уже после того, как будут построены все спектрограммы. Поэтому мастер считает ожидаемое число сниппетов сразу и сравнивает его с требуемым.

Параметр	По умолчанию	На что влияет	Цена ошибки
Частота дискретизации	`48 000 Гц`	Все записи приводятся к этой частоте перед анализом.	Смена частоты делает модель несовместимой с встроенными и требует полной пересборки данных.
Окно БПФ и перекрытие	`512 / 256`	Разрешение по времени и по частоте.	Слишком большое окно размывает короткие сигналы, слишком малое — тональные контуры.
Полоса частот	`0 – 16 000 Гц`	Какая часть спектра попадает в модель.	Сужение полосы отсекает часть сигналов; расширение добавляет шум и объём данных.
Квантили	`0.01 – 0.999`	Нормировка яркости спектрограммы.	Неверная нормировка делает слабые сигналы неразличимыми для сети.
Длительность сниппета	`4 с`	Окно, которое видит модель за один раз.	Короче самого длинного сигнала — модель не увидит его целиком.
Сниппетов в секунду	`3`	Плотность нарезки обучающей выборки.	Больше плотность — больше данных и времени обучения при слабом приросте качества.
Длина сегмента	`200 с`	Шаг чтения записи при нарезке.	Влияет на потребление памяти, но не на результат.
Отбрасывание пустых	`0.99`	Доля сниппетов без сигналов, исключаемых из выборки.	Без отбрасывания выборка почти целиком состоит из тишины и модель учится молчать.
Разбиение	`0.8 / 0.1 / 0.1`	Доли обучающей, проверочной и тестовой выборок.	Сумма ≠ 1 — предупреждение (см. V-1): <1 допустимо (неиспользованные участки сегмента); >1 нужно исправить. Малая проверочная выборка делает раннюю остановку случайной.



Макет 13. Мастер обучения, шаг 3 — спектрограммы и сниппеты

Два блока полей с исходными значениями и пояснением к каждому. Разбиение показано полосой: соотношение выборок видно, а не вычисляется в уме. Внизу — оценка объёма и времени подготовки.

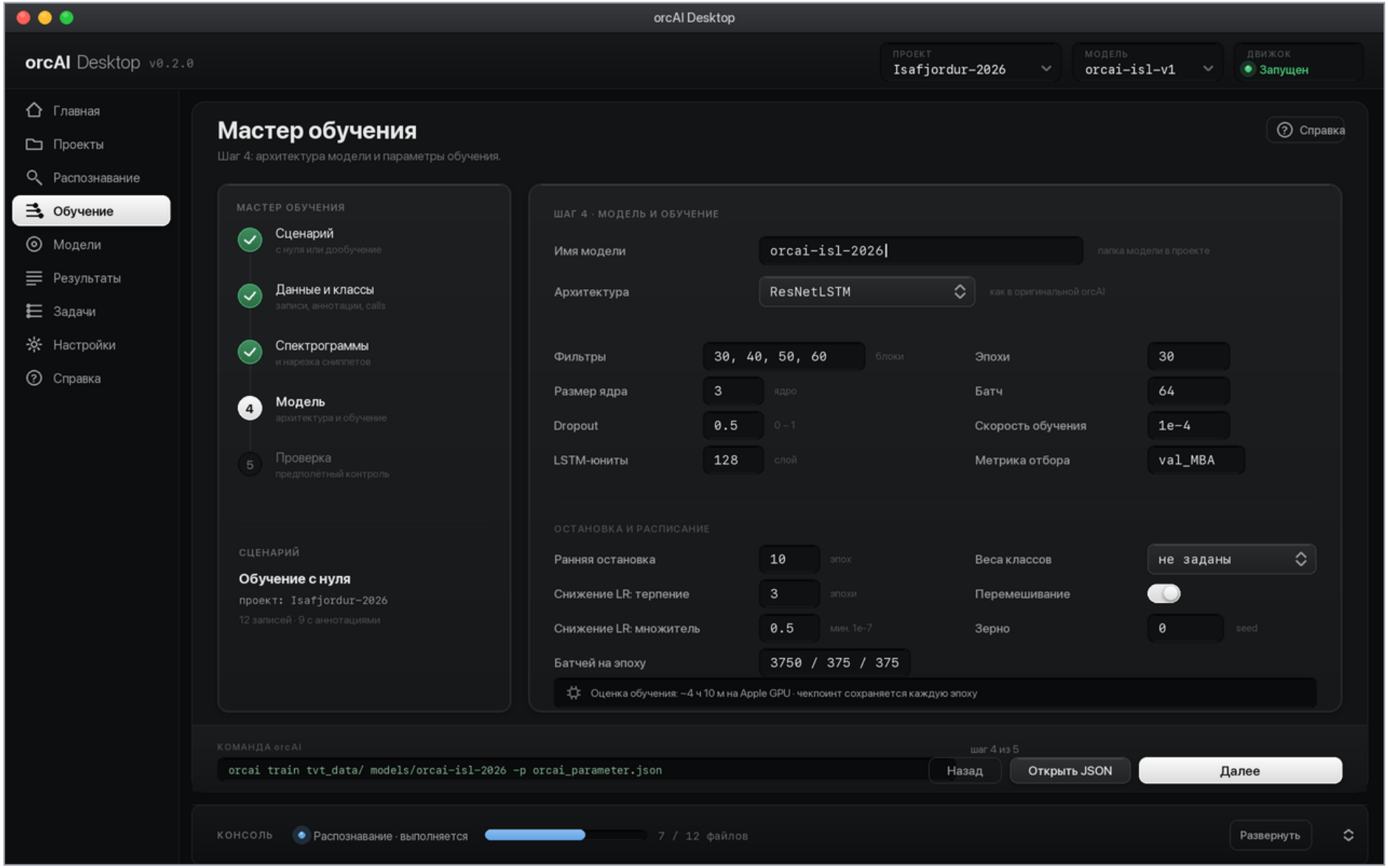
· Кнопка «Вернуть значения» возвращает параметры оригинала — страховка от случайной правки.

- Оценка пересчитывается при изменении плотности нарезки и длительности сниппета.
- При дообучении весь этот шаг заблокирован: параметры берутся из базовой модели.

8.5 Шаг 4: модель и обучение

Поля повторяют раздел model файла параметров и добавляют имя модели. Архитектур в оригинале две: ResNetLSTM (используется во встроенных моделях) и ResNet1DConv. Отдельного внимания заслуживает группа остановки и расписания: именно она определяет, когда обучение закончится фактически. Ранняя остановка срабатывает после десяти эпох без улучшения отслеживаемой метрики, а скорость обучения снижается вдвое после трёх таких эпох — поэтому заявленные тридцать эпох почти никогда не проходят целиком.

> **Метрика val_MBA**
>
> Отслеживаемая метрика по умолчанию — точность по маскированным бинарным выходам (Masked Binary Accuracy) на проверочной выборке. Маска нужна потому, что часть временных отсчётов сниппета не размечена; учитывать их как отрицательные примеры было бы ошибкой. Приложение показывает метрику под её оригинальным именем и поясняет смысл, но не заменяет её собственной.



Макет 14. Мастер обучения, шаг 4 — модель и обучение

Архитектура и имя модели сверху; ниже два столбца параметров сети и обучения; отдельным блоком — правила остановки и снижения скорости обучения. Внизу — оценка длительности обучения и напоминание о сохранении чекпоинта.

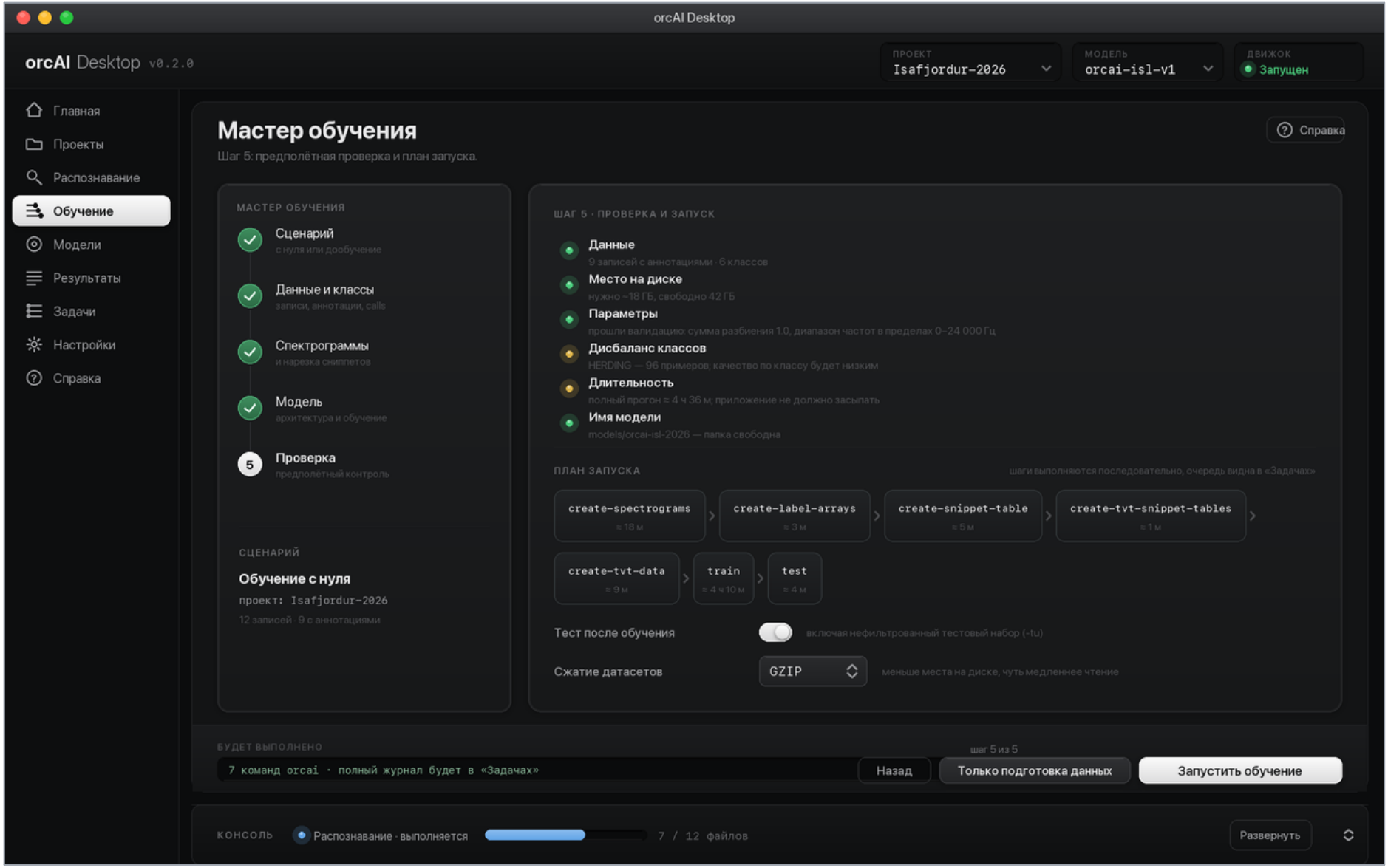
· Имя модели становится именем каталога и именем файла весов — это видно сразу.

- Веса классов по умолчанию не заданы, как в оригинале; поле доступно для правки.
- Зерно генератора вынесено на экран: без него повторяемость результата не гарантируется.

8.6 Шаг 5: предполётная проверка

Последний экран отвечает на вопрос «что сейчас произойдёт и сколько это займёт». Проверки делятся на блокирующие и предупреждающие: первые не дают запустить, вторые требуют осознанного согласия. План показывает конкретные команды, а не абстрактные «этапы».

Правило	Проверяется	Реакция	Основание
V-1	Сумма долей train + val + test равна 1	Предупреждение	Доли делят время каждого сегмента записи. Сумма меньше единицы означает, что часть участков сегмента не попадёт ни в одну выборку (в orcAI это допустимо); больше единицы — что доли пересекаются по смыслу ввода. Оригинал этого не проверяет — приложение обязано предупредить.
V-2	Верхняя граница полосы частот не превышает половины частоты дискретизации	Ошибка	Физическое ограничение спектрального анализа.
V-3	Перекрытие меньше размера окна БПФ	Ошибка	Иначе спектрограмма не строится.
V-4	Выбран хотя бы один класс	Ошибка	Пустой список классов делает обучение бессмысленным.
V-5	У каждого выбранного класса есть примеры в аннотациях	Ошибка для класса без примеров	Класс без примеров даёт мёртвый выход сети.
V-6	Классов с числом примеров ниже порога нет	Предупреждение	Обучение возможно, но качество по классу будет низким; решение остаётся за пользователем.
V-7	Свободного места достаточно для спектрограмм и датасетов	Предупреждение или ошибка	Спектрограммы — самый объёмный артефакт конвейера.
V-8	Каталог модели с таким именем свободен	Ошибка	Иначе результат перезапишет предыдущее обучение.
V-9	При дообучении набор классов совпадает с базовой моделью	Ошибка, поля заблокированы	Размер выхода сети зафиксирован при первом обучении (INV-2).
V-10	При дообучении параметры спектрограммы совпадают с базовой моделью	Ошибка, поля заблокированы	Форма входа зафиксирована при первом обучении (INV-3).
V-11	Для теста существует тестовый датасет	Шаг теста исключается из плана	Команда test читает test_dataset из каталога данных.
V-12	Для поиска гиперпараметров датасеты уже собраны	Шаг блокируется с пояснением	hpsearch работает с готовыми данными, а не с записями.
V-13	Сниппетов хватает на заданный размер выборок	Ошибка, запуск заблокирован	Размер выборки равен «число батчей × размер батча»: по умолчанию 240 000 обучающих, 24 000 проверочных и 24 000 тестовых сниппетов. Если нарезка дала меньше, подготовка прерывается с ошибкой посреди многочасового прогона — эту ситуацию нужно поймать до запуска.
V-14	Каталоги датасетов ещё не созданы	Ошибка с предложением очистить каталог	Оригинал отказывается писать в существующий каталог датасета; молча продолжить нельзя.



Макет 15. Мастер обучения, шаг 5 — проверка и запуск

Шесть проверок с цветными индикаторами: зелёные пройдены, жёлтые требуют внимания. План запуска — семь команд с оценкой времени каждой. Внизу можно ограничиться подготовкой данных, не запуская обучение.

· Оценка полного прогона выводится до запуска: пользователь понимает, что это часы, а не минуты.

- Кнопка «Только подготовка данных» полезна, когда обучение будет запущено позже или на другой машине.
- Все команды плана попадают в очередь «Задачи» и выполняются последовательно.

8.7 Ветка дообучения

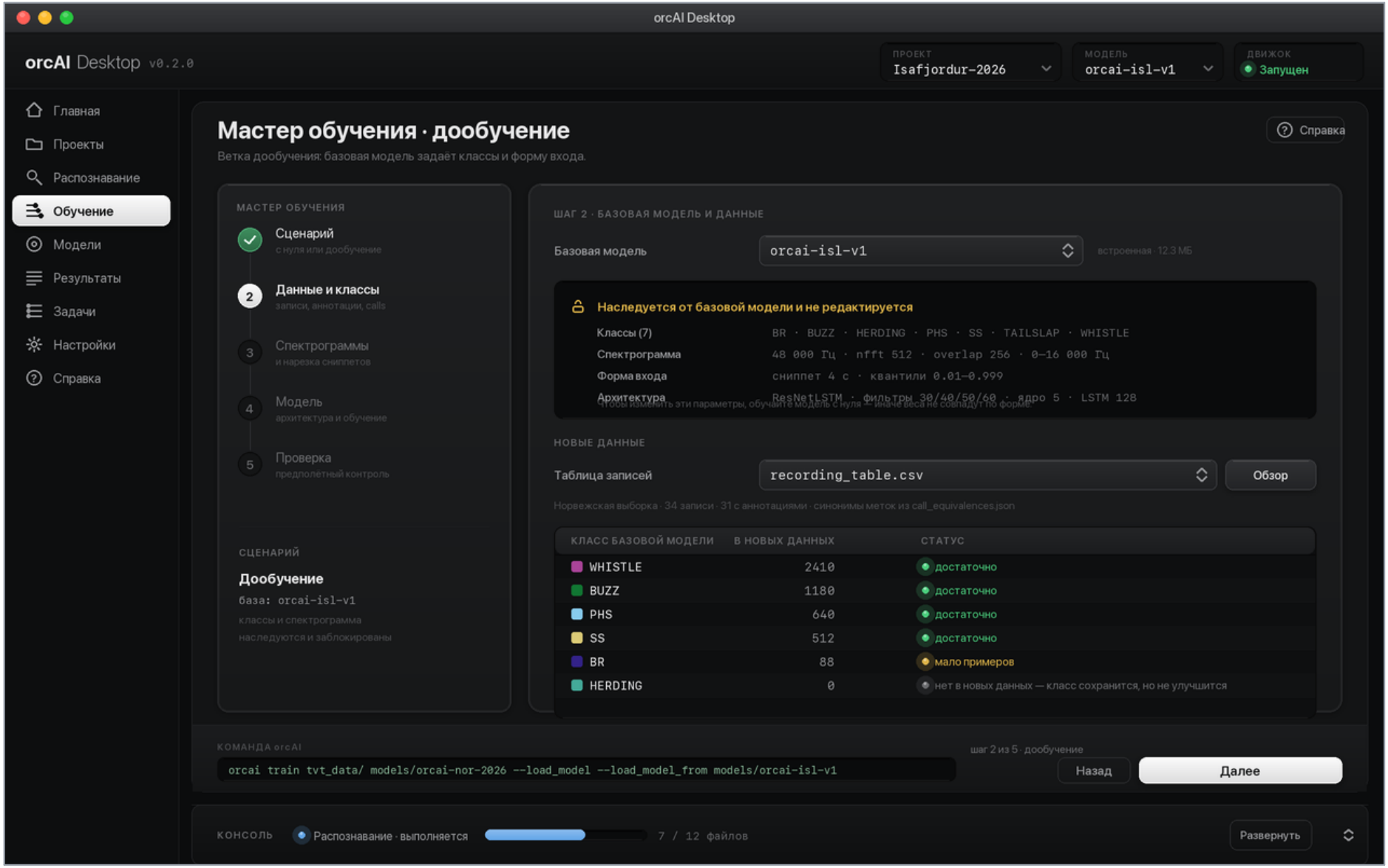
Дообучение — это продолжение обучения с весов готовой модели. В оригинале оно выполняется той же командой train с указанием каталога модели, откуда брать веса. Отсюда следует жёсткое ограничение, которое мастер обязан не обойти, а объяснить: форма входа и выхода сети зафиксирована при первом обучении. Значит, набор классов, параметры спектрограммы, длительность сниппета и архитектура наследуются от базовой модели и не подлежат изменению. Если пользователю нужен другой набор классов или другая полоса частот — это обучение с нуля, и мастер должен сказать об этом прямо, а не молча принять правку.

Почему эту проверку нельзя переложить на orcAI

При дообучении оригинал берёт из каталога модели только веса и архитектуру, а список классов и параметры спектрограммы читает из переданного файла параметров — и не сверяет одно с другим. Несовпадение проявится не понятным сообщением, а сбоем формы тензора где-то в середине обучения. Значит, проверка совместимости — обязанность приложения, а не приятное дополнение.

Наследуется от базовой модели	Задаётся заново
Набор классов и их порядок (определяет размер выхода сети).	Новые данные: таблица записей, каталог аннотаций, файл синонимов.
Все параметры спектрограммы: частота, окно, перекрытие, полоса, квантили.	Имя новой модели — исходная модель не изменяется.
Длительность сниппета и форма входа.	Число эпох: при дообучении обычно требуется меньше.
Архитектура и её гиперпараметры: фильтры, ядро, отсев, размер рекуррентного слоя.	Скорость обучения: как правило, ниже, чем при обучении с нуля.
—	Правила ранней остановки и снижения скорости обучения.

Мастер дополнительно сверяет классы базовой модели с новыми аннотациями. Класс, которого нет в новых данных, не исчезает из модели — он просто не будет улучшен; об этом сообщается явно, потому что иначе пользователь ожидает от дообученной модели равномерного улучшения по всем классам.



Макет 16. Мастер обучения, ветка дообучения

Выбор базовой модели и заблокированный блок наследуемых параметров с пояснением, почему их нельзя изменить. Ниже — новые данные и сверка классов базовой модели с аннотациями: видно, что HERDING и TAILSLAP в новой выборке отсутствуют.

· Строка команды внизу показывает train с параметрами --load_model и --load_model_from.

- Левая колонка напоминает выбранный сценарий и базовую модель на всех шагах.
- Исходная модель остаётся неизменной: результат сохраняется как новая модель проекта.

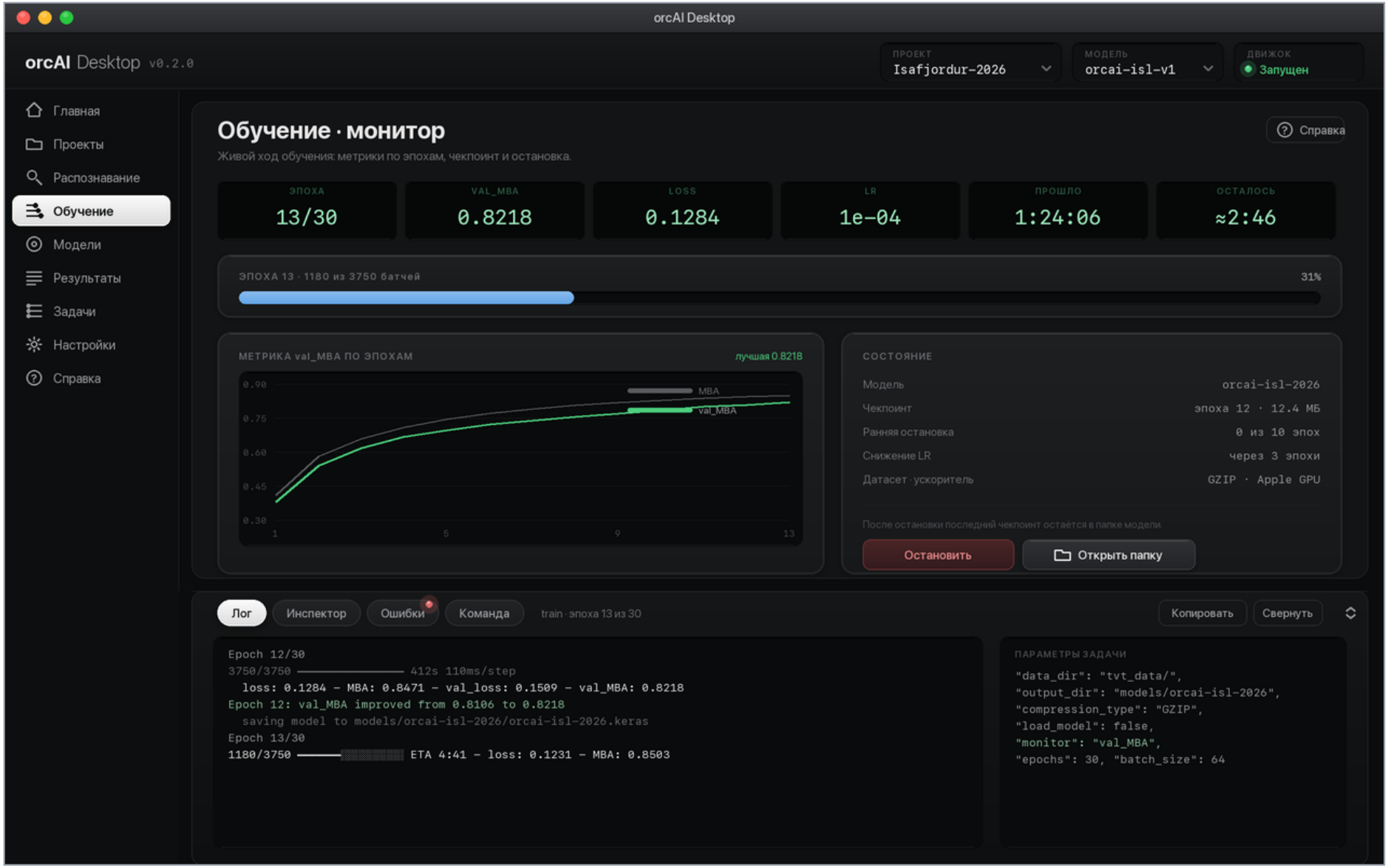
8.8 Монитор обучения

Обучение длится часы, поэтому наблюдение за ним — отдельная задача интерфейса. Монитор показывает то, что и так печатает orcAI в журнал, но в виде, пригодном для быстрого взгляда: номер эпохи, отслеживаемую метрику, функцию потерь, текущую скорость обучения, прошедшее и оставшееся время. График метрики по эпохам отвечает на главный вопрос — продолжает ли модель улучшаться.

Показатель	Откуда берётся	Зачем пользователю
Эпоха и прогресс по батчам	Вывод обучения	Понять, на каком этапе прогон и сколько осталось.
Метрика на проверочной выборке	monitor из параметров	Оценить, растёт ли качество.
Функция потерь	Вывод обучения	Заметить расхождение обучающей и проверочной кривых.
Скорость обучения	Расписание снижения	Понять, что сработало снижение скорости.
Счётчик эпох без улучшения	Правило ранней остановки	Предсказать, когда прогон завершится сам.
Чекпоинт	Файл весов в каталоге модели	Знать, что результат уже сохранён и остановка не потеряет работу.

Остановка обучения

Прерывание не должно означать потерю работы. Веса сохраняются при каждом улучшении метрики, поэтому после остановки в каталоге модели остаётся последний удачный чекпоинт. Продолжить можно тем же механизмом, что и дообучение, — указав этот каталог как источник весов.



Макет 17. Монитор обучения

Показания на ЖК-индикаторах, прогресс текущей эпохи, график метрики с отметкой лучшего значения и панель состояния. Консоль развёрнута: виден настоящий вывод обучения, включая сообщение о сохранении чекпоинта.

· Серая кривая — метрика на обучающей выборке, зелёная — на проверочной: расхождение видно сразу.

- Панель состояния показывает, сколько эпох осталось до срабатывания ранней остановки.
- Остановка доступна в один клик и не требует подтверждения через терминал.

8.9 Поиск гиперпараметров

Третий сценарий мастера работает с уже подготовленными датасетами и перебирает конфигурации из отдельного файла диапазонов: три набора фильтров, размер рекуррентного слоя (64 или 128), доля отсева (0.3, 0.4, 0.5), размер ядра (3, 5, 7) и размер батча. Перебор идёт методом Hyperband с ограничением в пятнадцать эпох на конфигурацию, поэтому число проб заранее не фиксировано. На выходе появляются файл лучших гиперпараметров, таблица всех проб и чекпоинт лучшей конфигурации; мастер предлагает перенести найденные значения в параметры проекта, после чего запускается полноценное обучение.

Что важно показать пользователю	Почему
Число конфигураций и суммарную оценку времени до запуска.	Перебор дольше обычного обучения в разы; это должно быть известно заранее.
Требование готовых датасетов.	Команда работает с данными, а не с записями; без подготовки шаг блокируется.
Диапазоны перебора в виде таблицы, а не файла.	Пользователь должен понимать, что именно перебирается.
Явное действие «перенести лучшую конфигурацию в параметры проекта».	Автоматическая подмена параметров была бы скрытым изменением научной постановки.

9. Дорожная карта

Пять этапов, каждый из которых заканчивается состоянием, пригодным для работы. Порядок продиктован не удобством разработки, а тем, что раньше нужно пользователю.

Этап	Версия	Тема	Состав работ	Что появляется у пользователя	Требования
Этап 1	**0.2**	**Достоверность и разбор результата**	Контекстная строка на всех экранах (сквозной фрагмент П-1, может опережать полную «Главную»); сквозной фундамент настроек (ЭК-3а / П-13а: RU/EN, масштаб, Full Screen); консоль с вкладками «Ошибки» и «Команда», регулируемая высота; полный набор параметров predict; очередь запуска с отменой и повтором; просмотрщик результатов с легендой, вероятностями и таблицей детекций.	Пользователь распознаёт проект, разбирает результат и понимает, какая команда была выполнена.	FR-3.*, FR-4.*, FR-10.4 – FR-10.8, FR-11.1, FR-11.8, FR-11.9, NFR-17, NFR-18
Этап 2	**0.3**	**Фильтрация, результаты, экспорт**	Редактор пределов длительности с предпросмотром; сводный раздел «Результаты» с фильтрами и показателями; экспорт CSV и меток Audacity; переключение источника данных.	Результаты выгружаются из приложения без обращения к файловой системе.	FR-5.*, FR-6.*
Этап 3	**0.4**	**Проект как каталог артефактов**	Вкладки проекта; мастер создания с мягким фокусом; панель готовности данных; таблица записей со всеми столбцами; обновление таблицы, исключения и дубликаты; просмотр и редактирование параметров проекта; список артефактов; настройки, влияющие на команды.	Состояние проекта полностью видно в интерфейсе; терминал не нужен для подготовки данных.	FR-2.*, FR-11.*
Этап 4	**0.5**	**Конвейер обучения и мастер**	Доска конвейера с шестью состояниями и артефактами; панель готовности данных; мастер обучения из пяти шагов с предполётной проверкой; монитор обучения; тестирование и метрики в карточке модели.	Модель обучается с нуля целиком из интерфейса, с честным отображением хода работ.	FR-7.*, FR-8.1 – FR-8.6, FR-8.9 – FR-8.14, FR-9.3, FR-9.4
Этап 5	**0.6**	**Дообучение и подбор**	Ветка дообучения с наследованием параметров и сверкой классов; сценарий hpsearch (UI мастера и приёмка — после готовых TVT-датасетов); перенос лучшей конфигурации; импорт и проверка совместимости моделей; сравнение распознавания с аннотациями.	Полное покрытие функционала оригинала; ни один сценарий не требует командной строки.	FR-8.7, FR-8.8, FR-7.12, FR-9.6 – FR-9.8, FR-4.9

9.1 Критерии приёмки этапов

Этап	Проверка на реальном проекте
Этап 1	На проекте из двенадцати записей пакетное распознавание проходит целиком; одну запись удаётся отменить и повторить; в консоли есть команда, совпадающая с той, что выполнил движок; в просмотрщике видны классы, вероятности и таблица детекций.

Этап	Проверка на реальном проекте
Этап 2	Пределы длительности задаются в таблице, предпросмотр совпадает с фактическим результатом фильтрации; экспортированный файл меток открывается в Audacity как дорожка.
Этап 3	Проект, созданный в командной строке, полностью читается приложением; мастер фокуса не блокирует команды orcAI; готовность данных совпадает с каталогом; после добавления записей таблица обновляется с сохранением ранее заданных каналов; изменение параметров проекта отражается в файле параметров.
Этап 4	Модель обучается с нуля через мастер без единого обращения к терминалу; состояния конвейера совпадают с фактическим содержимым каталога; после изменения параметров нарезки зависимые шаги помечаются устаревшими; остановка обучения сохраняет чекпоинт.
Этап 5	Встроенная модель дообучается на новых данных; попытка изменить набор классов при дообучении блокируется с объяснением; лучшая конфигурация переносится в параметры проекта явным действием.

9.2 Перенос на Linux и Windows

Перенос — отдельная ветка (ЭК-17, ЭК-18), но **подготовка начинается сразу после sidecar** (ЭК-2 / П-15): пути, регистр, шрифты, абстракция ускорителя. Параллельно с первыми экранами действует **ЭК-1 / П-19** (Clean Code): sidecar пишется по PEP 8 с первого коммита, а не «перед релизом». Функциональные требования при переносе не меняются (NFR-16); platform-код — только в оболочке и сборке (NFR-15).

Шаг	Состав работ	Признак завершения
Порт-0. Clean Code	Ruff/flake8 sidecar; именование, длина функций, без рефакторинга orcAI.	Каждый PR проходит линтер; NFR-19 выполнен для нового кода.
Порт-А. Подготовка	Пути через API оболочки, case-sensitive сравнение имён, отказ от системных шрифтов macOS, слой «ускоритель среды».	Сборка проходит на Linux в контейнере без правок UI и службы.
Порт-Б. Linux	Автономная сборка Python, AppImage и .deb, CUDA при наличии драйвера.	Predict и укороченный train на Ubuntu LTS без терминала.
Порт-В. Windows	MSI/NSIS, long paths, Authenticode, **DirectML для GPU**, CPU fallback с предупреждением до train.	На Win10/11 predict и train работают; без GPU — честное сообщение и CPU.
Порт-Г. Сопровождение	Одно описание сборки для трёх платформ; матрица приёмки.	Выпуск не требует ручных расхождений по платформам.

Стабильность оболочки

NFR-20 (отзывчивость UI, память, отмена задач) проверяется на этапах с длительными задачами (ЭК-3, ЭК-12–ЭК-14) и перед релизом ветки портов. Исходники bundled orcAI не рефакторятся.

10. Риски и приложения

Реестр рисков с мерами, сопоставление требований с командами оригинала, глоссарий и чек-лист приёмки.

10.1 Реестр рисков

ID	Риск	Вероятн.	Влияние	Мера
R-1	Обновление orcAI меняет состав или значения по умолчанию команд	Средняя	Высокое	Значения по умолчанию и список параметров читать из движка, а не хранить копией в приложении; при расхождении показывать предупреждение.
R-2	Долгое обучение прерывается сном компьютера	Высокая	Высокое	Предупреждать перед запуском; удерживать систему от сна на время задачи; сохранять чекпоинт каждую эпоху.
R-3	Нехватка места при построении спектрограмм	Средняя	Среднее	Оценивать объём до запуска, проверять свободное место, показывать занимаемый артефактами объём.
R-4	Пользователь ожидает от дообучения улучшения по всем классам	Высокая	Среднее	Сверка классов с новыми данными на шаге мастера и явное сообщение о классах без примеров.
R-5	Интерфейс расходится с оригиналом в именах артефактов	Низкая	Высокое	Имена формируются движком; приложение только отображает полученные пути.
R-6	Спектрограмма в интерфейсе строится иначе, чем вход модели	Низкая	Высокое	Использовать один и тот же код построения; запретить альтернативные режимы отображения (INV-1).
R-7	Очередь задач конкурирует за ускоритель	Средняя	Среднее	По умолчанию одна задача одновременно; параллельность — осознанная настройка.
R-8	Проект, созданный в другой версии, читается неверно	Средняя	Среднее	Хранить версию параметров в файле проекта; при несовпадении предупреждать, а не молча продолжать.
R-9	Перевод терминов искажает смысл	Средняя	Среднее	Имена классов, файлов и параметров не переводятся; переводится только пояснение.
R-10	Пользователь запускает обучение на данных без аннотаций	Средняя	Среднее	Предполётная проверка блокирует запуск и показывает, сколько записей размечено.
R-11	Пакетное распознавание в общий каталог перезаписывает результаты разных каналов	Средняя	Среднее	В этом режиме оригинал не добавляет номер канала в имя файла. Предупреждать, когда в очереди есть одна запись с разными каналами, и предлагать раскладку по подкаталогам.
R-12	Документация оригинала расходится с его же кодом	Высокая	Низкое	Опираться на исходный код, а не на README: там встречаются устаревшее имя команды filter-annotations, старое имя модели и режим «не сохранять результат», который в коде приводит к ошибке. Приложение всегда передаёт явный путь вывода.
R-13	Поведение расходится между платформами	Средняя	Среднее	Чек-лист приёмки прогоняется на каждой платформе; различия, которые нельзя устранить (отсутствие ускорителя под Windows), показываются пользователю честно и до запуска, а не выясняются по времени выполнения.

ID	Риск	Вероятн.	Влияние	Мера
R-14	Различение регистра в именах файлов на Linux	Средняя	Низкое	Сопоставление записей и аннотаций по имени файла ведётся с учётом регистра с самого начала, иначе на Linux в таблице записей появятся дубликаты.

Приложение А. Соответствие требований командам оригинала

Каждая команда orcAI закрыта требованиями документа. Требования без опоры в CLI есть только там, где командной строки по определению нет: окна, очередь, язык, наблюдаемость.

Команда orcAI	Требования	Где в интерфейсе
`init`	`FR-2.1, FR-2.12, FR-2.14, FR-2.15`	Создание проекта, мастер фокуса, файлы настроек
`create-recording-table`	`FR-2.3 – FR-2.6, FR-2.8, FR-2.10`	Таблица записей, каналы, аннотации, обновление
`predict`	`FR-3.1 – FR-3.15, FR-9.5`	Вкладка «Параметры», очередь запуска, выбор модели
`filter-predictions`	`FR-5.1 – FR-5.8`	Вкладка «Фильтрация», редактор пределов, предпросмотр
`create-spectrograms`	`FR-7.4, FR-8.5`	Шаг 1 конвейера, шаг 3 мастера
`create-label-arrays`	`FR-7.5, FR-8.3`	Шаг 2 конвейера, выбор классов и синонимов
`create-snippet-table`	`FR-7.6, FR-8.5`	Шаг 3 конвейера, параметры нарезки
`create-tvt-snippet-tables`	`FR-7.6, FR-7.7`	Шаг 4 конвейера, нефильтрованный тестовый набор
`create-tvt-data`	`FR-7.8`	Шаг 5 конвейера, выбор сжатия
`hpsearch`	`FR-7.12, FR-8.2`	Шаг 6 конвейера, третий сценарий мастера
`train`	`FR-7.9, FR-7.10, FR-8.6 – FR-8.14`	Шаг 7 конвейера, мастер, монитор, дообучение
`test`	`FR-7.11, FR-9.4, FR-9.6`	Шаг 8 конвейера, метрики в карточке модели

Приложение Б. Глоссарий

Соответствие русских терминов интерфейса именам оригинала. Правая колонка — то, что пользователь увидит в именах файлов и параметрах: эти слова не переводятся.

Термин интерфейса	Имя в orcAI	Значение
Распознавание	`predict`	Раздел интерфейса и команда orcAI; в RU — «Распознавание», не «предсказание».
Файл распознавания	`*_predicted.txt`	Артефакт команды predict; слово «предсказание» допустимо только для файлов и CLI, не для названия раздела.

Термин интерфейса	Имя в orcAI	Значение
Таблица записей	`recording_table.csv`	Перечень записей проекта с каналом, путями к файлу и аннотации.
Аннотация	`annotation`	Ручная разметка сигналов, служащая обучающим примером.
Класс сигнала	`call`	Тип акустического сигнала: WHISTLE, BUZZ, PHS, SS, BR, HERDING, TAILSLAP.
Синонимы меток	`call_equivalences`	Соответствие разных написаний меток одному классу.
Сниппет	`snippet`	Короткий фрагмент спектрограммы фиксированной длительности — единица обучения.
Массив меток	`label array`	Разметка, переведённая во временную сетку спектрограммы.
Выборки	`train / val / test`	Обучающая, проверочная и тестовая части данных.
Порог	`threshold`	Минимальная вероятность, при которой сигнал считается обнаруженным.
Средняя вероятность	`mean_p`	Уверенность модели в конкретной детекции.
Пределы длительности	`call_duration_limits`	Минимальная и максимальная длительность сигнала по классам.
Чекпоинт	`checkpoint`	Сохранённые веса модели на определённой эпохе.
Метрика val_MBA	`Masked Binary Accuracy`	Точность на проверочной выборке с учётом неразмеченных участков.
Дообучение	`fine-tuning`	Продолжение обучения с весов существующей модели на новых данных.
Движок	`orcAI engine`	Библиотека orcAI 3.0.2 внутри приложения, выполняющая все вычисления.
Sidecar	`sidecar`	Локальная служба, через которую интерфейс вызывает движок.
SQLite	–	Встроенное хранилище службы: очередь задач, настройки, реестр проектов.
DuckDB	–	Встроенный аналитический движок: индексы CSV, сводки детекций и метрик test.

Приложение В. Чек-лист приёмки

Четырнадцать проверок (К-1…К-14), каждая из которых отвечает на вопрос «не разошлись ли мы с оригиналом» или «не потеряли ли мы переносимость». Идентификаторы К-n не пересекаются с промтами реализации П-n из §11. Пройденный чек-лист — необходимое условие выпуска любой версии.

Пошаговая ручная форма с полями Pass/Fail и доказательствами: `docs/orcAI-Desktop-чек-лист-приёмки.md` (копируется в пакет Cursor).

№	Проверка	Основание
К-1	Все 12 команд оригинала запускаются из интерфейса	`Раздел 3, приложение A`
К-2	Значения по умолчанию совпадают с CLI	`INV-6`
К-3	Имена артефактов совпадают с теми, что создаёт orcAI	`INV-4`
К-4	Точная команда видна перед запуском и после него	`FR-3.14, FR-10.5`
К-5	Состояния шагов конвейера соответствуют содержимому каталога проекта	`FR-7.2`
К-6	Спектрограмма в интерфейсе построена тем же кодом, что и вход модели	`INV-1`
К-7	Дообучение с изменением классов заблокировано	`INV-2, V-9`
К-8	Фильтрация не изменяет исходный файл предсказаний	`INV-5`
К-9	Отмена задачи не оставляет частичных артефактов, помеченных как готовые	`NFR-6`
К-10	Полный сценарий обучения проходится без терминала	`NFR-9`
К-11	Приложение работает на чистой системе без Python и Homebrew	`NFR-1`
К-12	Интерфейс полностью переведён на русский и английский; имена файлов и классов не переведены	`NFR-10, FR-11.1`
К-13	Сценарии раздела 4 проходятся одинаково на macOS, Linux и Windows	`NFR-16`
К-14	Платформозависимый код не выходит за пределы оболочки и сборки пакета	`NFR-15`

Приложение Г. Комплект LICENSE / NOTICE

Обязательные файлы поставки (FR-11.10) и их связь с разделом «Справка» (П-16).

Файл	Содержание	Где проверять
`LICENSE`	Текст CC BY-SA 4.0 для orcAI Desktop (автор — Vadim Romanov).	Корень .app / установщика; вкладка «Лицензия» в Справке.
`NOTICE`	Attribution orcAI 3.0.2 и перечень изменений Desktop относительно upstream.	Корень пакета; фрагмент на m21-help-license.png.

Файл	Содержание	Где проверять
`THIRD_PARTY_LICENSES`	Лицензии bundled-зависимостей (Python, TensorFlow, Tauri и др.).	Корень пакета; вкладка «Сторонние» (m22).

FR-11.11: в UI и README запрещены формулировки, что ETH Zürich или авторы orcAI «одобрили» или «совместно выпустили» Desktop без их письменного согласия (CC BY-SA §2(a)(6)).

Итог документа

orcAI Desktop 0.1.8 — работающая основа с хорошо выбранным визуальным языком и правильной архитектурой: автономный движок, ручной старт, честный журнал. Недостаёт не столько функций, сколько наблюдаемости — приложение уже умеет запускать почти всё, но не показывает, что произошло. Документ описывает 141 требование (121 FR + 20 NFR), 24 сценария, 40 PNG (17 основных макетов + 21 дополнительный + 2 схемы), которые закрывают этот разрыв, не отступая ни в одном месте от поведения оригинальной orcAI 3.0.2.

11. Промты: реализация и тестирование

Двадцать пар «промт реализации + промт тестирования» (включая П-13a / Т-13a). Сначала П-n с макетом (если есть), затем Т-n на реальных данных. П-19 и П-15 — сразу после sidecar (ЭК-1, ЭК-2); П-17/П-18 — порты Linux/Windows.

Разработка продолжается в Cursor на основной машине, в каталоге проекта `orcai-desktop`. Схема сеанса: общий контекст → текст П-n + PNG-макет → реализация → новый сеанс или продолжение с текстом Т-n (без дописывания функций — только проверка). Макеты: `docs/ui-memory/tz2-mockups/` и `/tmp/orcAI-Desktop-TZ-макеты/`.

Как пользоваться	Что приложить / сдать
Открыть Cursor в каталоге проекта (macOS для П-1…П-16; Linux/Windows VM для П-17/П-18).	PNG-макет к П-n; к Т-n — скрины и краткий отчёт pass/fail.
Вставить общий контекст, затем П-n. После «Готово, когда» — вставить Т-n.	Т-n не меняет код без бага; при нахождении дефекта — фикс отдельным шагом, затем повтор Т-n.
Один раздел (этап ЭК) за сеанс. П-19=ЭК-1 и П-15=ЭК-2 — сразу после sidecar; П-15 → П-17 → П-18 для портов; повтор Т-19 перед релизом.	Для портов: d02 + DirectML (Windows) + лог TF. Для П-19 — отчёт ruff.
Full Screen и DPI входят в Т-13 и в приёмку П-17/П-18 (NFR-17, NFR-18).	Матрица разрешение × масштаб × Full Screen для Т-13.

Пара	Раздел	Макет	Этап	ТЗ
П-1 / Т-1	Главная	`m01, m37`	1	`FR-1, FR-2.16`
П-2 / Т-2	Проекты	`m02, m23–m25, m32–m36`	1–2	`FR-2`
П-3 / Т-3	Распознавание · Параметры	`m03`	1	`FR-3`
П-4 / Т-4	Распознавание · Результаты	`m04`	1	`FR-4`
П-5 / Т-5	Распознавание · Фильтрация	`m05`	2	`FR-5`

Пара	Раздел	Макет	Этап	ТЗ
П-6 / Т-6	Результаты	m06	2	FR-6
П-7 / Т-7	Обучение · конвейер	m07	3–4	FR-7
П-8 / Т-8	Мастер · шаги 1–5	m08–m12	4	FR-8
П-9 / Т-9	Мастер · дообучение	m13	5	FR-8
П-10 / Т-10	Монитор обучения	m14	4	FR-8
П-11 / Т-11	Модели	m15	4–5	FR-9
П-12 / Т-12	Задачи и консоль	m16, m26, m27	1	FR-10
П-13а / Т-13а	Настройки · фундамент (ЭК-3а)	m28	0.2	FR-11.1, 11.8–11.9, NFR-17/18
П-13 / Т-13	Настройки · интеграция (ЭК-10)	m17, m31	0.4	FR-11.2–11.5, 11.9
П-14 / Т-14	Sidecar: контракт	d01	0	NFR-3–8, FR-10.*
П-15 / Т-15	Подготовка к переносу	d02	0 (ЭК-2)	NFR-15, NFR-16
П-16 / Т-16	Справка · manual + лицензия / авторы	m18–m22, m38	3	FR-11.6–11.14
П-17 / Т-17	Портирование Linux + CUDA/CPU	d02	ветка	NFR-1, 12, 15–18
П-18 / Т-18	Портирование Windows + DirectML	d02	ветка	NFR-1, 12, 15–18
П-19 / Т-19	Clean Code · стабильность · производительность	d01	0 (ЭК-1)	NFR-3, 4, 19, 20

Общий контекст — вставляется перед любым из промптов

```
Контекст проекта (не менять без явного указания):
· Workspace: /Users/vadimromanov/.cursor/projects/empty-window/orcai-desktop
· orcAI Desktop — Tauri 2 + React/TypeScript + Python-sidecar (FastAPI, runner.py) + встроенная orcAI 3.0.2.
· Движок не форкается. Любая предметная логика — вызов команды orcAI, а не собственная реализация.
· UI только Lab: тёмная тема, левое меню из девяти разделов, нижняя консоль. В 0.1.8: «Лог + Инспектор»; целевые вкладки «Ошибки» и «Команда» — по П-12 (m26, m27).
· Языки RU и EN. В RU predict называется «Распознавание» (не «предсказание» в UI; «предсказание» — только для имён файлов *_predicted.txt). Имена файлов, классов и параметров не переводятся.
· Sidecar и движок стартуют вручную, порт динамический. Закрытие окна скрывает приложение.
· Версия кода 0.1.8. Подпись v0.2.x на макетах — целевой chrome Т3; версию не бампать, сборку .app не ломать.
· Значения по умолчанию берутся из orcAI, а не задаются в приложении заново.
· Окно: обычный и полноэкранный режим; UI следует разрешению и системному DPI (мин. комфортно 1280×800; проверить также 1920×1080 и масштаб 125/150 %).
· Python (sidecar и модули приложения): Clean Code + PEP 8 с первого коммита (ЭК-1 / П-19); проверка ruff/flake8. Код внутри бандла orcAI не рефакторить.
· Кросс-платформа: пути через API оболочки, case-sensitive имена, абстракция ускорителя (ЭК-2 / П-15) — до портов Linux/Windows.
· Макеты: docs/ui-memory/tz2-mockups/ и docs/cursor-handoff/mockups/.
· Обоснование: docs/cursor-handoff/orcAI-Desktop-TZ-dorozhnaya-karta.pdf (или /tmp/…), разделы 3, 7, 8, 11 и 12.
```

Одно правило важнее остальных

Если предложенное решение противоречит поведению оригинальной orcAI 3.0.2 — меняется решение, а не движок. Движок внутри пакета не форкается и не правится; любая предметная логика остаётся вызовом команды orcAI.

Ускорители на портах

macOS: Metal. Linux: CUDA при NVIDIA, иначе CPU с предупреждением. Windows (базовый MSI): **DirectML** для GPU; при недоступности — CPU с предупреждением до train. WSL2+CUDA — опциональный канал, не смешивать с MSI. UI показывает фактическое устройство до запуска train (NFR-12, раздел 1.7).

П-1 · Главная

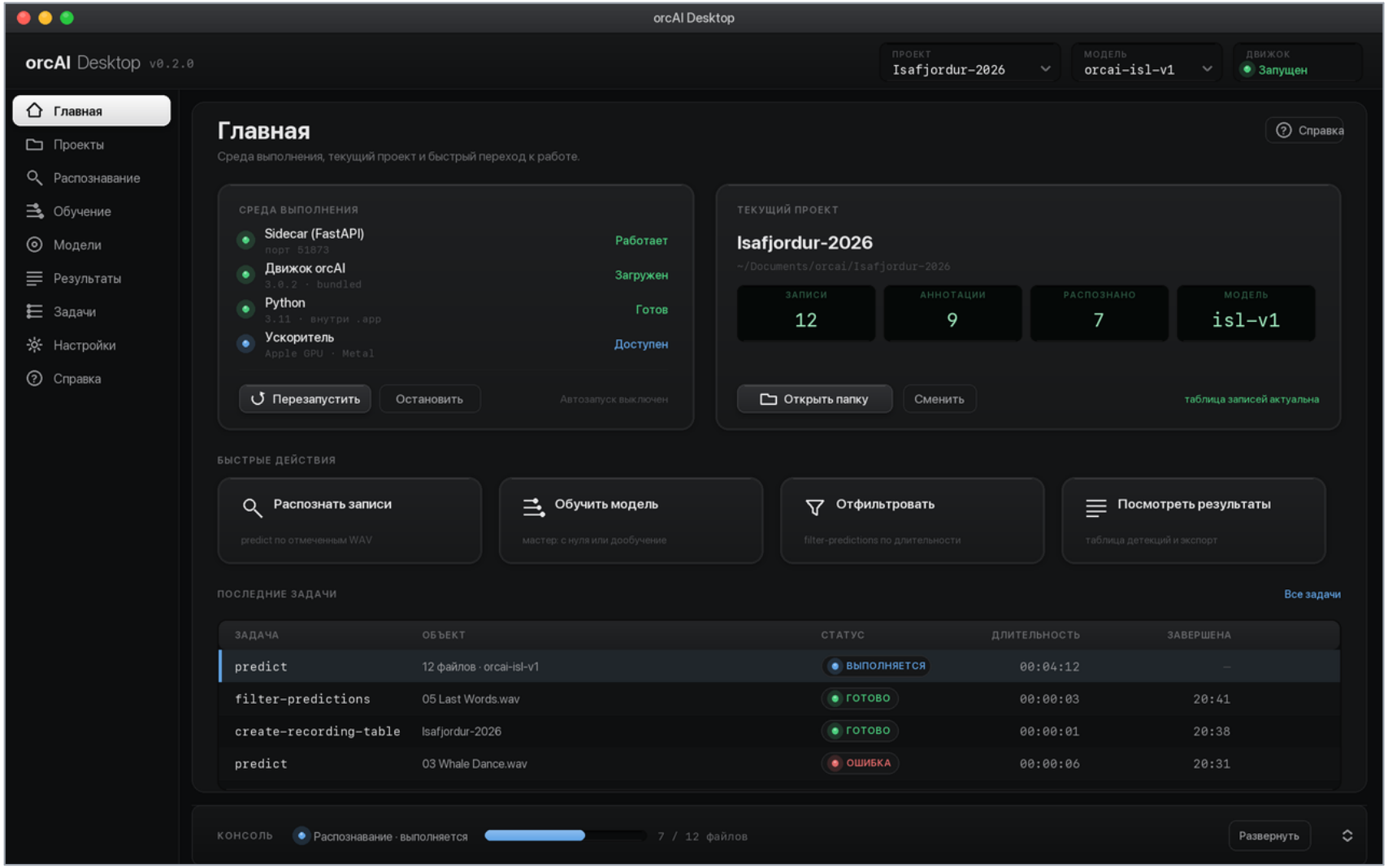
макет: m01-home.png
FR-1.1 — FR-1.7, FR-2.11, FR-2.16 · UC-01

Цель: Превратить экран статуса в экран запуска работы

```
Переделай раздел «Главная» по приложенному макету.

Сделать:
1. Панель «Среда выполнения»: четыре строки — Sidecar, движок orcAI, Python, ускоритель.
У каждой строки индикатор, версия или путь мелким шрифтом и своё состояние.
Кнопки «Перезапустить» и «Остановить» + переключатель автозапуска (по умолчанию выключен).
2. Карточка текущего проекта: имя, путь, четыре показателя на ЖК-индикаторах —
записи, аннотации, распознано, активная модель. Кнопки «Открыть папку» и «Сменить».
Отдельная строка о том, актуальна ли таблица записей относительно содержимого каталога.
По макету m37 — панель «Готовность данных»: записи, аннотации, predict, датасеты, модель
с индикаторами по фактическому содержимому каталога (FR-2.16), не по «типу проекта».
3. Блок «Быстрые действия»: четыре кнопки с подписью команды orcAI под каждой.
4. Таблица «Последние задачи»: задача, объект, статус, длительность, время завершения.
5. Контекстная строка вверху справа (проект, модель, движок) — общий компонент,
он должен появиться на всех экранах приложения, а не только здесь.

Не делать: автозапуск движка по умолчанию; показ показателей, которых нет в данных проекта.
Готово, когда: на реальном проекте все четыре показателя и панель готовности заполнены из recording_table.csv,
файлов предсказаний и каталога models/, а не захардкожены.
```



Макет к П-1 · Главная

Макет к заданию П-1 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-1

Т-1 · Тест: Главная

после П–1
проверка критериев готовности · FR–1.1 – FR–1.7, FR–2.11, FR–2.16

Цель: подтвердить, что реализация П-1 соответствует ТЗ на реальных данных.

```
Протестируй раздел «Главная» после реализации П-1. Не дописывай функции — только проверка.

Сценарий:
1. Запусти sidecar+движок вручную; на Главной четыре строки среды показывают реальные версии/порт.
2. Открой реальный проект: показатели записи/аннотации/распознано/модель совпадают с диском;
панель готовности (m37) совпадает с артефактами каталога.
3. Нажми каждое быстрое действие — переход в нужный раздел с ожидаемым контекстом.
4. Перезапуск и остановка движка с Главной; автозапуск по умолчанию выключен.
5. Включи полноэкранный режим и масштаб 125%/150% (если уже есть в Настройках) — карточки не обрезаются.

Ожидаемый результат: строка «Готово, когда» из П-1 выполняется; нет статуса «готово» у компонента среды или задачи, если он/она не готовы.
Зафиксируй: список багов с шагом воспроизведения; скрин Главной 1280×800 и Full Screen.
```

П-2 · Проекты

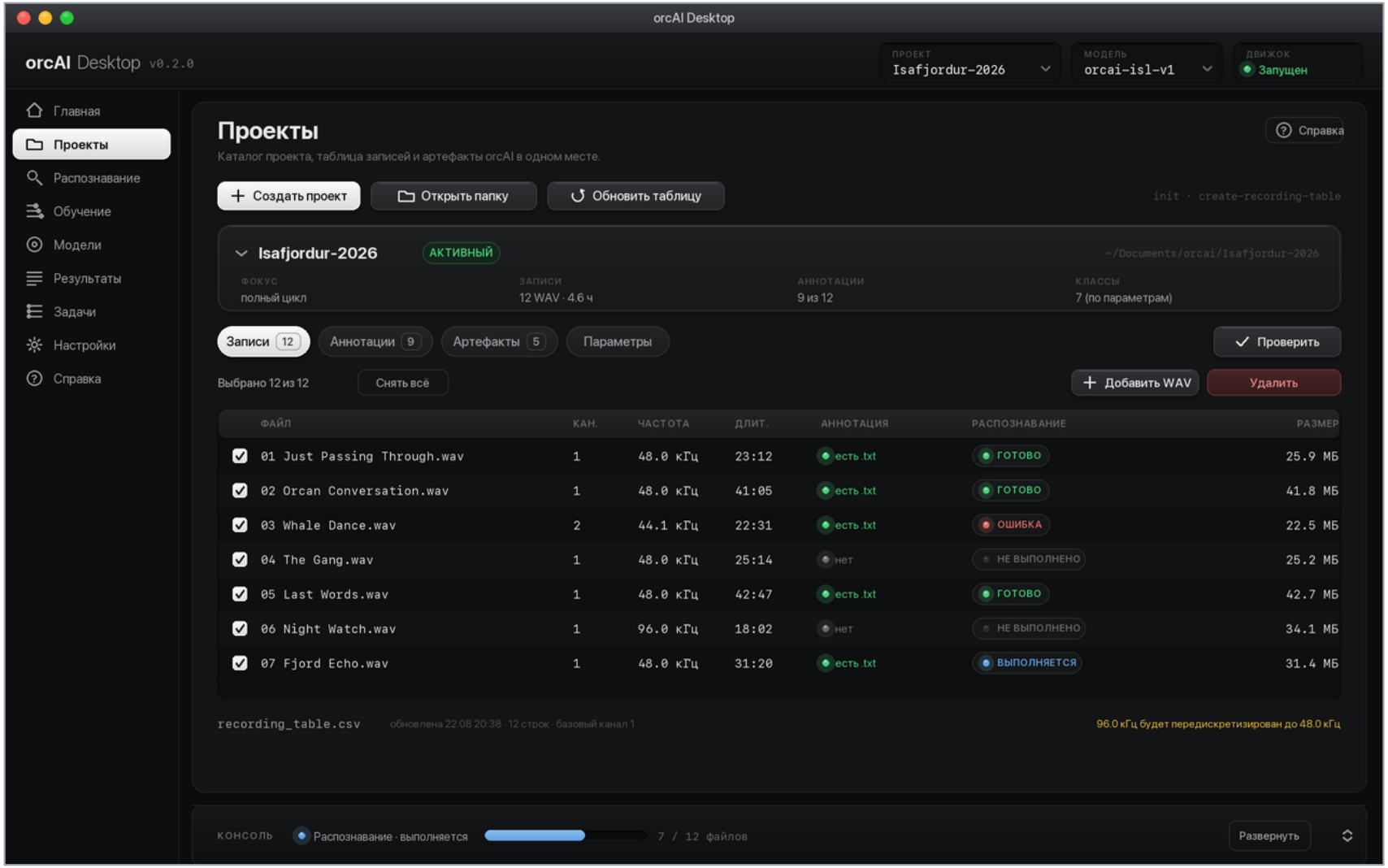
макет: m02-projects.png
FR-2.1 – FR-2.17 · UC-02 – UC-05, UC-17, UC-24

Цель: Показать проект как единый каталог orcAI: вкладки, мягкий фокус при создании, готовность данных

```
Переделай раздел «Проекты» по приложенным макетам.

Сделать:
1. Четыре вкладки: «Записи», «Аннотации», «Артефакты», «Параметры».
2. Мастер «Создать проект» (m32): три карточки мягкого фокуса — полевое распознавание,
обучение модели, полный цикл. Фокус задаёт подсказки и быстрые действия, но не блокирует
команды orcAI (FR-2.14, FR-2.17). Сохранять только в SQLite Desktop (FR-2.15), не в JSON orcAI.
3. Панель готовности данных в шапке проекта и на Главной (m36, m37): записи, аннотации,
predict, датасеты, модель — состояния «можно / нужно / нет данных» по артефактам на диске (FR-2.16).
4. Пустое состояние (m33): после init — подсказки «добавьте WAV → обновите таблицу».
5. Вкладка «Записи» — полная таблица recording_table.csv: recording, channel, duplicate,
base_dir_recording, rel_recording_path, base_dir_annotation, rel_annotation_path.
Плюс вычисляемые столбцы: длительность, частота, состояние распознавания.
Канал редактируется в строке. Записи 96 кГц помечаются: они будут приведены к 48 кГц.
6. Кнопка «Обновить таблицу» = create-recording-table -ut -up. Учесть: оригинал отказывается
перезаписывать существующий CSV, поэтому обновление всегда явное действие с подтверждением.
7. Вкладка «Аннотации»: столбцы классов из таблицы записей. Пустая ячейка означает
«класс замаскирован», а не «класса нет» — подписать это прямо в интерфейсе.
8. Вкладка «Артефакты»: спектрограммы, массивы меток, датасеты, модели, предсказания —
с числом файлов, занимаемым объёмом и кнопкой открытия каталога.
9. Вкладка «Параметры»: просмотр и редактирование <имя>_orcai_parameter.json с валидацией.
10. Полевой (m34) и лабораторный (m35) фокусы — разные empty states и акценты вкладок,
но оба пути (predict и train) остаются доступны с подсказкой, если не хватает данных.

Не делать: типа проекта «только predict» / «только train» с hard lock; полей фокуса в CSV или JSON orcAI;
переименования столбцов оригинала.
Готово, когда: проект из `orcai init` в терминале открывается полностью; фокус переключается;
готовность совпадает с файлами на диске.
```



Макет к П-2 · Проекты

Макет к заданию П-2 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-2

Т-2 · Тест: Проекты

после П–2
проверка критериев готовности · FR–2.1 – FR–2.17

Цель: подтвердить, что реализация П-2 соответствует ТЗ на реальных данных.

```
Протестируй «Проекты» после П-2 на проекте, созданном через `orcai init` в терминале.

Сценарий:
1. Создание с каждым из трёх фокусов — подсказки меняются, раздел «Обучение» не скрыт навсегда.
2. CLI-проект без записи в SQLite: открывается полностью; фокус = «полный цикл» или по артефактам.
3. Все четыре вкладки; CSV совпадает с UI (включая channel).
4. Готовность: после добавления WAV и аннотаций индикаторы обновляются без пересоздания проекта.
5. Пустая ячейка класса в аннотациях подписана как маскирование.
6. Столбец распознавания синхронен с predicted.txt на диске.
7. Регистр имён: два файла, отличающиеся только регистром, не схлопываются (подготовка к Linux).

Ожидаемый результат: критерий готовности П-2 выполнен; нет блокировки predict/train по фокусу.
Зафиксируй: скрины m33/m34/m35-состояний и панели готовности.
```

П-3 · Распознавание · Параметры

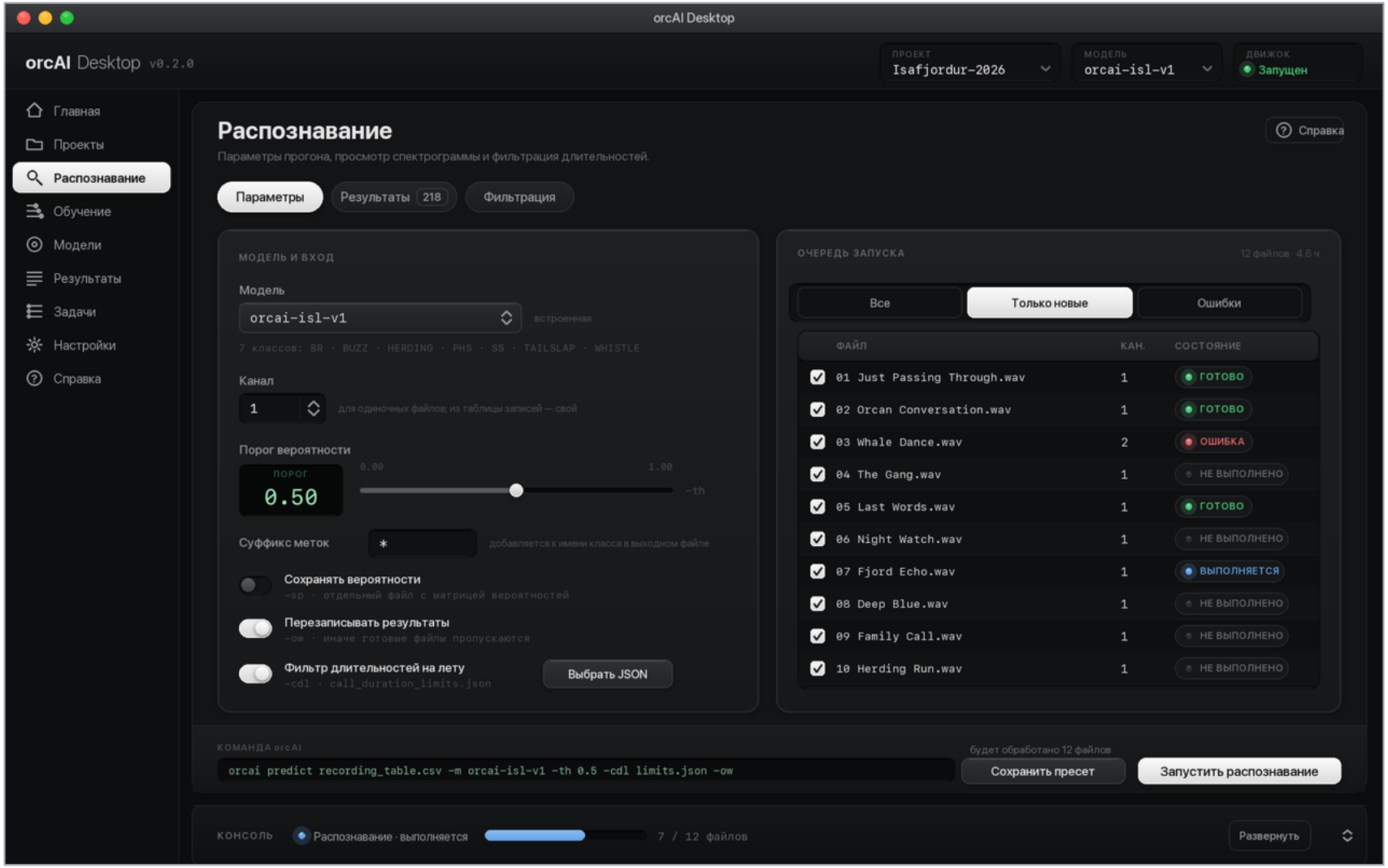
макет: m03-predict-params.png
FR-3.1 – FR-3.15 · UC-06, UC-07, UC-09

Цель: Дать полный набор параметров predict и предсказуемый запуск

```
Переделай вкладку «Параметры» раздела «Распознавание» по приложенному макету.

Сделать:
1. Две колонки: слева модель и параметры, справа очередь записей проекта с чекбоксами.
2. Полный набор параметров команды predict: модель (-m) или каталог модели (-md), канал (-c),
порог (-th, по умолчанию 0.5), суффикс меток (-ls, по умолчанию *), сохранение вероятностей (-sp),
каталог результата (-o), перезапись (-ow), пределы длительности (-cdl), подробность (-v).
У каждого поля подпись с коротким ключом CLI.
3. Очередь: фильтры «все / только новые / с ошибкой», счётчик выбранных, оценка времени.
4. Строка команды orcai внизу экрана, пересобираемая при любом изменении параметра,
с кнопкой копирования.
5. Сохранение набора параметров как пресета проекта.
6. Предупреждение: при пакетном запуске с указанием каталога вывода (-o) оригинал не добавляет
номер канала в имя файла. Если в очереди несколько строк одной и той же записи
(один WAV) с разными значениями channel — результаты в общем каталоге перезапишут друг друга;
показать это предупреждение до запуска.

Не делать: своих значений по умолчанию; скрытых параметров; режима «не сохранять результат»
(в коде 3.0.2 он приводит к ошибке, хотя упомянут в справке CLI).
Готово, когда: строка команды в интерфейсе совпадает с той, что реально выполнил sidecar.
```



Макет к П-3 · Распознавание · Параметры

Макет к заданию П-3 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-3

Т-3 · Тест: Распознавание · Параметры	после П–3 проверка критериев готовности · FR–3.1 – FR–3.15

Цель: подтвердить, что реализация П-3 соответствует ТЗ на реальных данных.

```
Протестируй вкладку «Параметры» распознавания после П-3.

Сценарий:
1. Измени каждый параметр — строка команды внизу пересобирается; скопированная строка запускается в sidecar без правок.
2. Пресет сохраняется и восстанавливается.
3. Пакетный -o: при одной записи с двумя каналами показывается предупреждение о коллизии имён.
4. Нет режима «не сохранять результат».
5. Очередь: фильтры все/новые/ошибки и оценка времени того же порядка, что фактический прогон (±50 % на выборке из 3 файлов).

Ожидаемый результат: команда UI ≡ команда в журнале задачи.
Зафиксируй: diff строки команды и скрин с предпросмотром.
```

П-4 · Распознавание · Результаты

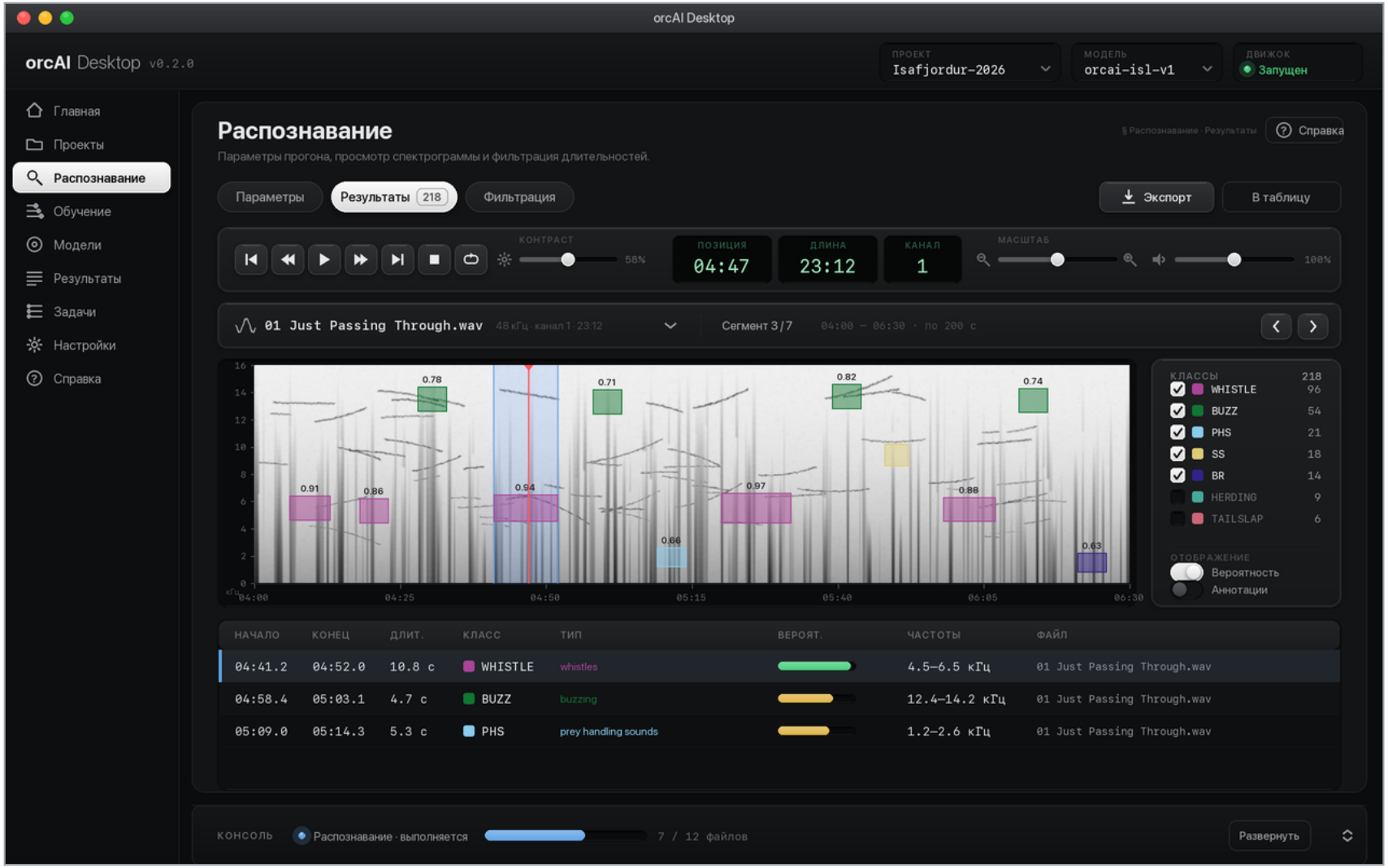
макет: m04-predict-studio.png
FR-4.1 — FR-4.10 · UC-08, UC-14

Цель: Из просмотрщика спектрограммы сделать рабочее место разбора

```
Переделай вкладку «Результаты» раздела «Распознавание» по приложенному макету.

Сделать:
1. Транспортная панель: перемотка, воспроизведение, цикл, контраст, ЖК-индикаторы позиции,
длительности и канала, масштаб, громкость.
2. Строка выбора записи и сегмента (сегменты по 200 с) со стрелками перехода.
3. Спектрограмма строится тем же кодом, что и вход модели: make_spectrogram, полоса 0–16 кГц,
квантили 0.01–0.999, серая шкала. Никаких альтернативных режимов отрисовки.
4. Легенда классов справа: цвет, код (WHISTLE…), счётчик детекций, чекбокс видимости.
Расшифровки типов (whistles, buzzing, …) — только в колонке «Тип» таблицы, теми же цветами.
Классы без детекций показаны, но приглушены.
Палитра — Paul Tol / orcAI: WHISTLE #AA4499, BUZZ #117733, PHS #88CCEE, SS #DDCC77,
BR #332288, HERDING #44AA99, TAILSLAP #CC6677.
5. Переключатели «Вероятность» (подпись mean_p на наложении) и «Аннотации»
(ручная разметка поверх распознанного — для сверки).
6. Таблица детекций под спектрограммой: начало, конец, длительность, класс (код),
тип (расшифровка), вероятность, диапазон частот, файл. Выделение строки и наложения синхронно в обе стороны.

Не делать: собственного расчёта спектрограммы; цветных шкал вместо серой.
Готово, когда: клик по строке таблицы прокручивает спектрограмму к нужному месту и наоборот.
```



Макет к П-4 · Распознавание · Результаты

Макет к заданию П-4 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-4

Т-4 · Тест: Распознавание · Результаты

после П–4
проверка критериев готовности · FR–4.1 – FR–4.10

Цель: подтвердить, что реализация П-4 соответствует ТЗ на реальных данных.

```
Протестируй студию «Результаты» распознавания после П-4.

Сценарий:
1. Спектрограмма серая 0–16 кГц; цвет только у оверлеев; источник — make_spectrogram / display_kind=orcai.
2. Клик по строке таблицы ↔ выделение на спектрограмме синхронны.
3. Легенда классов: скрытие класса убирает оверлеи; счётчики совпадают с таблицей сегмента;
в легенде — только коды классов; колонка «Тип» показывает whistles/buzzing/… теми же цветами.
4. Свёрнутая vs развёрнутая консоль: при развёрнутой студия остаётся работоспособной (скролл/свёртка таблицы).
5. Full Screen и 1920×1080: спектрограмма масштабируется, оси читаемы.

Ожидаемый результат: критерий П-4; нет альтернативных режимов отрисовки.
Зафиксируй: скрин сегмента с оверлеями и баги синхронизации.
```

П-5 · Распознавание · Фильтрация

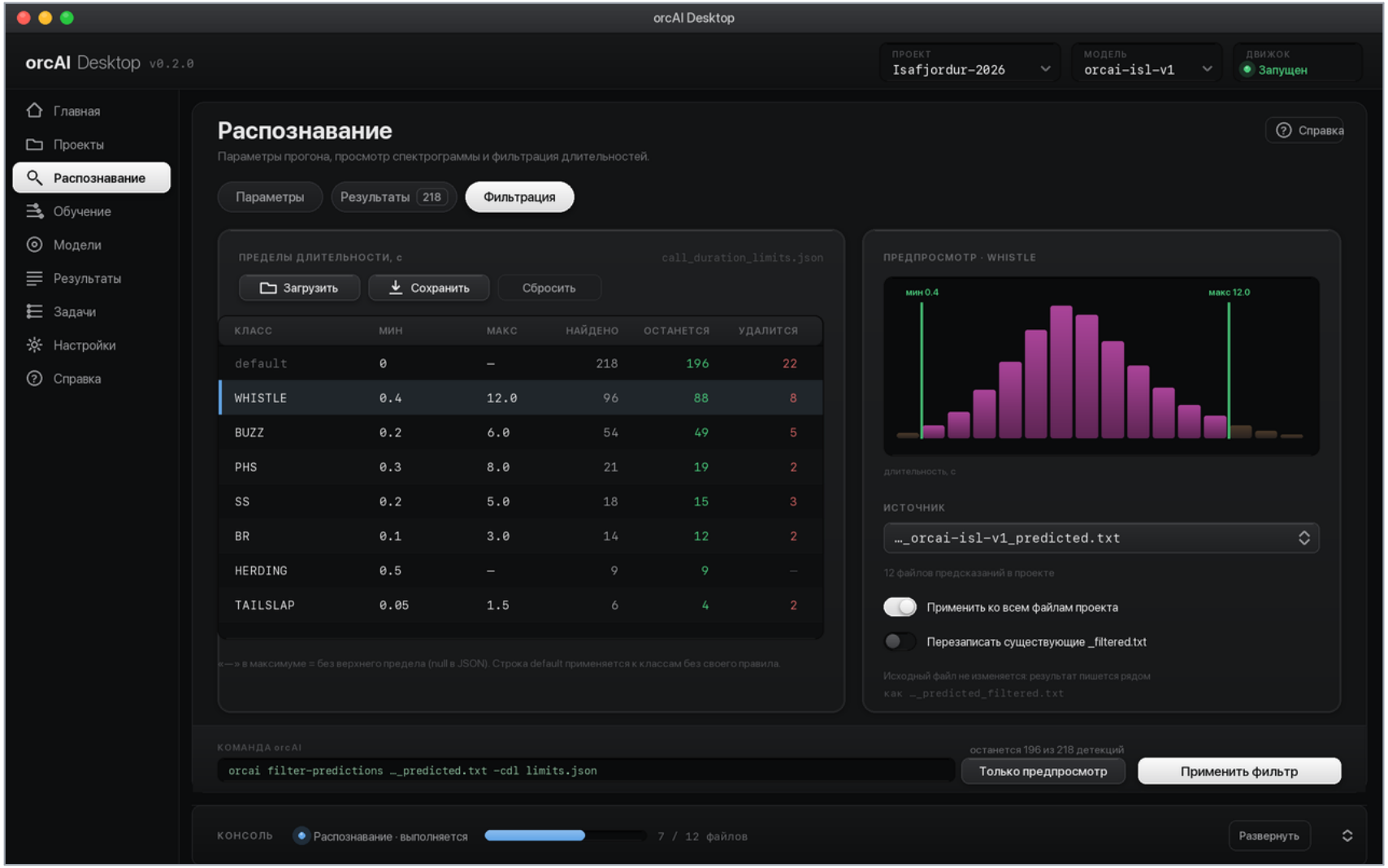
макет: m05-predict-filter.png
FR-5.1 — FR-5.8 · UC-10, UC-11

Цель: Сделать пределы длительности видимыми и проверяемыми до применения

```
Переделай вкладку «Фильтрация» раздела «Распознавание» по приложенному макету.

Сделать:
1. Редактор call_duration_limits: строка default плюс строка на каждый класс,
поля «мин» и «макс» в секундах, пустое значение = без ограничения.
2. Столбцы последствий на каждую строку: найдено, останется, удалится — пересчёт вживую.
3. Справа гистограмма длительностей выбранного класса с отметками текущих границ.
4. Выбор источника: конкретный файл предсказаний или все файлы проекта.
5. Кнопки «Загрузить» и «Сохранить» набора пределов в JSON — набор переносится между проектами.
6. Строка команды orcai filter-predictions и итог «останется N из M детекций».

Важно: исходный файл предсказаний не изменяется никогда. Результат пишется рядом
с суффиксом _filtered (имя вида …_predicted_filtered.txt или как формирует orcAI 3.0.2 —
сверить с фактическим output_file команды; в UI и Т3 кратко: «_filtered»).
Поля start/stop в *_predicted.txt и пределы в JSON — в секундах; не переводить из отсчётов.
Пустое поле «макс» = без верхней границы; пустое «мин» = без нижней границы (не подставлять 0 молча).
Готово, когда: предпросмотр совпадает с фактическим результатом запуска команды.
```



Макет к П-5 · Распознавание · Фильтрация

Макет к заданию П-5 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-5

Т-5 · Тест: Распознавание · Фильтрация

после П–5
проверка критериев готовности · FR–5.1 – FR–5.8

Цель: подтвердить, что реализация П-5 соответствует ТЗ на реальных данных.

```
Протестируй «Фильтрацию» после П-5.

Сценарий:
1. Предпросмотр «останется/удалится» совпадает с результатом filter-predictions.
2. Исходный predicted.txt не изменён; рядом появился *_filtered.
3. JSON пределов сохраняется/загружается между проектами.
4. Пустой мин/макс = без ограничения для класса.
5. Гистограмма отражает выбранный класс.

Ожидаемый результат: критерий П-5.
Зафиксируй: до/после счётчиков и пути файлов.
```

П-6 · Результаты

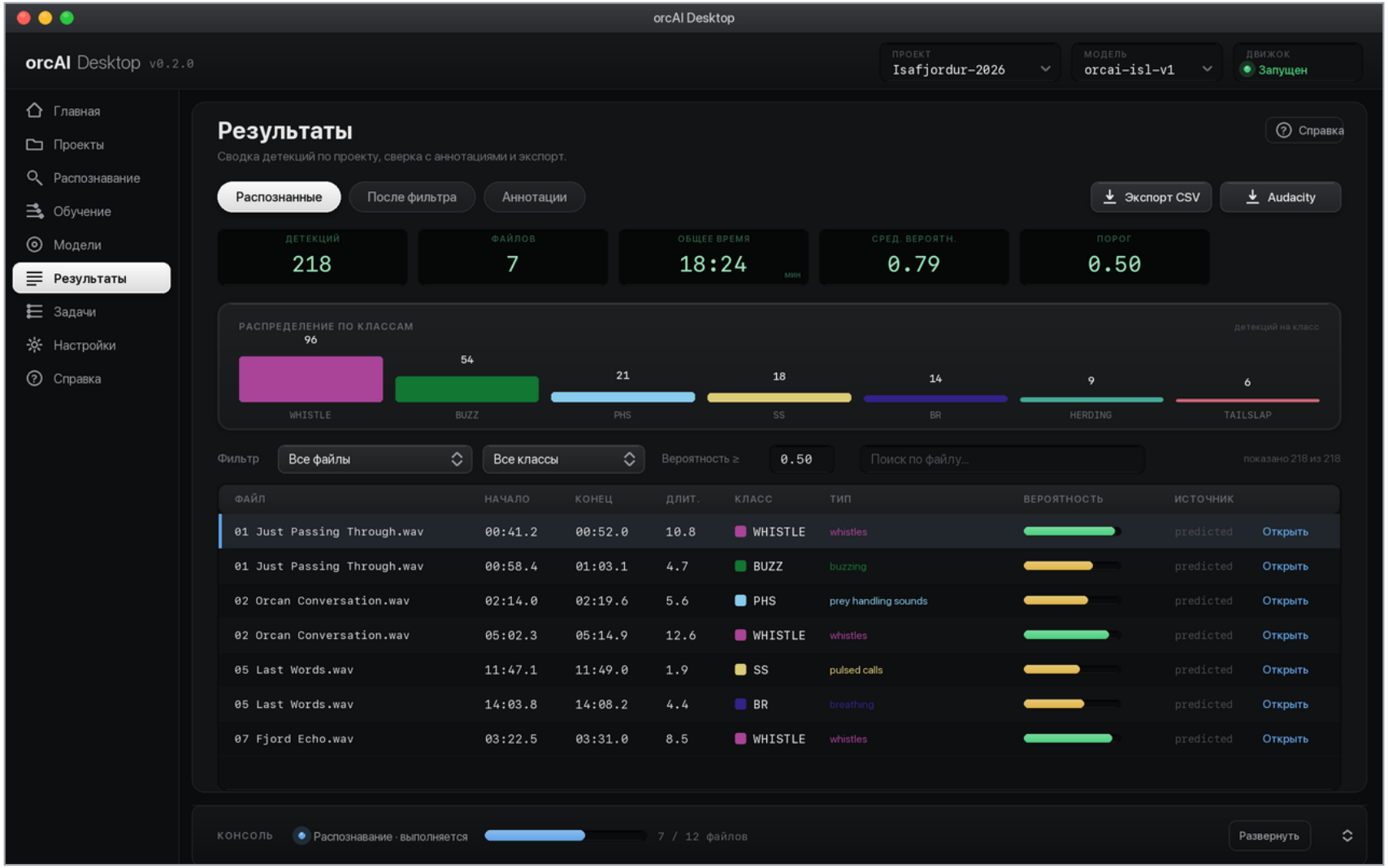
макет: m06-results.png
FR-6.1 – FR-6.6 · UC-12, UC-13

Цель: Дать сводный взгляд на весь проект и выгрузку без файлового менеджера

```
Переделай раздел «Результаты» по приложенному макету.

Сделать:
1. Полоса показателей: всего детекций, записей с детекциями, суммарная длительность,
средняя вероятность, использованная модель и порог.
2. Распределение по классам: горизонтальные полосы со счётчиками; под кодом — расшифровка
(whistles, buzzing, …) цветом класса.
3. Сводная таблица всех детекций проекта: столбцы «Класс» (код WHISTLE…) и «Тип» (расшифровка);
фильтры по записи, классу, вероятности и диапазону длительностей; сортировка по любому столбцу.
4. Переключатель источника: исходные предсказания или отфильтрованные.
5. Экспорт: CSV с текущими фильтрами и метки Audacity по записям.
6. Переход из строки таблицы к этому же месту во вкладке «Результаты» распознавания.

Не делать: пересчёта вероятностей и агрегатов, которых нет в файлах orcAI.
Готово, когда: экспортированный файл меток открывается в Audacity как дорожка.
```



Макет к П-6 · Результаты

Макет к заданию П-6 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-6

Т-6 · Тест: Результаты

после П–6
проверка критериев готовности · FR–6.1 – FR–6.6

Цель: подтвердить, что реализация П-6 соответствует ТЗ на реальных данных.

```
Протестируй раздел «Результаты» после П-6.

Сценарий:
1. Сводные показатели совпадают с суммой predicted/filtered файлов проекта.
2. Фильтры таблицы сужают выборку; экспорт CSV содержит только видимые строки.
3. Экспорт Audacity открывается как дорожка меток.
4. Переход из строки в студию распознавания попадает в тот же файл/время.
5. Переключатель исходные/отфильтрованные меняет источник данных.

Ожидаемый результат: критерий П-6.
Зафиксируй: файл экспорта и скрин сводки.
```

П-7 · Обучение · конвейер

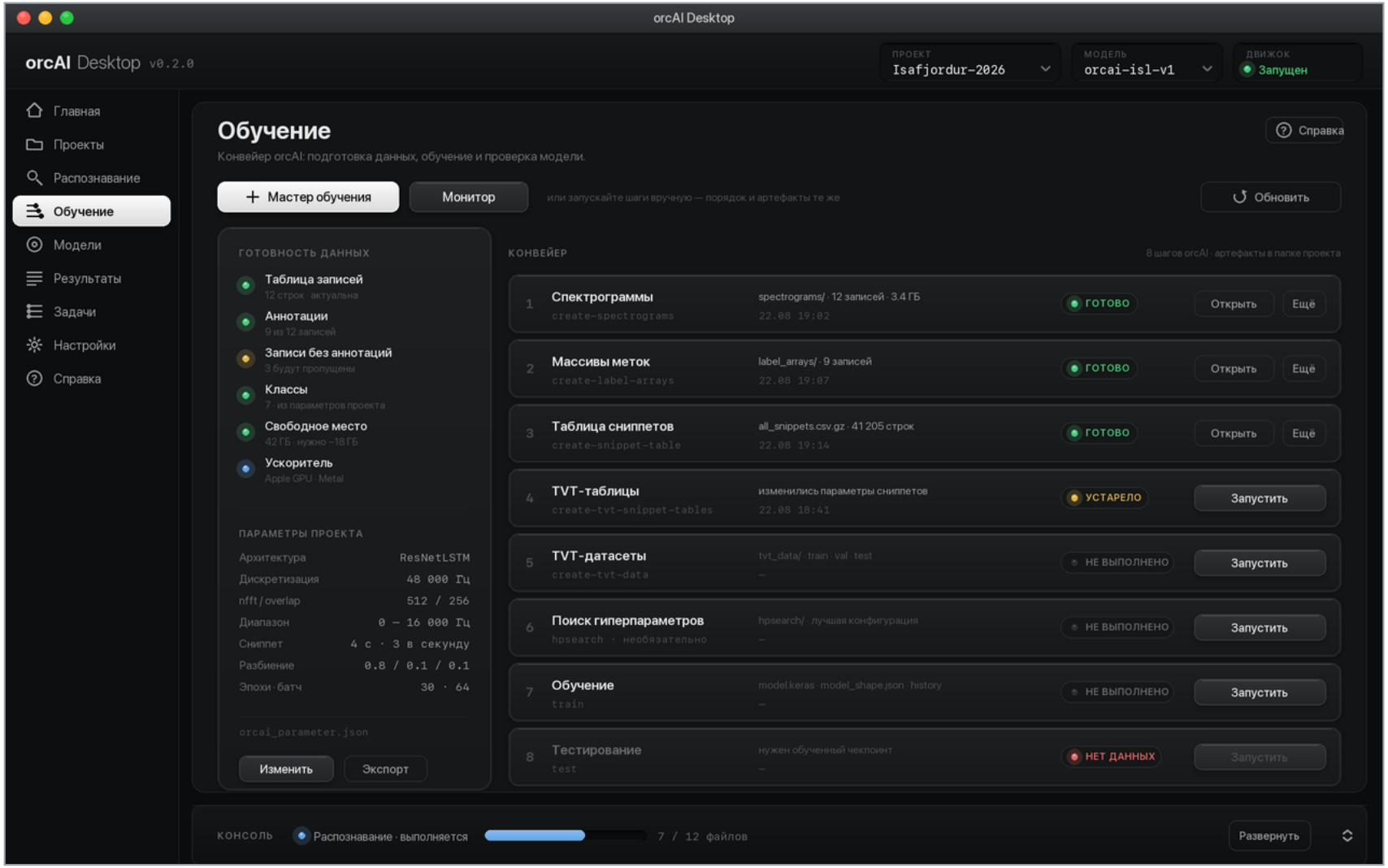
макет: m07-train-pipeline.png
FR-7.1 – FR-7.13 · UC-16, UC-18

Цель: Заменить список кнопок доской конвейера с достоверными состояниями

```
Переделай раздел «Обучение» по приложенному макету.

Сделать:
1. Доска из восьми шагов в порядке оригинала: create-spectrograms, create-label-arrays,
create-snippet-table, create-tvt-snippet-tables, create-tvt-data, hpsearch, train, test.
Шаг hpsearch опционален для плана «с нуля» (FR-8.1: 7 команд без hpsearch); полный UI сценария hpsearch — этап 0.6.
2. Шесть состояний шага — дословно как FR-7.2: не выполнено, выполняется, готово, устарело, нет данных, ошибка.
«Нет данных» = шаг нельзя запустить из-за отсутствия зависимости; в UI рядом всегда текст причины
(не голая неактивная кнопка). «Устарело» — когда изменились параметры, от которых зависит артефакт шага.
3. В карточке шага: команда, созданные артефакты, их объём и время создания.
4. Панель готовности данных: записей всего, с аннотациями, размеченных классов,
ожидаемое число сниппетов против требуемого (n_batch × batch_size).
5. Панель ключевых параметров проекта, влияющих на конвейер, с переходом к их редактированию.
6. Кнопка входа в мастер обучения.

Важно: состояния определяются по фактическому содержимому каталога проекта, а не по флагам
в настройках приложения. Слово «ГОТОВО» означает «выполнено», а не «можно запустить».
Готово, когда: удаление каталога артефакта извне немедленно меняет состояние шага в интерфейсе.
```



Макет к П-7 · Обучение · конвейер

Макет к заданию П-7 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-7

Т-7 · Тест: Обучение · конвейер	после П–7 проверка критериев готовности · FR–7.1 – FR–7.13

Цель: подтвердить, что реализация П-7 соответствует ТЗ на реальных данных.

```
Протестируй конвейер обучения после П-7.

Сценарий:
1. Удали каталог артефакта с диска — состояние шага меняется без перезапуска приложения.
2. «Устарел» появляется после смены влияющего параметра.
3. «Нет данных» всегда с причиной (FR-7.3).
4. Готовность данных: сниппеты vs n_batch×batch_size считаются верно.
5. Слово «ГОТОВО» = выполнено, не «можно запустить».

Ожидаемый результат: критерий П-7.
Зафиксируй: таблица шаг→состояние до/после удаления артефакта.
```

П-8 · Мастер обучения · шаги 1–5

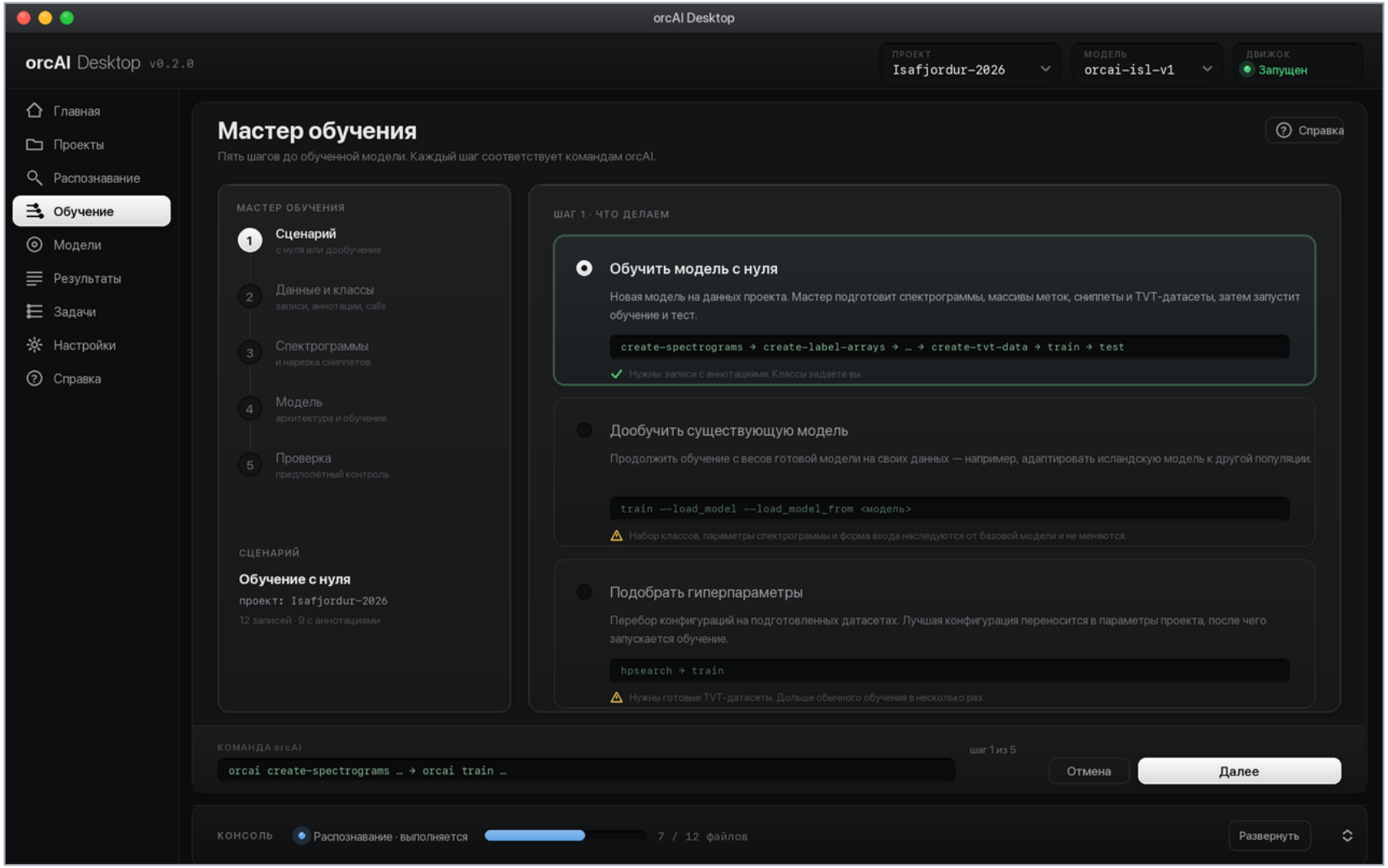
макет: m08-wizard-1-scenario.png
FR-8.1 – FR-8.6, FR-8.9 – FR-8.11 · UC-16, UC-18, UC-22

Цель: Собрать мастер обучения с нуля как оркестратор штатной цепочки команд

```
Создай мастер обучения по пяти приложенным макетам.

Шаг 1 «Сценарий»: обучение с нуля, дообучение, поиск гиперпараметров. У каждого варианта
показать цепочку команд и главное ограничение до выбора, а не после.
Шаг 2 «Данные и классы»: таблица записей, каталог аннотаций, файл синонимов меток,
выбор классов из фактически встреченных с числом примеров, покрытием и длительностью.
Метки вне списка классов игнорируются оригиналом — показать, сколько их отбрасывается.
Шаг 3 «Спектрограммы и сниппеты»: все поля разделов spectrogram и snippets с исходными
значениями, кнопка возврата к ним, оценка объёма и времени, ожидаемое число сниппетов.
Шаг 4 «Модель»: имя, архитектура (ResNetLSTM или ResNet1DConv), параметры сети и обучения,
отдельным блоком правила ранней остановки и снижения скорости обучения.
Шаг 5 «Проверка и запуск»: предполётные проверки, план из конкретных команд с оценкой
времени, кнопка «Только подготовка данных» и кнопка запуска всего плана.

Мастер не вводит ни одного параметра, которого нет в orcAI. Он пишет собранные значения
в <имя>_orcai_parameter.json и ставит команды в очередь «Задачи» в неизменном порядке.
Для сценария «с нуля» план = 7 команд (create-spectrograms → … → create-tvt-data → train → test),
без hpsearch. hpsearch попадает в план только если на шаге 1 выбран сценарий поиска гиперпараметров.
Готово, когда: модель обучается с нуля целиком из интерфейса, без обращения к терминалу.
```



Макет к П-8 · Мастер обучения · шаги 1–5

Макет к заданию П-8 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-8

Т-8 · Тест: Мастер обучения · шаги 1–5

после П–8
проверка критериев готовности · FR–8.1 – FR–8.6, FR–8.9 – FR–8.11

Цель: подтвердить, что реализация П-8 соответствует ТЗ на реальных данных.

```
Протестируй мастер обучения (с нуля) после П-8 на маленьком наборе данных.

Сценарий:
1. Пять шагов; ни одного параметра вне orcAI; запись в *_orcai_parameter.json корректна.
2. Предполёт блокирует запуск без аннотаций / при нехватке места.
3. План команд в очереди «Задачи» в порядке оригинала.
4. «Только подготовка данных» не запускает train.
5. Полный прогон до появления .keras (можно на урезанных эпохах для теста).

Ожидаемый результат: критерий П-8 без терминала.
Зафиксируй: список команд из вкладки «Команда» и путь к модели.
```

П-9 · Мастер обучения · дообучение

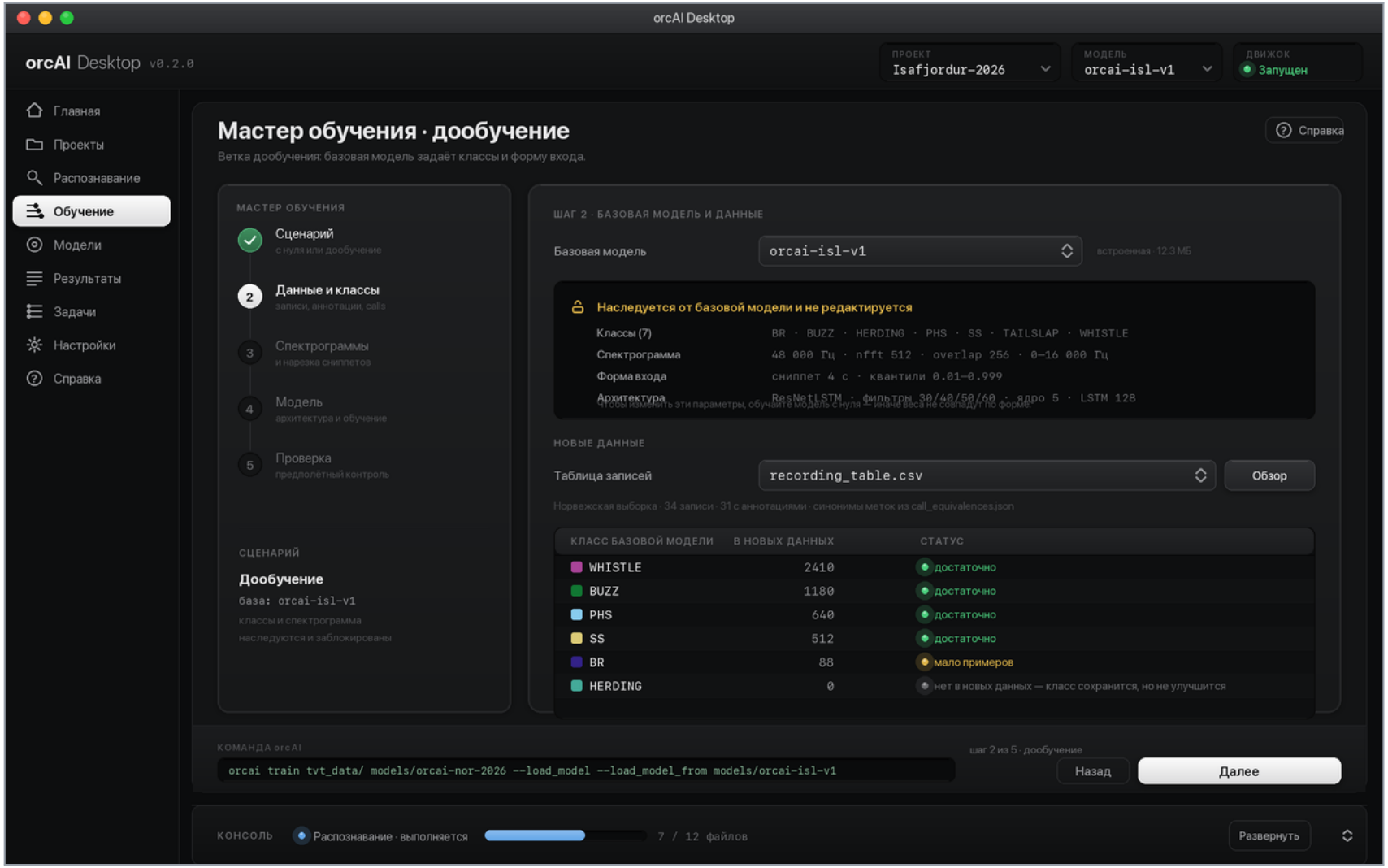
макет: m13-wizard-finetune.png
FR-8.7, FR-8.8 · UC-21

Цель: Ветка дообучения с наследованием параметров и сверкой классов

```
Добавь в мастер обучения ветку дообучения по приложенному макету.

Сделать:
1. Выбор базовой модели: встроенная или обученная в проекте.
2. Блок наследуемых параметров, недоступных для правки, с объяснением причины:
набор классов и их порядок, все параметры спектрограммы, длительность сниппета,
архитектура и её гиперпараметры. Форма входа и выхода сети зафиксирована при первом обучении.
3. Задаются заново: новые данные, имя новой модели, число эпох, скорость обучения,
правила остановки. Исходная модель не изменяется.
4. Сверка классов базовой модели с новыми аннотациями: у каких классов есть примеры,
у каких нет. Класс без примеров не исчезнет из модели, но и не улучшится — сказать это прямо.
5. Запуск: train с --load_model и --load_model_from, строка команды видна перед запуском.

Важно: сама orcAI эту совместимость не проверяет. При дообучении она берёт из каталога
модели только веса и архитектуру, а список классов и параметры спектрограммы читает
из переданного файла параметров. Несовпадение проявится сбоем формы тензора посреди
обучения. Проверка целиком на стороне приложения.
Готово, когда: попытка изменить набор классов при дообучении заблокирована с объяснением.
```



Макет к П-9 · Мастер обучения · дообучение

Макет к заданию П-9 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-9

Т-9 · Тест: Мастер обучения · дообучение

после П–9
проверка критериев готовности · FR–8.7, FR–8.8

Цель: подтвердить, что реализация П-9 соответствует ТЗ на реальных данных.

```
Протестируй ветку дообучения после П-9.

Сценарий:
1. Наследуемые параметры недоступны для правки.
2. Попытка сменить набор классов блокируется с объяснением.
3. Сверка классов показывает классы без новых примеров.
4. В команде есть --load_model / --load_model_from; исходная модель не перезаписана.
5. Несовместимая база (другие параметры спектрограммы) не стартует.

Ожидаемый результат: критерий П-9; INV-2 соблюдён на стороне приложения.
Зафиксируй: скрин блокировки и строку команды.
```

П-10 · Монитор обучения

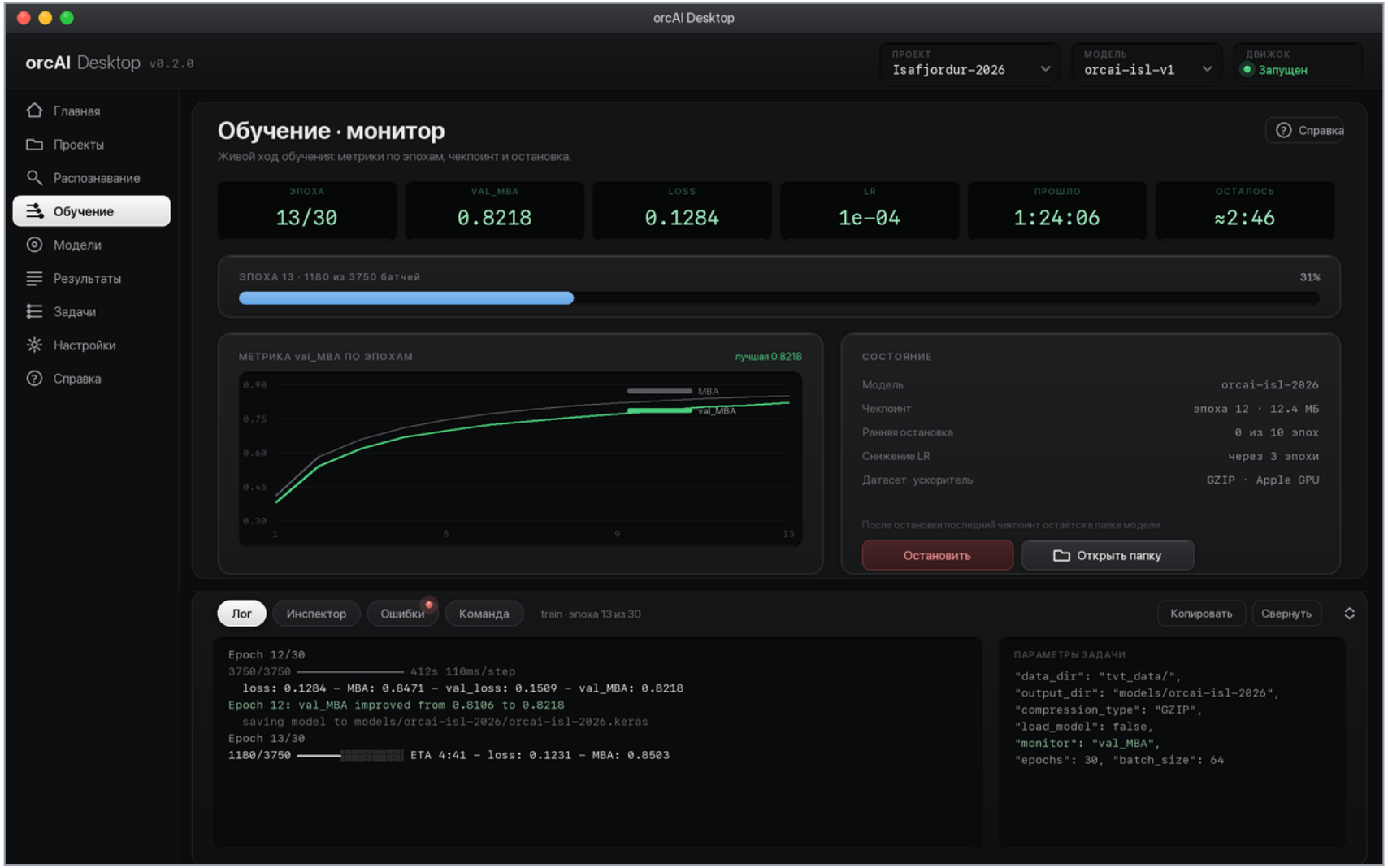
макет: m14-train-monitor.png
FR-8.12 – FR-8.14 · UC-19, UC-20

Цель: Наблюдение за многочасовым обучением без чтения сырого журнала

```
Сделай монитор обучения по приложенному макету.

Сделать:
1. ЖК-индикаторы: эпоха из максимума, отслеживаемая метрика (val_MBA), функция потерь,
текущая скорость обучения, прошедшее и оставшееся время.
2. Прогресс текущей эпохи по батчам.
3. График метрики по эпохам: кривая обучающей и проверочной выборок, отметка лучшего значения.
4. Панель состояния: модель, чекпоинт и его размер, сколько эпох осталось до срабатывания
ранней остановки, когда снизится скорость обучения, используемый ускоритель.
5. Кнопки «Остановить» и «Открыть папку». Остановка не теряет работу: веса сохраняются
при каждом улучшении метрики, продолжить можно через ветку дообучения.
6. Консоль внизу показывает настоящий вывод обучения, без пересказа.

Значения берутся из потока вывода train и из training_history.json, ничего не вычисляется заново.
Готово, когда: после принудительной остановки в каталоге модели остаётся валидный чекпоинт.
```



Макет к П-10 · Монитор обучения

Макет к заданию П-10 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-10

Т-10 · Тест: Монитор обучения	после П–10 проверка критериев готовности · FR–8.12 – FR–8.14

Цель: подтвердить, что реализация П-10 соответствует ТЗ на реальных данных.

```
Протестируй монитор обучения после П-10.

Сценарий:
1. ЖК и график читают training_history.json / stdout train, без своих формул.
2. Остановка сохраняет валидный чекпоинт; продолжение через дообучение возможно.
3. Ускоритель на панели совпадает с тем, что пишет TF в логе.
4. Консоль показывает сырой вывод.
5. Оценка оставшегося времени не «ломается» на первой эпохе.

Ожидаемый результат: критерий П-10.
Зафиксируй: путь чекпоинта после Stop и скрин монитора.
```

П-11 · Модели

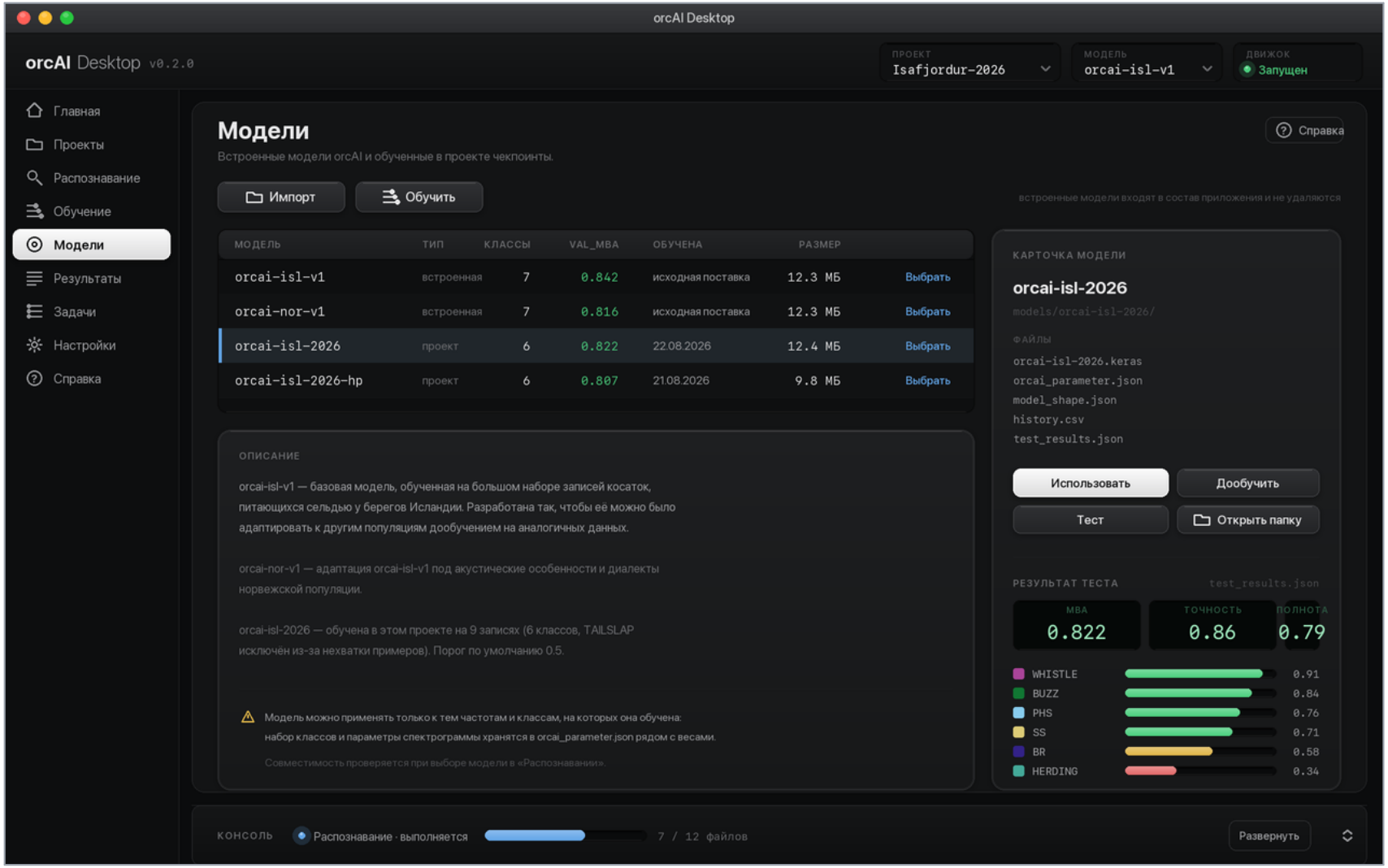
макет: m15-models.png
FR-9.1 – FR-9.8 · UC-23, UC-24

Цель: Карточка модели, отвечающая на вопрос «можно ли применить её к этим данным»

```
Переделай раздел «Модели» по приложенному макету.

Сделать:
1. Единый список: встроенные модели и модели, обученные в проекте, с датой и метрикой.
Пояснить происхождение встроенных: orcai-nor-v1 — это orcai-isl-v1, дообученная
на норвежских данных в течение тридцати эпох.
2. Карточка выбранной модели: состав каталога (<имя>.keras, orcai_parameter.json,
model_shape.json, training_history.json, подкаталог test), классы, параметры спектрограммы,
форма входа, размер.
3. Метрики последнего теста: общая метрика и разбивка по классам — точность, полнота, F1.
Данные читаются из test/<датасет>_metrics.json и <датасет>_confusion_table.csv.
4. Действия: использовать для распознавания, дообучить, протестировать, открыть каталог, импортировать.
5. Проверка совместимости модели с проектом по классам и параметрам спектрограммы
при назначении модели активной.

Не делать: удаления и правки встроенных моделей; пересчёта метрик своими силами.
Готово, когда: «Дообучить» из карточки открывает мастер сразу в режиме дообучения этой модели.
```



Макет к П-11 · Модели

Макет к заданию П-11 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-11

Т-11 · Тест: Модели

после П–11
проверка критериев готовности · FR–9.1 – FR–9.8

Цель: подтвердить, что реализация П-11 соответствует ТЗ на реальных данных.

```
Протестируй «Модели» после П-11.

Сценарий:
1. Встроенные isl/nor на месте; nor описан как дообучение isl.
2. Метрики теста читаются из test/*_metrics.json без пересчёта.
3. «Дообучить» открывает мастер в режиме этой модели.
4. Назначение несовместимой модели с проектом блокируется.
5. Импорт сторонней модели с корректным каталогом.

Ожидаемый результат: критерий П-11.
Зафиксируй: скрин карточки и результат проверки совместимости.
```

П-12 · Задачи и консоль

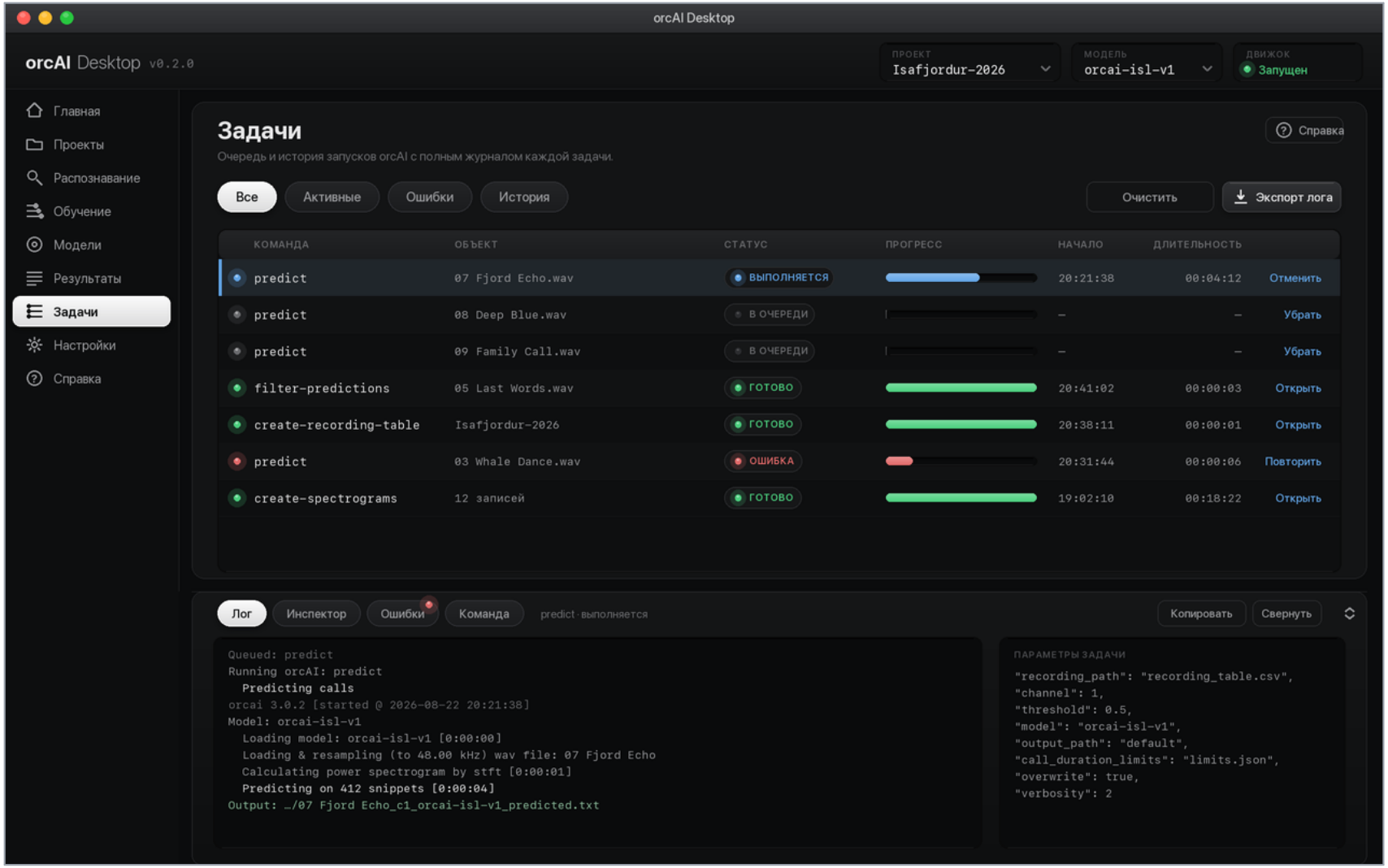
макет: m16-tasks.png
FR-10.1 – FR-10.9 · UC-15

Цель: Превратить журнал в управляемую очередь

```
Доработай раздел «Задачи» и нижнюю консоль по приложенному макету.

Сделать:
1. Очередь с состоянием, прогрессом по файлам, временем начала и длительностью.
2. Действия в строке: отменить выполняющуюся, убрать из очереди ожидающую,
повторить неудавшуюся с теми же параметрами, открыть результат.
3. Вкладки консоли: «Лог», «Инспектор», «Ошибки», «Команда».
«Ошибки» с индикатором количества и переходом к проблемной задаче.
«Команда» — точная строка orcai, пригодная для запуска в терминале.
4. Экспорт журнала задачи в файл.
5. Свёрнутая консоль показывает текущую задачу и прогресс, не отнимая места у содержимого.
6. История задач сохраняется между запусками приложения.

Важно: журнал показывается как есть, без пересказа. Отмена задачи не должна оставлять
частично записанные артефакты, помеченные как готовые.
Готово, когда: повтор задачи не трогает остальную очередь, а отмена корректно убирает недописанное.
```



Макет к П-12 · Задачи и консоль

Макет к заданию П-12 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-12

Т-12 · Тест: Задачи и консоль

после П–12
проверка критериев готовности · FR–10.1 – FR–10.9

Цель: подтвердить, что реализация П-12 соответствует ТЗ на реальных данных.

```
Протестируй «Задачи» и консоль после П-12.

Сценарий:
1. Отмена во время выполнения не оставляет «готовых» частичных артефактов.
2. Повтор ошибки не трогает чужие задачи в очереди.
3. Вкладки Лог/Инспектор/Ошибки/Команда; вкладка «Команда» допускает копирование строки в терминал.
4. История переживает перезапуск приложения.
5. Свёрнутая консоль показывает прогресс; развёрнутая — полный лог.

Ожидаемый результат: критерий П-12.
Зафиксируй: кейс отмены + экспорт журнала.
```

П-13a · Настройки · фундамент

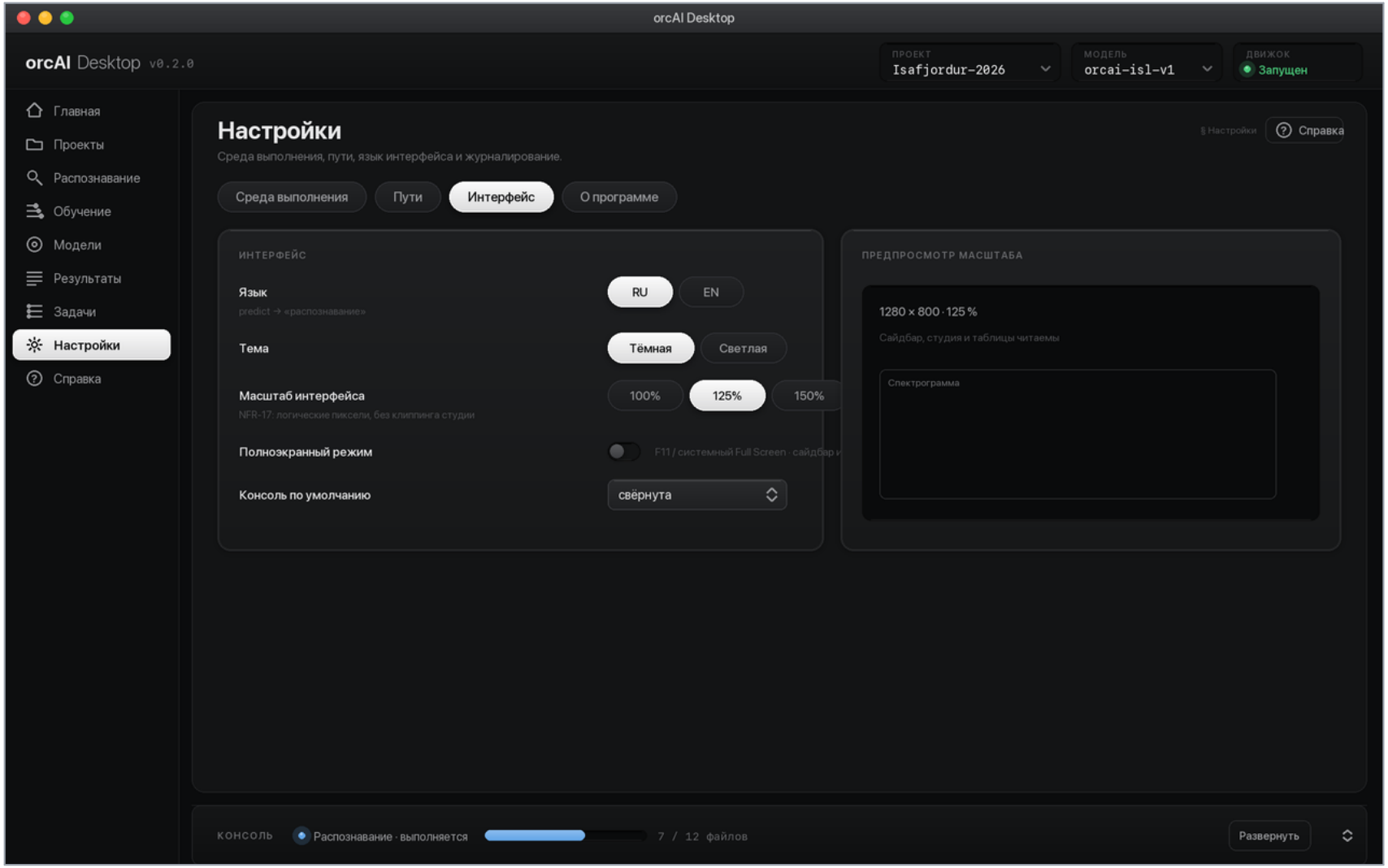
макет: m28-settings-interface.png
FR-11.1, FR-11.8, FR-11.9, NFR-17, NFR-18 · UC-01

Цель: Заложить сквозной слой настроек до предметных экранов: i18n, масштаб, persist

```
Этап ЭК-3a. Не делать вкладки «Среда выполнения», «Пути» и «О программе» — они в П-13 (ЭК-10).

Сделать:
1. Единый settings store с persist (Tauri/store): язык, тема, hide-on-close, масштаб UI.
2. Раздел «Настройки → Интерфейс» по m28: язык RU/EN без перезапуска и с запоминанием;
тема; поведение при закрытии окна (hide-on-close);
масштаб (системный / 100 / 125 / 150 %); Full Screen (F11 / системный — сайдбар и консоль доступны).
3. Обёртка i18n для всех последующих экранов: подписи через ключи, не литералы в JSX.
Под переключателем языка — подсказка: «Распознавание» = predict; полный глоссарий — в «Справке» (П-16).
4. Chrome hooks: масштаб и Full Screen работают на уже существующих экранах 0.1.8
(sidebar, консоль, текущие разделы).

Не делать: -v, порт sidecar, параллельность задач, автозапуск движка, вкладку «Пути» (m31),
панель Sidecar/движок (m17), «О программе» (m29 — вместе с П-16).
Готово, когда: RU↔EN на sidebar и chrome без сырых ключей; 150 % и Full Screen не ломают UI;
настройки переживают перезапуск; последующие ЭК (predict и далее) сразу используют i18n.
```



Макет к П-13а · Настройки · фундамент

Макет к заданию П-13а — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-13а

Т-13а · Тест: Настройки · фундамент	после П–13а проверка критериев готовности · FR–11.1, FR–11.8, FR–11.9, NFR–17, NFR–18

Цель: подтвердить, что реализация П-13а соответствует ТЗ на реальных данных.

```
Протестируй сквозной фундамент настроек после П-13а (ЭК-3а).

Сценарий:
1. RU↔EN на sidebar и на 2–3 уже существующих экранах 0.1.8: нет сырых ключей; кнопки не обрезаются.
2. Масштаб: системный, 100%, 125%, 150% — sidebar и консоль читаемы.
3. Full Screen: вход и выход; консоль разворачивается.
4. Перезапуск приложения: язык и масштаб сохранены.
5. hide-on-close работает независимо от языка.

Ожидаемый результат: критерии П-13а; NFR-17/18 на текущем UI 0.1.8.
Зафиксируй: скрин RU и EN одного экрана + матрица масштаб×FullScreen (pass/fail).
На ЭК-6/7 повтори пункт 1–3 на студии predict — регрессия i18n/DPI.
```

П-13 · Настройки · интеграция

макет: m17-settings.png
FR-11.2 — FR-11.5, FR-11.9 · UC-01

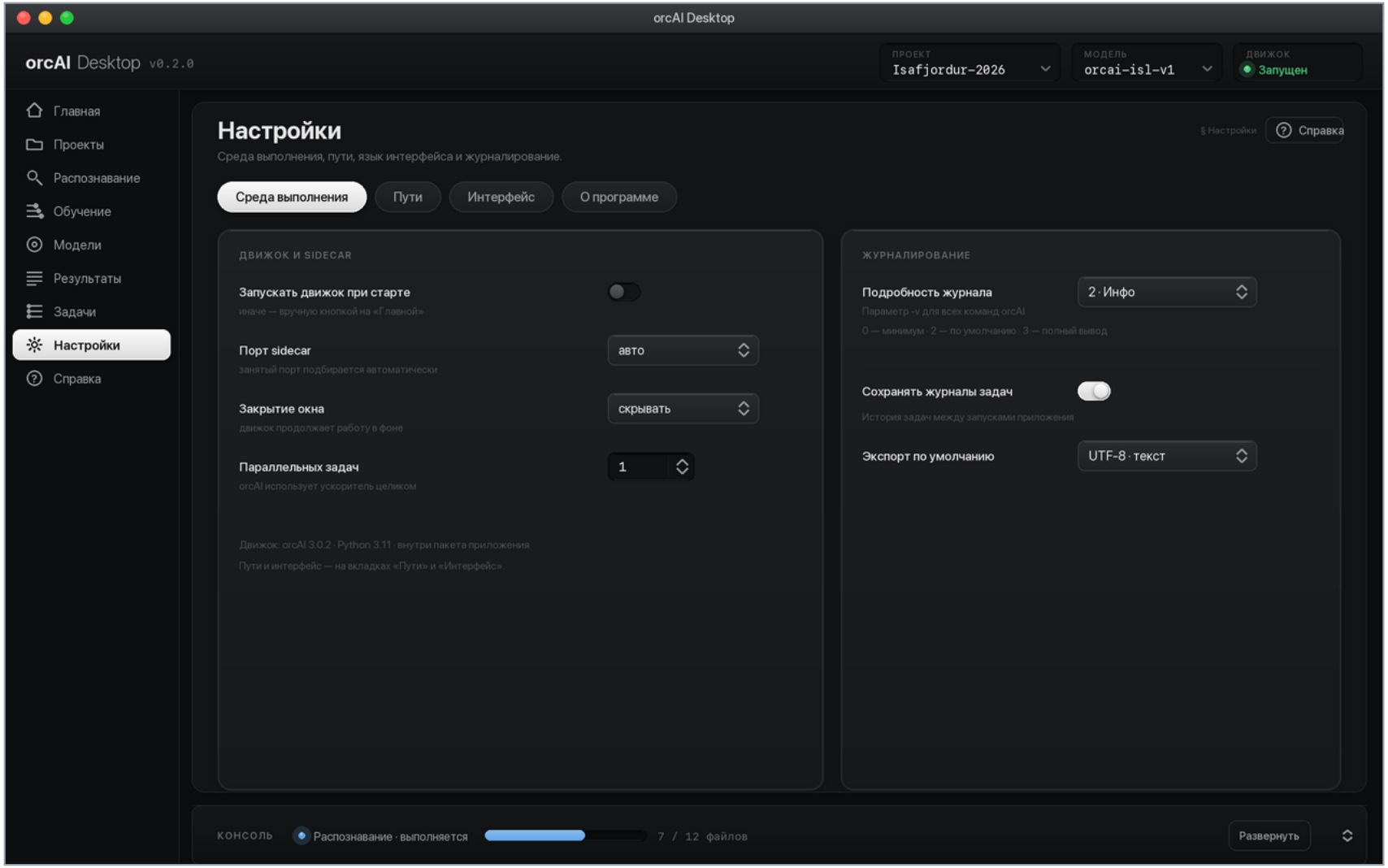
Цель: Довести «Настройки» до полного макета: среда выполнения, пути, связь с командами

```
Доработай раздел «Настройки» по макетам m17 и m31. Вкладка «Интерфейс» (m28) уже из П-13а — не дублировать.

Сделать:
1. Группа «Среда выполнения» (m17): подробность журнала (-v, 0–3, ко всем новым задачам),
порт sidecar, число параллельных задач (по умолчанию 1), автозапуск движка.
2. Группа «Пути» (m31): каталог проектов, кэш, журналы — «Обзор» и «Открыть» через API Tauri.
3. Внизу «Среда выполнения»: версия приложения, версия движка orcAI и способ поставки.

Экран m29-settings-about.png — один раз вместе с П-16 (кнопка «Подробнее» → «Справка → О Desktop»).

Правило перевода и адаптив — как в П-13а (NFR-17, NFR-18).
Готово, когда: -v из настроек попадает во все новые задачи; пути открываются;
панель среды совпадает с фактическим статусом sidecar/движка (как на Главной m01).
```



Макет к П-13 · Настройки · интеграция

Макет к заданию П-13 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-13

Т-13 · Тест: Настройки · интеграция	после П–13 проверка критериев готовности · FR–11.2 – FR–11.5, FR–11.9

Цель: подтвердить, что реализация П-13 соответствует ТЗ на реальных данных.

```
Протестируй интеграционную часть «Настроек» после П-13 (ЭК-10).

Сценарий:
1. «Среда выполнения»: -v меняется и попадает в следующую зада predict; параллельность = 1 по умолчанию.
2. «Пути»: «Обзор» и «Открыть» работают; пути без строковой склейки '/'.
3. Статус Sidecar/движка на m17 совпадает с Главной и фактическим runtime.
4. Регрессия П-13а: RU/EN, масштаб 150 %, Full Screen на predict-студии (m04) после ЭК-7.

Ожидаемый результат: критерии П-13; FR-11.2–11.5.
Зафиксируй: скрин m17 + строка команды с -v из консоли «Команда».
```

П-14 · Sidecar: контракт и очередь

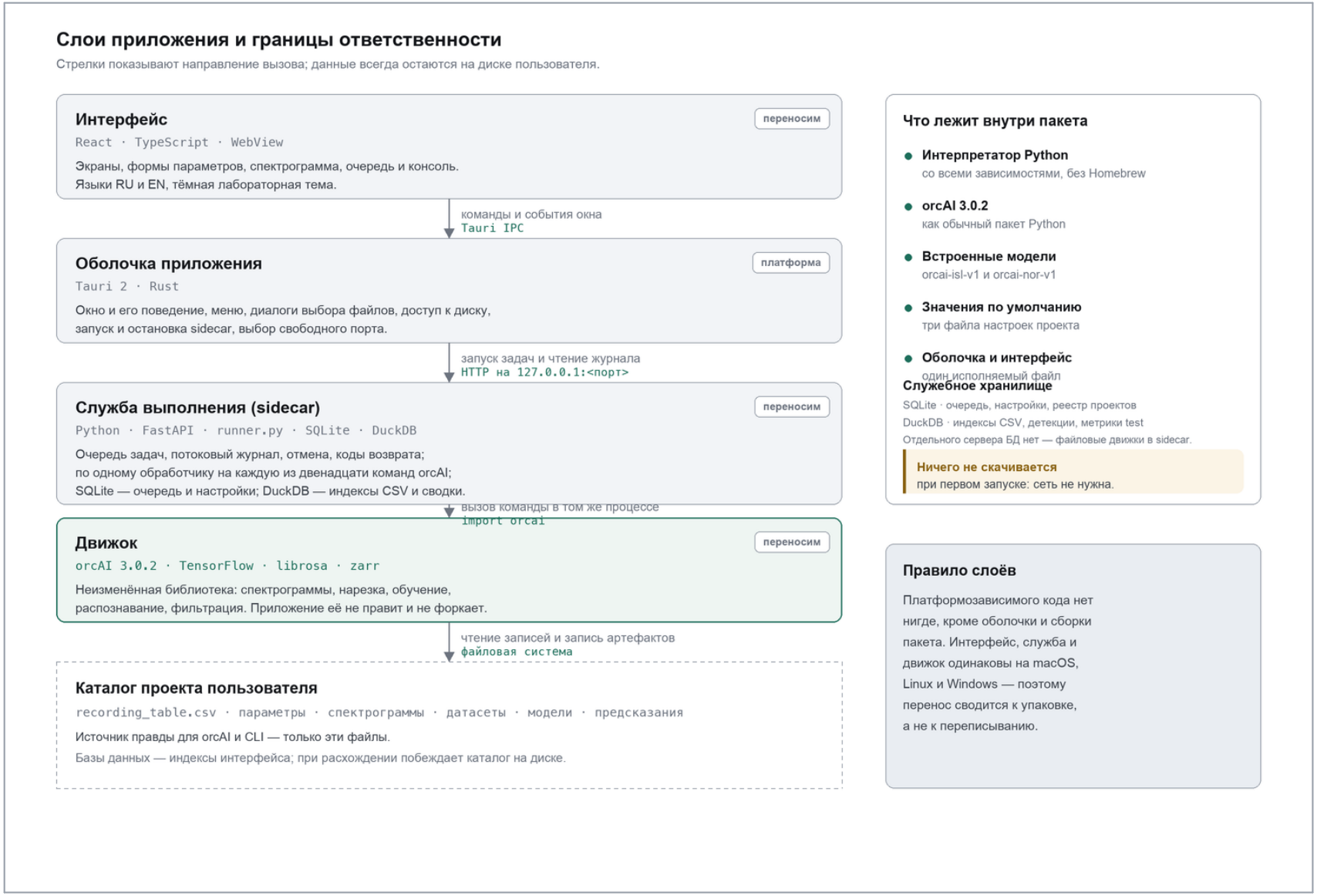
макет: d01-architecture.png
NFR-3 — NFR-8, FR-10.* · все

Цель: Привести службу выполнения в состояние, на которое можно опереться в UI

```
Доработай Python-sidecar (runner.py) как основу для всех разделов интерфейса.

Сделать:
1. По одному обработчику на каждую из двенадцати команд orcAI, с полным набором параметров.
Значения по умолчанию читаются из движка, а не дублируются в коде службы.
2. Очередь задач: постановка, отмена, повтор, приоритет; по умолчанию одна задача одновременно.
3. Потоковая отдача журнала (по строкам) и прогресса; переживает переподключение UI.
4. Возврат точной строки команды, которая была выполнена, — для вкладки «Команда».
5. Коды возврата и человеческий текст ошибки с сохранением исходного сообщения движка.
6. Отмена задачи убирает частично записанные артефакты или явно помечает их незавершёнными.
7. Ответы содержат абсолютные пути созданных файлов — интерфейс их только отображает.

Важно: служба не пересчитывает результаты orcAI и не переименовывает её файлы.
Готово, когда: любой экран приложения может показать, что выполняется, чем это закончилось
и какой командой было запущено.
```



Макет к П-14 · Sidecar: контракт и очередь

Макет к заданию П-14 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-14

T-14 · Тест: Sidecar: контракт и очередь

после П-14
проверка критериев готовности · NFR-3 — NFR-8, FR-10.*

Цель: подтвердить, что реализация П-14 соответствует ТЗ на реальных данных.

```
Протестируй контракт sidecar после П-14.

Сценарий:
1. Каждая из 12 команд вызывается через API с дефолтами движка.
2. Отмена create-tvt-data / train — артефакты не помечены готовыми.
3. Реконнект UI: поток лога продолжается.
4. Ответ содержит абсолютные пути созданных файлов.
5. Параллельность по умолчанию = 1.

Ожидаемый результат: критерий П-14.
Зафиксируй: набор curl/HTTP примеров и коды ошибок.
```

П-15 · Подготовка к переносу на Linux и Windows

макет: d02-porting.png
NFR-15, NFR-16 · все

Цель: Убрать платформенные допущения, пока их немного

```
Проведи ревизию кода на переносимость. Функциональность не меняется.

Сделать:
1. Убрать из кода интерфейса и службы все строковые пути с косой чертой;
каталоги пользователя запрашивать через API Tauri, пути собирать средствами платформы.
2. Сопоставление записей и аннотаций по имени файла выполнять с учётом регистра
(на Linux «Whale.wav» и «whale.wav» — разные файлы).
3. Убрать зависимость от шрифтов, которые есть только в macOS; задать запасные семейства.
4. Управление службой — только по HTTP; никаких сигналов процессам и абсолютных путей
к интерпретатору в коде интерфейса.
5. Экран среды показывает найденный ускоритель и способ его использования, каким бы он ни был,
без ветвлений «если macOS». Абстракция: Metal / CUDA / CPU / неизвестно.
6. Вынести всё платформозависимое в слой оболочки и в конфигурацию сборки.
7. Заложить API для Full Screen и DPI-scale в оболочке (реализация платформ — в П-17/П-18).

Не начинать релизные сборки Linux/Windows здесь — это П-17 и П-18.
Готово, когда: сборка проходит в контейнере Linux без единой правки в коде интерфейса и службы.
```



Макет к П-15 · Подготовка к переносу на Linux и Windows

Матрица переноса. П-15 — подготовка общего кода; П-17 Linux и П-18 Windows — отдельные поставки и ускорители.

Сразу после реализации — парный тест Т-15

Т-15 · Тест: Подготовка к переносу на Linux и Windows

после П-15
проверка критериев готовности · NFR-15, NFR-16

Цель: подтвердить, что реализация П-15 соответствует ТЗ на реальных данных.

```
Протестируй подготовку к переносу после П-15 (ещё без релиза Linux/Windows).

Сценарий:
1. Поиск в коде UI/sidecar: нет склейки путей через '/' строками; используются API путей.
2. Сборка/тесты в Linux-контейнере проходят без правок React/FastAPI (как заявлено).
3. Экран среды не ветвится if macOS для текста ускорителя.
4. Сопоставление WAV/аннотаций case-sensitive в логике таблицы.
5. Шрифты имеют fallback-стек.

Ожидаемый результат: критерий П-15.
Зафиксируй: список оставшихся platform-if и отчёт контейнерной сборки.
```

П-16 · Справка · авторы и лицензия

макет: m18-help-about.png
FR-11.6–FR-11.14 · все

Цель: Сделать «Справку» юридическим домом приложения: attribution, CC BY-SA 4.0, orcAI

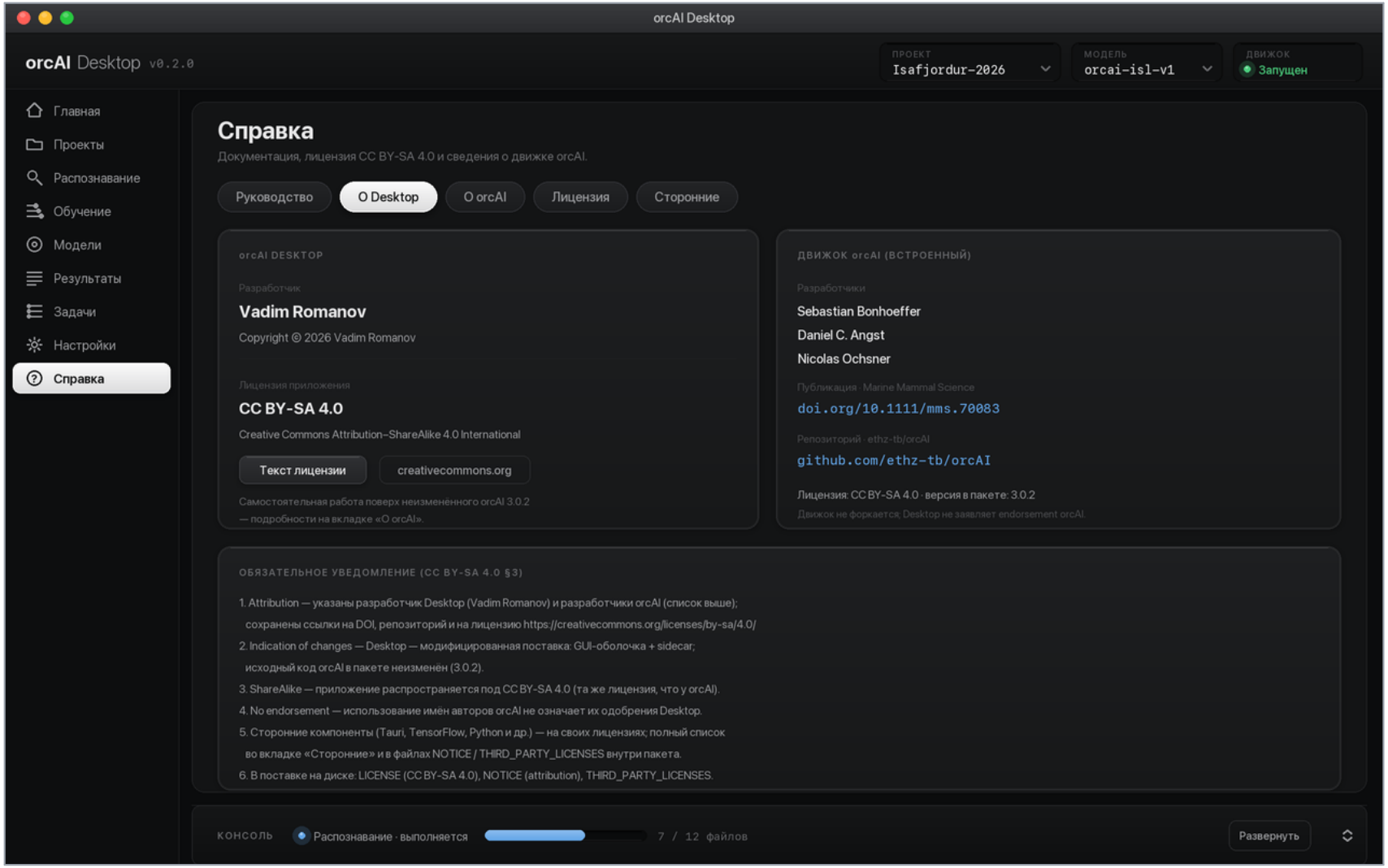
```
Доработай раздел «Справка» по приложенным макетам m18–m22, m19, m38.
Порядок пунктов сайдбара не менять: лицензия живёт здесь, а не отдельным пунктом меню.

Вкладки внутри раздела:
1. «Руководство» — полное руководство пользователя RU/EN (bundled docs/manual/ru|en/,
manifest.json → help_topics). Слева оглавление, справа чтение главы; m19 — landing,
m38 — пример главы «Распознавание · Результаты».
2. «О Desktop» — автор приложения: Vadim Romanov; Copyright © 2026 Vadim Romanov;
лицензия приложения CC BY-SA 4.0; кнопки «Текст лицензии» и ссылка на
https://creativecommons.org/licenses/by-sa/4.0/
3. «О orcAI» — встроенный движок 3.0.2 неизменён; авторы оригинала (полный список
из статьи): Sebastian Bonhoeffer, Anna Selbmann, Daniel C. Angst, Nicolas Ochsner,
Patrick J. O. Miller, Filipa I. P. Samarra, Chérine D. Baumgartner;
DOI https://doi.org/10.1111/mms.70083 ; GitHub https://github.com/ethz-tb/orcAI ;
лицензия движка CC BY-SA 4.0.
4. «Лицензия» — краткий чек-лист CC §3 (attribution, indication of changes, ShareAlike,
no endorsement) + содержимое файла NOTICE из поставки.
5. «Сторонние» — список лицензий зависимостей (или открытие THIRD_PARTY_LICENSES).

Кнопка «Справка» в chrome рабочих экранов (FR-11.13), **не** в разделе «Справка»:
– открывает «Справка → Руководство» на главе/якоре текущего раздела (см. manifest.json);
– на m04 рядом с кнопкой показан hint «§ Распознавание · Результаты»;
– на m17–m31, m28 — hint «§ Настройки»; на экранах m18–m22, m38 кнопки в chrome нет.

Также:
6. В пакете сборки обязательны файлы LICENSE, NOTICE, THIRD_PARTY_LICENSES (приложение Г ТЗ).
7. «Настройки → О программе»: одна строка «© Vadim Romanov · CC BY-SA 4.0 · orcAI 3.0.2»
и кнопка «Подробнее» → вкладка «О Desktop» в Справке.
8. Запрещено писать, что ETH / авторы orcAI «одобрили» или «совместно выпустили» Desktop,
без их письменного согласия (CC §2(a)(6) no endorsement).

Язык руководства и UI переключаются вместе (FR-11.1, FR-11.14); имена собственные,
DOI, CLI и имена классов не «переводятся».
Готово, когда: пользователь из Справки читает manual на выбранном языке; кнопка «Справка»
на любом экране ведёт в нужную главу; в .app лежат docs/manual/ и три файла NOTICE-комплекта.
```



Макет к П-16 · Справка · авторы и лицензия

Юридический блок Справки: автор Desktop, attribution orcAI, CC BY-SA 4.0 §3, файлы LICENSE/NOTICE в поставке.

Сразу после реализации — парный тест Т-16

Т-16 · Тест: Справка · авторы и лицензия

после П–16
проверка критериев готовности · FR–11.6–FR–11.14

Цель: подтвердить, что реализация П-16 соответствует ТЗ на реальных данных.

```
Протестируй «Справку» и комплект LICENSE/NOTICE после П-16.

Сценарий:
1. Вкладки: есть «О Desktop» с Vadim Romanov и CC BY-SA 4.0; «О orcAI» со всеми авторами статьи, DOI и GitHub.
2. Нет формулировок endorsement («одобрено ETH/авторами orcAI»).
3. Ссылка на creativecommons.org/licenses/by-sa/4.0/ открывается; файл LICENSE в пакете совпадает по смыслу.
4. В поставке на диске: LICENSE, NOTICE, THIRD_PARTY_LICENSES.
5. RU/EN переключают подписи, manual и UI, но не имена и DOI.
6. «Руководство»: оглавление (m19), чтение главы (m38); в .app bundled docs/manual/.
7. «Справка» со студии (m04) → глава «Распознавание · Результаты» (manifest.json).

Ожидаемый результат: FR-11.10–FR-11.14, приложение Г Т3.
Зафиксируй: скрин «О Desktop» + «Руководство» (глава) и список файлов в корне пакета.
```

П-17 · Портирование на Linux

макет: d02-porting.png
NFR-1, NFR-12, NFR-15 – NFR-18 · все

Цель: Автономная поставка Linux с честным GPU/CPU и тем же UX, что на macOS

```
Собери и доведи orcAI Desktop под Linux x86-64 (цель — Ubuntu LTS). Выполнять после П-15.

Поставка:
1. Автономный Python внутри пакета (без системного pip/Homebrew-аналогов).
2. Форматы: AppImage (запуск без установки) и .deb; контрольная сумма вместо нотаризации.
3. Включить в пакет нужные native-библиотеки чтения WAV / звука.
4. Следить за минимальной версией glibc; документировать поддерживаемые дистрибутивы.

Ускоритель обучения (критично):
5. При наличии NVIDIA + совместимых CUDA/cuDNN (в пакете или системе) — train/hpsearch на GPU;
экран «Среда» показывает «NVIDIA · CUDA» и имя устройства из TensorFlow.
6. Без GPU: обучение на CPU; до запуска train — предупреждение и оценка времени порядка
часов/суток, а не «как на Mac». Не маскировать CPU под «GPU».
7. Не обещать AMD ROCm, пока сборка TF в поставке её явно не включает.

Платформенные тонкости:
8. Регистр имён файлов значим: Whale.wav ≠ whale.wav в таблице записей.
9. Поведение окна: нет Dock; hide-on-close → трей или выход — настройка, согласованная с DE
(GNOME/KDE/другие); не копировать macOS-семантику слепо.
10. Full Screen и DPI (100–200 %) на X11 и Wayland: сайдбар, консоль, студия (NFR-17/18).
11. Пути только через API оболочки; длинные пути и не-ASCII в $HOME проверяются.

Не делать: форка orcAI; отдельных «linux-only» экранов UI.
Готово, когда: на чистой Ubuntu LTS без системного Python проходит UC распознавания и
хотя бы укороченный train; устройство в UI совпадает с логом TF.
```



Макет к П-17 · Портирование на Linux

Матрица переноса. П-15 — подготовка общего кода; П-17 Linux и П-18 Windows — отдельные поставки и ускорители.

Сразу после реализации — парный тест Т-17

Т-17 · Тест: Портирование на Linux

после П–17
проверка критериев готовности · NFR–1, NFR–12, NFR–15 – NFR–18

Цель: подтвердить, что реализация П-17 соответствует ТЗ на реальных данных.

```
Протестируй поставку Linux после П–17 на чистой Ubuntu LTS (VM).

Сценарий:
1. AppImage/.deb ставится без системного Python; predict на тестовом WAV успешен.
2. С NVIDIA: среда показывает CUDA/GPU; train использует GPU (видно в логе TF).
3. Без NVIDIA: честное «CPU», оценка времени увеличена, обучение не падает на старте.
4. Два файла Whale.wav / whale.wav – две строки таблицы.
5. Full Screen и масштаб 125%/150% на X11 и (если доступно) Wayland.

Ожидаемый результат: критерии П–17 и NFR–16 на Linux.
Зафиксируй: версии драйвера/CUDA, скрины среды, время 1 эпохи GPU vs CPU.
```

П-18 · Портирование на Windows

макет: d02-porting.png
NFR-1, NFR-12, NFR-15 — NFR-18 · все

*Цель: Установщик Windows с **DirectML** для GPU и честным CPU fallback*

```
Собери и доведи orcAI Desktop под Windows 10/11 x86-64. Выполнять после П-15 (ЭК-2).

Поставка:
1. Установщик MSI или NSIS; опционально portable zip.
2. Launcher .exe вместо символических ссылок на Python (symlink на Windows часто недоступен).
3. Встроенный Python и зависимости внутри каталога установки; PATH пользователя не обязателен.
4. Authenticode: без подписи описать поведение SmartScreen; с подписью — штатная установка.
5. Включить длинные пути (longPathAware / политика) — проекты orcAI легко превышают 260 символов.

Ускоритель (критично — без DirectML Windows-машина мало полезна для train):
6. Базовая поставка включает **DirectML** (tensorflow-directml-plugin или согласованный эквивалент
для TensorFlow в bundled orcAI). Экран среды: «DirectML · GPU» при успешной инициализации.
7. Если DirectML недоступен (старый GPU, нет DX12): predict на CPU допустим; перед train/hpsearch —
модальное предупреждение о длительности и явный выбор «продолжить на CPU».
8. Не показывать «Metal»/«CUDA» на Windows. Не ставить tensorflow-metal на Windows.
9. WSL2+CUDA — опциональный отдельный канал документации, не смешивать с базовым MSI.

Платформенные тонкости:
10. Разделитель путей, регистр (сравнение case-sensitive сохраняем в общем коде).
11. Закрытие окна: сворачивание в notification area (tray), если выбрано hide-on-close.
12. Per-Monitor DPI Awareness V2: 100/125/150/200 %; Full Screen; студия без клиппинга.
13. Защитник Windows / Controlled Folder Access: документировать разрешение записи в проекты.
14. Кодовые страницы: UTF-8 для путей проектов с кириллицей.

Не делать: форка orcAI; «тихого» GPU, которого нет; ломки .app-сборки macOS.
Готово, когда: на чистой Windows 10/11 MSI проходит predict; train на DirectML завершает ≥1 эпоху;
при отсутствии GPU — предупреждение до старта обучения; NFR-17/18 выполнены.
```



Макет к П-18 · Портирование на Windows

Матрица переноса. П-15 — подготовка общего кода; П-17 Linux и П-18 Windows — отдельные поставки и ускорители.

Сразу после реализации — парный тест Т-18

Т-18 · Тест: Портирование на Windows

после П-18
проверка критериев готовности · NFR-1, NFR-12, NFR-15 – NFR-18

Цель: подтвердить, что реализация П-18 соответствует ТЗ на реальных данных.

```
Протестируй поставку Windows после П-18 на чистой Windows 10/11.

Сценарий:
1. MSI/NSIS: установка, запуск без «missing python»; long path >260 символов в проекте.
2. DirectML: среда показывает GPU; train (1 эпоха) завершается на DirectML.
3. Без DirectML (VM/старый GPU): predict на CPU; предупреждение до train; 1 эпоха на CPU по явному согласию.
4. Hide-to-tray / закрытие окна по настройке; SmartScreen при наличии/отсутствии Authenticode описан.
5. Per-Monitor DPI 100/150/200 % и Full Screen: студия и таблицы без клиппинга.

Ожидаемый результат: критерии П-18 и NFR-12/16/17/18 на Windows.
Зафиксируй: DirectML yes/no, матрица DPI, лог устройства TF.
```

П-19 · Clean Code, стабильность и производительность

макет: d01-architecture.png
NFR-3, NFR-4, NFR-19, NFR-20 · все · **Этап Cursor:** ЭК-1 (сквозной)

*Цель: С **первого коммита** sidecar следует Clean Code/PEP 8; убрать риски зависаний и утечек без смены поведения orcAI*

```
Проведи целевой аудит и правку кода приложения (не бандла orcAI) по NFR-19 и NFR-20.
Этап ЭК-1 выполняется сразу после ЭК-0 (sidecar) и действует на каждом последующем этапе;
перед релизом ветки портов — повторный прогон Т-19.

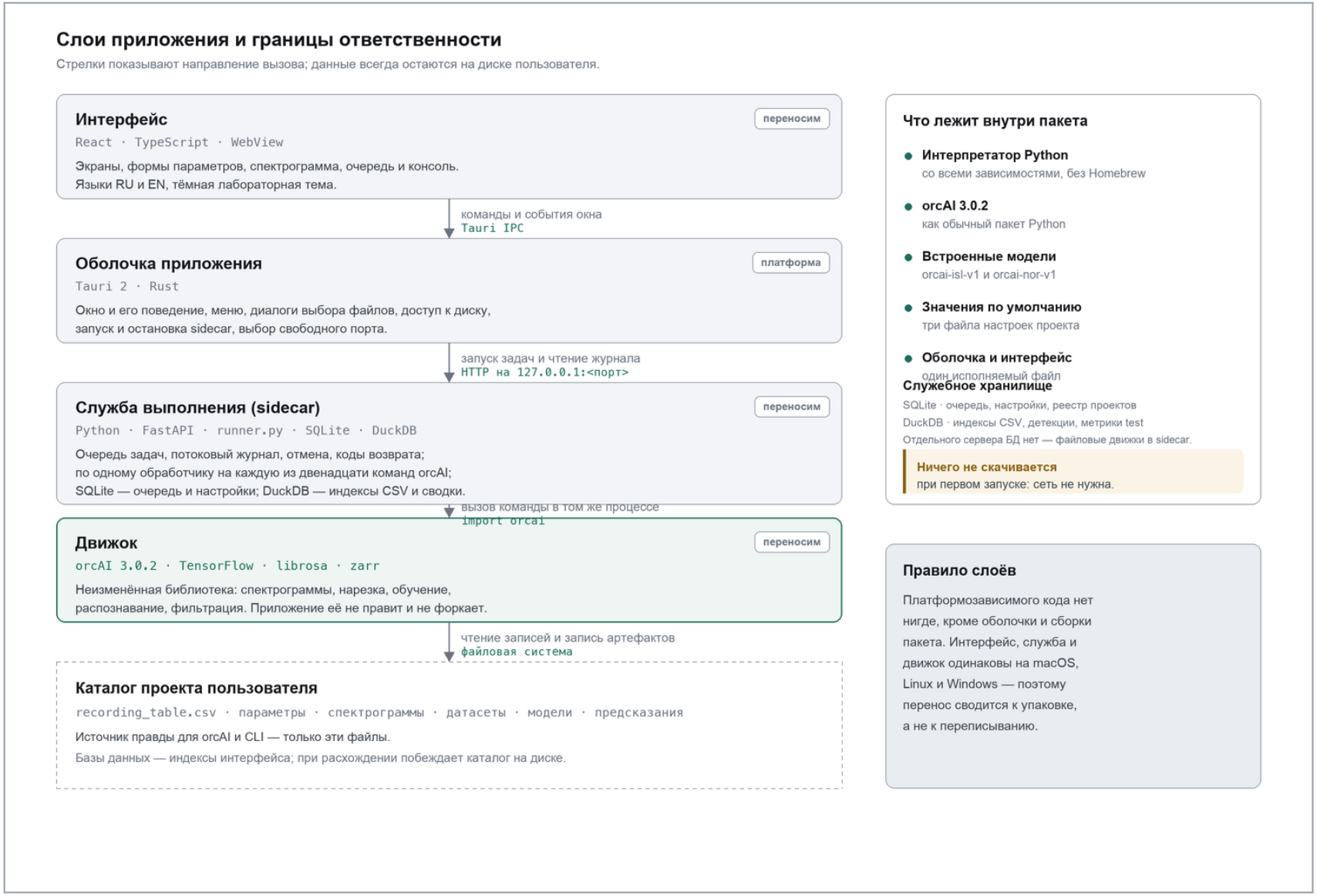
Границы работ:
1. В зоне правки: Python-sidecar (runner.py и связанные модули), очереди задач, обёртки CLI,
любые .py приложения вне каталога встроенной orcAI.
2. Вне зоны правки: исходники bundled orcAI 3.0.2; имена артефактов и семантика команд.
React/TS править только при доказанной утечке памяти или блокировке UI >200 мс;
запрещены изменения chrome, порядка сайдбара и плотности макетов m01–m38.

Clean Code и PEP 8 (NFR-19):
3. Включи и прогони ruff (предпочтительно) или flake8+isort по sidecar; устрани error-уровень.
4. Рефакторить функции длиннее 40 строк только если в них смешаны I/O, парсинг и контракт API;
иначе не дробить «для красоты». Одна функция — одна ответственность.
5. Имена: глаголы для действий, существительные для данных; без однобуквенных имён вне индексов.
6. Удали мёртвый код, закомментированные блоки «на всякий случай», дублирующие обёртки.
7. Явные ошибки: не глотай Exception молча; сохраняй исходный текст orcAI в журнале (NFR-8).
8. noqa только точечно и с комментарием «почему».

Стабильность и производительность (NFR-20):
9. Любая работа >100 мс — вне UI-потока (async/очередь sidecar); индикатор прогресса не реже 2 с (NFR-4).
10. Проверь отмену задач: нет «готовых» частичных артефактов (NFR-6); нет висящих процессов Python.
11. Спектрограмма/таблица: виртуализация или пагинация при большом числе строк; не пересоздавай
тяжёлые объекты на каждый кадр прокрутки.
12. Нет накопления setInterval/listeners при монтировании разделов; размонтирование очищает ресурсы.
13. Повторный запуск той же команды с теми же параметрами не должен менять контракт путей (NFR-7).

Запрещено:
– «Оптимизировать» алгоритм orcAI, пересчитывать спектрограмму своим кодом, менять дефолты CLI.
– Массовый рефакторинг «ради красоты» без проходящих Т-* на затронутых экранах.
– Отключать линтер глобально.

Готово, когда: ruff/flake8 по sidecar без error; краткий отчёт «было/стало» по 3 модулям;
ручной стресс 30+ мин (переключение разделов + одна длинная задача) без подвисаний UI >200 мс
и без монотонного роста памяти после окончания задачи.
```



Макет к П-19 · Clean Code, стабильность и производительность

Макет к заданию П-19 — целевой вид раздела. Меняется наполнение, не язык интерфейса.

Сразу после реализации — парный тест Т-19

Т-19 · Тест: Clean Code, стабильность и производительность

после П-19
проверка критериев готовности · NFR-3, NFR-4, NFR-19, NFR-20

Цель: подтвердить, что реализация П-19 соответствует ТЗ на реальных данных.

```
Протестируй результат П-19 (Clean Code / стабильность / производительность). Не расширяй рефакторинг.

Сценарий:
1. ruff или flake8(+isort) по sidecar: ноль ошибок уровня error; noqa только с комментарием.
2. Каталог bundled orcAI не изменён (git status / diff по бандлу пуст).
3. Дефолты CLI и имена артефактов совпадают с orcAI 3.0.2 на одной контрольной команде predict.
4. Стресс ≥30 мин: длинная задача + переключение разделов; нет подвисаний UI >200 мс; после завершения задачи RSS не растёт без нагрузки.
5. Отмена во время выполнения по-прежнему не оставляет «готовых» частичных артефактов (NFR-6).

Ожидаемый результат: NFR-19 и NFR-20; отчёт «было/стало» по трём модулям приложен.
Зафиксируй: вывод линтера, график/цифры памяти, список изменённых файлов вне orcAI.
```

Итог документа

orcAI Desktop 0.1.8 — работающая основа с правильной архитектурой. Документ задаёт 141 требований (121 FR + 20 NFR), 24 сценария, 40 PNG (m01–m38 + d01–d02), матрицу переноса и 20 пар заданий (реализация + тест), включая Linux/Windows (DirectML), Clean Code с первого коммита (П-19), и масштаб — без отступления от orcAI 3.0.2.

12. Этапы продолжения разработки в Cursor

Каждый этап — доработка одного раздела (или сквозного слоя). Порядок ЭК-0…ЭК-3а…ЭК-18: сначала sidecar, затем **Clean Code и подготовка к портам** (их правила действуют на каждом следующем этапе), затем **сквозной фундамент настроек (ЭК-3а)**, predict, остальные экраны, обучение и порты Linux/Windows.

Раздел 9 задаёт продуктовые версии 0.2–0.6. Раздел 11 даёт тексты промптов. Ниже — операционный план для Cursor: что открывать, в каком порядке делать разделы и что лежит в прилагаемой папке пакета.

12.1 Как запускать этап в Cursor

Шаг	Действие
1	Открыть папку пакета (см. ниже) в Cursor рядом с репозиторием orcai-desktop или скопировать её в docs/cursor-handoff/.
2	Первым сообщением сеанса вставить START_HERE.md (или «Общий контекст» из промты.md), приложить PDF ТЗ и нужный PNG из mockups/.
3	Выполнить ровно один этап ЭК-n: сначала промт реализации П-*, затем Т-*.
4	Не переходить к следующему ЭК, пока не выполнен критерий «Готово, когда» и отчёт по Т-* (pass/fail).
5	Версию приложения не бампать без явной просьбы; .app не ломать.

Правило одного этапа

В одном сеансе Cursor — один ЭК (одна пара П-n / Т-n, кроме ЭК-11 и ЭК-5, где несколько PNG прилагаются сразу). **ЭК-3а** — отдельный сеанс между ЭК-3 и ЭК-6. ЭК-1 и ЭК-2 — сквозные: их критерии проверяются на каждом последующем этапе.

12.2 Порядок этапов по разделам

Этап	Раздел приложения	Промты	Макет	Что сделать	Признак готовности
ЭК-0	**Sidecar и очередь**	`П–14 → Т–14`	`d01`	Опора для всех экранов: 12 команд, лог, отмена, пути артефактов.	Любой UI-раздел может показать задачу и команду.
ЭК-1	**Clean Code · с первого коммита**	`П–19 → Т–19`	`d01`	Ruff/PEP 8 sidecar, границы рефакторинга, отзывчивость UI — правило каждого этапа.	Линтер без error; новый код следует соглашениям (NFR-19).

Этап	Раздел приложения	Промты	Макет	Что сделать	Признак готовности
ЭК-2	**Подготовка к портам**	П–15 → Т–15	d02	Пути через API, регистр имён, абстракция ускорителя, без platform-кода в UI.	Сборка проходит в Linux-контейнере без правок интерфейса.
ЭК-3	**Задачи и консоль**	П–12 → Т–12	m16, m26, m27	Очередь; вкладки Лог/Инспектор/Ошибки/Команда; свёрнутая консоль.	Длительные задачи наблюдаемы без терминала.
ЭК-3а	**Настройки · фундамент**	П–13а → Т–13а	m28	i18n RU/EN, persist, масштаб, Full Screen, hide-on-close — до predict.	Следующие экраны сразу на ключах перевода; DPI не ломает chrome.
ЭК-4	**Главная**	П–1 → Т–1	m01, m37	Среда, карточка проекта, готовность данных, быстрые действия, контекст-чипы.	Стартовый экран запуска работы.
ЭК-5	**Проекты**	П–2 → Т–2	m02, m23–m25, m32–m36	Вкладки Записи/Аннотации/Артефакты/Параметры; мастер фокуса; готовность данных.	Проект читается целиком; фокус не блокирует CLI.
ЭК-6	**Распознавание · Параметры**	П–3 → Т–3	m03	Полный predict, очередь файлов, строка команды, пресеты.	Запуск распознавания без угадывания флагов.
ЭК-7	**Распознавание · Результаты**	П–4 → Т–4	m04	Студия: спектрограмма orcai, легенда, таблица сегмента, транспорт.	Разбор детекций на записи.
ЭК-8	**Распознавание · Фильтрация**	П–5 → Т–5	m05	Редактор call_duration_limits, предпросмотр, filter-predictions.	Фильтр длительностей с проверкой до применения.
ЭК-9	**Результаты (сводка)**	П–6 → Т–6	m06	Сводка проекта, фильтры, экспорт CSV и Audacity.	Выгрузка без файлового менеджера.
ЭК-10	**Настройки · интеграция**	П–13 → Т–13	m17, m31	Среда выполнения (-v, порт, параллельность), пути; m29 — с П-16.	-v и пути влияют на команды; статус sidecar достоверен.
ЭК-11	**Справка · руководство + лицензия**	П–16 → Т–16	m18–m22, m19, m38	Руководство RU/EN, deep link «Справка»; LICENSE/NOTICE/THIRD_PARTY.	CC BY-SA 4.0 + LICENSE/NOTICE/THIRD_PARTY.
ЭК-12	**Обучение · конвейер**	П–7 → Т–7	m07	Восемь шагов, шесть состояний, готовность данных.	Достоверный конвейер по диску проекта.
ЭК-13	**Мастер обучения**	П–8 → Т–8	m08–m12	Пять шагов с нуля, параметр-файл, очередь команд.	Обучение с нуля без терминала.
ЭК-14	**Монитор обучения**	П–10 → Т–10	m14	ЖК, график, ускоритель, остановка с чекпоинтом.	Наблюдение за train.
ЭК-15	**Модели**	П–11 → Т–11	m15	Карточка, метрики теста, совместимость, вход в дообучение.	Модель как объект проекта.
ЭК-16	**Дообучение**	П–9 → Т–9	m13	Наследование параметров, сверка классов, --load_model.	Ветка fine-tune без поломки формы сети.
ЭК-17	**Порт Linux**	П–17 → Т–17	d02	AppImage/.deb, CUDA/CPU, DPI/Wayland.	Чистая Ubuntu: predict + укороченный train.

Этап	Раздел приложения	Промты	Макет	Что сделать	Признак готовности
ЭК-18	**Порт Windows + DirectML**	`П-18 → Т-18`	`d02`	MSI, DirectML для GPU, long paths, DPI.	Win10/11: predict и train с ускорителем или честным CPU.

12.3 Соответствие продуктовой дорожной карте (§9)

Версия	Этапы Cursor	Смысл
0.2	`ЭК-0 … ЭК-3, **ЭК-3а**, ЭК-6, ЭК-7 (+ контекстная строка из П-1)`	Sidecar, Clean Code, подготовка к портам, консоль, i18n/DPI, predict, студия
0.3	`ЭК-8, ЭК-9`	Фильтрация и сводный раздел «Результаты»
0.4	`ЭК-4, ЭК-5, ЭК-10, ЭК-11`	Главная, проект-каталог, настройки (интеграция), Справка (5 вкладок)
0.5	`ЭК-12 … ЭК-15`	Конвейер, мастер, монитор, модели
0.6	`ЭК-16`	Дообучение (и hpsearch в рамках П-7/П-8 при необходимости)
ветка	`ЭК-17, ЭК-18`	Linux (CUDA/CPU) и Windows (DirectML + CPU fallback)
сквозно	`ЭК-1, ЭК-2`	Clean Code (NFR-19) и кросс-платформенные контракты (NFR-15) — с первого коммита

12.4 Прилагаемая папка для Cursor

Рядом с PDF поставляется каталог пакета. Его можно целиком перенести на основную машину разработки и указать Cursor'у как контекст этапа.

Путь	Содержимое
`/tmp/orcAI-Desktop-Cursor-пакет`	Полный пакет: README, START_HERE, этапы, промпты, mockups/, копия ТЗ PDF
`/Users/vadimromanov/.cursor/projects/empty-window/orcai-desktop/docs/cursor-handoff/`	Копия в репозитории (docs/cursor-handoff/) для работы из workspace
`mockups/`	Все PNG: m01–m38, d01–d02 (40 файлов)
`manual/`	Руководство RU/EN + manifest.json (help_topics для deep link «Справка»)
`промпты.md`	20 пар П-n / Т-n (включая П-13а / Т-13а) с общим контекстом
`этапы.md`	Таблица ЭК-0…ЭК-3а…ЭК-18 и порядок вставки в Cursor
`START_HERE.md`	Первое сообщение сеанса: роль агента, правила, ссылки на файлы пакета
`orcAI-Desktop-TZ-dorozhnaya-karta.pdf`	Полное ТЗ (этот документ)

Что сказать Cursor в первом сообщении

«Открой docs/cursor-handoff/START_HERE.md и этапы.md. Выполняем этап ЭК-N по промты.md: сначала П-…, макет из mockups/, затем Т-…. Обоснование — в PDF ТЗ, §3 / §7 / §12. Движок orcAI не форкать.» — и приложить PNG этапа.

Итог документа

Документ задаёт требования, сценарии, макеты, лицензию CC BY-SA 4.0 (Vadim Romanov + авторы orcAI), промты реализации и тестирования и операционные этапы ЭК-0…ЭК-3а…ЭК-18 для продолжения в Cursor. Пакет docs/cursor-handoff/ (и /tmp/orcAI-Desktop-Cursor-пакет/) — готовая передача контекста на основную машину разработки.